

MASTERING BEHAVIORAL INTERVIEWS

The Guide to
Storytelling in Tech



AUSTEN McDONALD

Mastering Behavioral Interviews: The Guide to Storytelling in Tech

by Austen McDonald

Copyright © 2025 Austen McDonald. All rights reserved.

austenmc@thebehavioral.tech

Cover Design: Jelena Mirkovic Jankovic, @boja99designs

Interior Design: the author

Editor: Bernice Martin Delcorde

Version: 1.0

Typeset in Minion Pro, Josefin Sans, Interstate, and
InterstateMono.

While the author has used good faith efforts to ensure that the information and instructions contained in this work are accurate, the author disclaims all responsibility for errors or omissions, including, without limitation, responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk.

ISBN 979-8-9932943-1-5

DEDICATION

For the builders,
The dreamers,
The leaders.
For the storytellers
Who doubt theirs is worth telling.
For my wife,
Who's changed me from the inside out.
For my children,
Liam and Brooke:
It's a privilege to be part of your story.

How to Use This Book

This book is presented from beginning to end, but you can take other paths.

Path 1: I'm new to behavioral interviews or Big Tech

- [Part 1: Fundamentals](#)
- [Part 2: Preparation Roadmap](#)
- [Key Takeaways](#)
- [Big Tech Behaviorals](#)
- [Appendix: Question Repository and Scripts](#)

Path 2: I'm experienced with behavioral interviews

- [Identifying Career Stories](#)
- [Scaffolding Stories](#)
- [The Big Three Questions](#)
- [Part 3: Refining Responses](#)
- [Common Pitfalls](#)
- [Appendix: Question Repository and Scripts](#)

Path 3: I'm in a hurry

- [What Interviewers Assess: Signal Areas](#)
- [Identifying Career Stories](#)
- [Why CARL is Better than STAR](#)
- [The Big Three Questions](#)
- [Big Tech Behaviorals](#)

- *Common Pitfalls*
- *Prepare Questions for the Interviewer*

CONTENTS

PART 0: INTRODUCTION

Get Started Here

What this Book Offers

Why You Need this Book

How to Use this Book

Path 1: I'm new to behavioral interviews or Big Tech

Path 2: I'm experienced with behavioral interviews

Path 3: I'm in a hurry.

Symbols used in this book

Key Takeaways

Common Pitfalls

Common Pitfalls for Senior Candidates and Leaders

PART 1: FUNDAMENTALS

How Behavioral Interviews Work

Why Companies Conduct Behavioral Interviews

What You Get Out of Behavioral Interviews

What Interviewers Look For

How Interviewers Assess You: Behavioral Questions

Your Role as a Candidate: Decode, Select, Deliver

Decode: Understanding the Question

The Basics: Job Description and Prep Materials

What Interviewers Assess: Signal Areas

Company Values: Signal Areas in Disguise

Select: Choosing a Response

How to Choose a Story.

Common Story Selection Challenges

What if I don't have a relevant story at all?

Should I lie or make up stories?

Should I repeat stories?

Choosing Responses to Values Questions

Choosing Responses to Hypothetical Questions

Deliver: Telling a Good Story.

[What Makes a Good \(Interview\) Story](#)
[STAR: A Simple Approach to Story Structure](#)
[Why CARL is Better than STAR](#)
[Story Construction Guidelines](#)
[Including Details](#)
[Examples of Stories](#)
[Telling Longer Stories](#)
[Adjusting Stories to Fit the Question](#)
[Responding to Values Questions](#)
[Responding to Hypothetical Questions](#)

PART 2: PREPARATION ROADMAP

Step 0: A Plan for Preparation

Step 1: Identifying Career Stories

Step 2: Extracting Story Detail

Step 3: Scaffolding Stories

Step 4: The Big Three Questions

[Tell Me About Yourself](#)

[Tell Me About Your Favorite Project](#)

[Tell Me About a Time You Resolved a Conflict](#)

Step 5: Prepare Questions for the Interviewer

Step 6: Practice

Step 7: What to do after an interview

PART 3: REFINING RESPONSES

Pitfalls by Signal Area

Improving Your Communication

[The Golden Rule: Guide the Interviewer](#)

[Adapt to Your Interviewer's Style](#)

[Tuning the Level of Detail](#)

[How Fast Should You Speak?](#)

Toward Better Stories

[Avoid Rigid Storytelling](#)

[Raise the Stakes Early](#)

[The Menu Technique: Letting Your Interviewer Choose](#)

[The Halo Effect: Getting Mileage Out of Lists](#)

Handling Sensitive Conversations

Questions About Weaknesses, Mistakes, and Regrets

Thinking Defensively

Handling Difficult Career Situations

Layoffs

Getting fired

Career break

Mental health breaks

Job hopping/short stints

Failed startup

Managing the Human Element

Connecting Authentically

Handling Difficult Interviewers

Confidence

PART 4: ADAPTING TO CONTEXTS

Behaviorals Throughout Your Career

New Graduates

Big Tech vs. Startups vs. Traditional Company

Big Tech Behaviorals

The Shift in Scale

The Shift in Culture: The Silicon Valley Mythos

Out of the Garage

The Lone Hacker

Move Fast and Break Things

Fail Fast

Embrace Conflict

Change the World

Being the Builder: Adapting your Responses to Big Tech

Specialized Interview Types

Calls with Recruiters

Screening Interviews

Leadership Interviews

Follow-Up Interviews

Deep Dives or Project Retrospectives

Cross-Functional or Partner Interviews

[Presentations](#)

[Hiring Manager Chats and Team Matching](#)

[A Note from the Author](#)

[Appendix: Question Repository and Scripts for Mock Interviews](#)

[Appendix: Questions to Ask the Interviewer](#)

[Appendix: Behavioral Interview Feedback](#)

[Appendix: How to Conduct Behavioral Interviews](#)

[Appendix: Leveling at Tech Companies](#)

[Acknowledgments](#)

[About the Author](#)

PART 0

INTRODUCTION

Behavioral interviews assess who you are and this book teaches you how to present yourself to tech companies.

GET STARTED HERE

The behavioral interview is a non-technical round where the interviewer, often the hiring manager, discusses your previous experience to assess your fitness for their open role. It's the culture fit interview, the soft skills interview, the "Tell me about a time when..." interview.

Behavioral interviewers are forecasters. They know that with people, unlike investments, *past performance is a predictor of future results*. Technical skills can be taught and the latest tips on managing up can be absorbed in an evening read, but who you are as a person—how you think, how you act, and how you interact—these emerge over years and change slowly.

As AI handles more implementation work, your ability to communicate your judgment, leadership, and problem-solving becomes the primary way companies distinguish senior from junior talent.

How to Succeed at the Behavioral

To present yourself as the right fit for the role, demonstrating both your capabilities and your alignment with the company's needs, you need three things:

- 1. An understanding of what interviewers actually evaluate.**
Beyond the questions they ask lies a structured framework (what we call *signal areas*) that predict job success. Decode what they're really assessing and you can target your responses strategically.
- 2. Clarity on your own professional story.** Your stories are better than you think, but you need to excavate the behaviors that made you successful, identify which experiences best demonstrate your capabilities, and understand how to translate them for the listener.
- 3. Compelling delivery.** Strong stories require structure and practice. You'll learn how to organize responses so interviewers extract the signal they need to confidently advocate for hiring you, using techniques that make your narratives both memorable and efficient.

What this Book Offers

This book is designed as a progressive guide for *builders*—software engineers, product managers, designers, and the rest of tech teams—to equip you to master behavioral interviews, with each section deepening your readiness:

- **Part 1: Fundamentals.** The mechanics of behavioral interviews, including understanding what interviewers value and how to leverage the *Decode, Select, Deliver* mental model to navigate any behavioral question.
- **Part 2: Preparation Roadmap.** A step-by-step preparation roadmap, from identifying your stories through constructing complete responses and practicing until they feel natural rather than rehearsed.
- **Part 3: Refining Responses.** Turning average stories into standout responses.
- **Part 4: Adapting to Contexts.** Advice for specialized interview types and how to approach different types of companies, including FAANG+. [\[1\]](#)

The content draws from proven approaches I’ve developed coaching over 200 candidates, forged through conducting over 1,000 Big Tech behavioral interviews, reviewing thousands of interview packets as a Meta hiring committee chair, training hundreds of interviewers, and managing 200+ engineers over two decades in Silicon Valley.

Why You Need this Book

Candidates tend to deprioritize behavioral interview preparation, especially among individual contributors. This is a mistake:

Behavioral interviews determine who gets the roles that matter most. Soft skills distinguish junior from senior employees, even among individual contributors. Seniority correlates with scope, and scope is about handling ambiguity—higher-level work involves more people, pressure, and stakes.

Behavioral interviews are more important in the age of AI. Now that anyone can build working solutions using everyday language, organizations have shifted their attention away from *how* to build and toward *what* to

build. Skills once reserved for product managers or leaders—understanding business context, driving alignment, communicating vision—are now essential for all builders. These soft skills are evaluated in the behavioral.

[\[2\]](#)

In a competitive job market, the behavioral is where you can stand out. The year I published this (2025), there were over 100,000 tech workers laid off. Your experience and your personality can really shine in a behavioral. Learn how to be memorable.

Common misconceptions about behavioral interviews

Let me dispel some common misconceptions:

- **No prep for behaviorals is required, just be yourself.** This isn't great dating advice and it's not great interview advice either. While behaviorals are about you and your experiences, understanding what interviewers actually want and crafting stories that communicate it effectively in a high-pressure setting requires preparation.
- **The best way to prepare is by prepping as many specific questions in advance.** You really don't know what you'll be asked. Better to understand what interviewers are looking for and know how to tell key stories from your career than prep for specific scenarios. With a few exceptions, these are not like coding interviews where questions are often repeated across companies.
- **All you need is the STAR method.** Search for behavioral interviews online and you'll find countless articles about STAR. Applying STAR puts you ahead of someone with no prep, but a simple story structure doesn't tell you which stories to choose, how to position yourself for a top-tier company, or how to adjust your approach for different contexts. [\[3\]](#)
- **The manager is just looking for social fit.** When a hiring team member conducts your behavioral, they do want to see if you'll mesh with the team. But that's not all—they're trying to predict how you'll perform in the role.

How to Use this Book

Different starting points require different approaches to preparation and you can use this book in many ways. Here are three common paths.

Path 1: I'm new to behavioral interviews or Big Tech

You'll benefit most from the complete journey through the book. Don't skip the foundational sections on the Decode, Select, Deliver loop in Part 1. Follow the step-by-step framework in Part 2 to prepare a story catalog you can use in your next interview and be confident in delivering those stories.

After you've had some practice, proceed to Part 3 to improve your storytelling and review [Big Tech Behaviorals \[→\]](#) if you plan to interview at a FAANG+.

Path 2: I'm experienced with behavioral interviews

If you already have a collection of stories and understand basic response structure, concentrate on elevating your approach. Be sure to go back and read any sections that reference unfamiliar concepts as you jump around.

1. Work through [Step 1: Identifying Career Stories](#), referencing [Decode: Understanding the Question](#) to ensure your story catalog covers the intended signals.
2. Ensure you have complete versions of your core stories in a condition to deliver by working through [Step 3: Scaffolding Stories](#).
3. Review your "Big Three" responses in [Step 4: The Big Three Questions](#) to ensure they meet expectations at your target level.
4. Read through the finer points of delivery in Part 3.
5. Study the [Common Pitfalls](#)—experienced candidates often have subtle issues that limit their effectiveness.
6. Practice the interview scripts in [Appendix: Question Repository and Scripts for Mock Interviews](#) to build fluency in matching stories to questions spontaneously.

Path 3: I'm in a hurry

If you have an interview scheduled in days rather than weeks:

1. Read [What Interviewers Assess: Signal Areas](#) to understand what interviewers prioritize in stories.

2. Quickly inventory your key projects and experiences using guidelines in [Step 1: Identifying Career Stories](#).
3. Learn basic response structure from [Why CARL is Better than STAR](#) and apply it to your top stories.
4. Prepare strong answers to the “Big Three” questions from [Step 4: The Big Three Questions](#) and practice them aloud.
5. Review [Common Pitfalls](#) to avoid obvious mistakes.
6. Prepare thoughtful questions for your interviewer using [Step 5: Prepare Questions for the Interviewer](#).
7. If interviewing at FAANG+, read through [Big Tech Behaviorals](#).

Symbols used in this book

Throughout the notes in this book you’ll find footnote links to outside resources and markers classifying the many example responses.

✓ Good Example

✗ Poor Example

KEY TAKEAWAYS

After spending the better part of a decade leading hiring committees at FAANG+ companies, training hundreds of engineers and managers on how to conduct behavioral interviews, and coaching over 200 candidates through this process, here are the key insights you'll take away from this book:

Actions over outcomes: The interviewer cares less about what happened and more about what you did to make it happen. Your role in driving results, not the results themselves, predicts your future performance.

Signal area alignment and understanding the “Question behind the Question”: Every behavioral question maps to specific competencies companies use to evaluate candidates. Learning to recognize and address these underlying assessments transforms meandering storytelling into strategic communication.

[4]

When an interviewer asks about an impactful project or about conflict resolution, they're not actually interested in the plot of the story—they're assessing your judgment, emotional intelligence, and relationship management skills. The plot supports these conclusions. Learning to decode the real assessment helps you give relevant, compelling responses.

The mythology matters: Tech companies operate on certain cultural assumptions about how work should happen, how people should interact, and what constitutes success. Understanding this mythology, whether you embrace it or not, helps you choose the right stories and use the right language to make a positive impact on interviewers.

[5]

Preparation enables authenticity: The goal isn't to act like somebody else, but to become more articulate about who you are already. Structured preparation helps you represent your authentic capabilities more effectively under pressure.

Storytelling as a professional skill: The ability to clearly articulate your contributions, lessons learned, and decision-making process isn't just for interviews—it's essential for career advancement, performance reviews, and leadership effectiveness.

[6]

Practice creates fluency: Like any complex skill, behavioral interviewing improves with deliberate practice. Recording yourself, working with others, and refining your stories builds the fluency that separates compelling candidates from merely qualified ones.

[\[7\]](#)

Common Pitfalls

These themes are repeated throughout the feedback I've given my coaching clients:

Not enough actions: “I built the feature” tells the interviewer almost nothing about your capabilities. They need to understand your specific contributions, decision-making process, and problem-solving approach. Junior candidates especially tend to summarize rather than detail their activities and motivations.

[\[8\]](#)

Inadequate preparation: Some candidates worry that preparation will make them sound scripted. In reality, poor preparation leads to nervous rambling and unclear responses that don't represent your capabilities accurately. Preparation enables you to be authentically yourself under pressure.

[\[9\]](#)

Missing the underlying assessment: When asked about “a time you disagreed with your manager,” candidates often focus on the disagreement details rather than demonstrating judgment, communication skills, and professional maturity—aspects of you the interviewer actually cares about.

[\[10\]](#)

Disorganized narratives: Rambling stories without clear structure leave interviewers confused about how your story informs their decision about you.

[\[11\]](#)

Unclear ownership: Constantly saying “we did this” or “[it] happened” instead of “I did this” makes it impossible for the interviewer to assess your individual contributions. They're hiring you, not your entire previous team. Leaders frequently fall into this trap.

Context overload: Spending five minutes explaining the technical architecture before mentioning a single action you took kills momentum and loses the interviewer's attention. Context should be minimal and focused—just enough for them to understand why your actions mattered. If you haven't shared a concrete action in a while, you're providing too much context.

[\[12\]](#)

Picking the wrong stories: Using junior-level examples for senior roles immediately signals you don't understand the target position's expectations. Your stories should match the scope they expect you to handle.

[\[14\]](#)

Single-dimension storytelling: Candidates who only discuss technical challenges while ignoring leadership, collaboration, and strategic thinking miss opportunities to demonstrate the full scope of capabilities needed for senior roles. Leaders who only cover business context seem underqualified to handle the technical, people, and process parts.

[\[15\]](#)

Fairy tale endings: Real projects have tradeoffs, unexpected challenges, and imperfect outcomes. Stories where everything goes perfectly and everyone is happy sound either fabricated or demonstrate lack of self-awareness about complexity.

Shallow job/company-specific tailoring: Not reviewing the company values and job description or not reflecting those in your stories prevents you from connecting your experiences to their specific culture and needs.

[\[16\]](#)

Common Pitfalls for Senior Candidates and Leaders

While senior candidates (typically Staff+ or L6+) face many of the same challenges as other candidates, they encounter unique pitfalls that can undermine their interviews. These stem from both higher expectations and the complexity of leadership work itself.

[\[17\]](#)

Lack of structure leading to incomplete signal coverage: Unlike junior candidates who might be guided through their stories, senior candidates are

expected to proactively demonstrate their scope. The problem compounds when interviewers interrupt or dive into rabbit holes, as they often do at this level. As the senior candidate, you need to guide the conversation toward the most relevant signal—which means knowing what that signal is in advance.

[\[18\]](#)

Shallow treatment of the complexity of leadership. Leaders are expected to choose and tell stories with deep tradeoffs, complex situations, and to come out of those experiences with meaningful learnings to share with others.

Neglecting the full spectrum of leadership: Senior roles require influence across multiple dimensions, but candidates often tell stories from only one perspective. I frequently hear stories focused purely on technical decisions or product outcomes, missing the equally important people leadership; risk, capacity, and change management; strategy work; and stakeholder management aspects.

[\[19\]](#)

Communicating actions, leaving out frameworks: Senior candidates should demonstrate systematic thinking, including the decision-making framework that led to those actions. Simply listing individual actions makes you sound like an executor rather than a strategic thinker and opens you up to misinterpretation.

[\[20\]](#)

Verbosity: Communication is central to these roles, so talking too much is an easy habit to fall into. There's a lot to cover in limited time and you need to adapt to the signal requested by the interviewer and provide only the most relevant context.

[\[21\]](#)

Opening yourself to uncharitable interpretations: Senior candidates face higher scrutiny, which means stories, or even individual actions, that might be acceptable for more junior candidates can raise red flags when told by someone at a senior level. This requires thinking defensively about how your stories might be interpreted.

[\[22\]](#)

Unprocessed emotional baggage: Leading leaves scars. If you don't process those challenges in an intentional way, your conversation will betray more emotions than you intend, which will heighten the interviewer's evaluation of risk in your candidacy.

Shifting blame: As a leader you're responsible for the team's results so appearing to shift that responsibility to others will be frowned upon.

[1] FAANG: Facebook, Amazon, Apple, Netflix, Google, an admittedly out-of-date acronym. Perhaps MAANGO (Meta, Apple, Amazon, NVIDIA, Google, OpenAI) will become popular. [↑](#)

[2] *Why Behavioral Interviews Matter More than Ever* [↑](#)
<https://thebehavioral.tech/p/why-behavioral-interviews-matter>

[3] STAR: Situation, Task, Action, Results, a common story construction framework. See [What Makes a Good \(Interview\) Story](#).
[↑](#)

[4] [Decode: Understanding the Question](#) [↑](#)

[5] [The Shift in Culture: The Silicon Valley Mythos](#) [↑](#)

[6] [Toward Better Stories](#) [↑](#)

[7] [Step 6: Practice](#) [↑](#)

[8] [Identifying Actions](#) [↑](#)

[9] [Step 3: Scaffolding Stories](#) [↑](#)

[10] [Decode: Understanding the Question](#) [↑](#)

[11] [Why CARL is Better than STAR](#) [↑](#)

[12] [Tuning the Level of Detail](#) [↑](#)

[14] [Select: Choosing a Response](#) [↑](#)

[15] [Step 2: Extracting Story Detail](#) ↑

[16] [How to Research and Apply Company Values](#) ↑

[17] Level 6, typically called Staff level if it has a name. See [Appendix: Leveling at Tech Companies](#). ↑

[18] [Telling Longer Stories](#) ↑

[19] [Additional signal areas for leadership roles](#) ↑

[20] [Coming up with a framework](#) ↑

[21] [Tuning the Level of Detail](#) ↑

[22] [Thinking Defensively](#) ↑

PART 1

FUNDAMENTALS

Before identifying, structuring, and refining your responses, understand what happens in a behavioral interview and know how to deliver what interviewers expect.

HOW BEHAVIORAL INTERVIEWS WORK

On the surface, behavioral interviews are conversations about your past career experiences, but that simplicity masks a sophisticated motivation, structure, and evaluation process. If you understand the premise behind behavioral interviews and the mindset of the interviewer, and leverage that knowledge to engage strategically, you'll increase your chances of being hired in the job and at the level you want.

Why Companies Conduct Behavioral Interviews

Behavioral interviews, like all interviews, are a kind of forecast, designed to predict how well you will perform in the target role. Like all forecasters, interviewers reflect on the past and the present to understand the future, applying as many empirical methods as they can but also relying on a considerable dose of subjective judgment.

Past performance predicts future results. With people, unlike investments, past performance actually is a predictor of future results. Actions come from within—your beliefs, your values, your thoughts, the integrated sum of your past experiences—these determine your steps. Because humans tend to make fundamental changes to their habits, personality, and interactions slowly, behavioral patterns observed in your previous roles are likely to repeat in your next one.

Companies assess against known success patterns. Organizations know what makes their team members successful. Through years of observation and data, they've identified the attributes that distinguish high performers from poor or average ones. Behavioral interviews provide a structured way to evaluate whether you possess these critical capabilities.

Companies have codified this structure into semi-regularized competencies, often called *signal areas*. They combine these with their own *company values* to arrive at formal evaluation criteria to assess during the hiring process.

Leveling requires calibrated evaluation. Companies don't just assess whether you're good or bad—they evaluate whether your capabilities match the target level. The way a new grad PM advocates for a feature is very

different from a senior director's department-wide transformation to meet changing market forces, even though the interviewer acquires signal on ownership and communication from both stories.

Companies have level-specific expectations that exist across their chosen signal areas and company values. Understanding these expectations will help you choose stories and calibrate how you tell them. Check out [Appendix: Leveling at Tech Companies](#) as a reference for how Big Tech company levels map to responsibilities.

What You Get Out of Behavioral Interviews

While interviewers are assessing you, behavioral interviews also serve your interests as a candidate.

Tell your story on your terms. Unlike technical or competency-based interviews where you're engaging with problems defined by the interviewer, behaviorals give you direct control over your narrative. Yes, you're answering questions posed by the interviewer, but you choose which projects to highlight, which details to emphasize, and how to frame your contributions.

Pitch and justify your candidacy. Instead of completely relying on the evaluation methodology of the company, the behavioral gives you a forum for directly pitching yourself. If your resume raises questions—career gaps, lateral moves, unconventional background—behavioral interviews provide the forum to address these and demonstrate why your unique path makes you the right fit.

Understand what the company values. The questions interviewers ask and where they probe deeper reveal the organization's priorities. This helps you assess whether the company's values align with yours and whether you'd thrive in their culture.

Assess your interest in the team. Since behaviorals are often conducted by members of the hiring team, frequently the hiring manager themselves, you'll get a chance to see if working with those people will be fulfilling and enjoyable.

What Interviewers Look For

Behavioral interviewers operate with both a *structured* and *intuitive* evaluation process.

The structured framework: signal areas and repeatable behaviors. Each time they ask you something, there's a "question behind the question" designed to assess one or more of the signal areas and company values we introduced earlier.

As interviewers listen to your responses, they're looking for the behaviors you've demonstrated in the past that are repeatable in their environment. It's where the interview gets its name. They're not just interested in what happened, they want to understand what you did to make it happen, because your actions reveal patterns that will likely recur.

The intuitive framework: culture fit and pattern matching. In addition to consciously looking for signal areas, interviewers pattern-match your stories against archetypes and cultural expectations of their company and industry. This is how they determine whether you sound like someone who would thrive in their specific environment.

This pattern matching happens largely below conscious awareness. When they hear how you approached a conflict or made a decision under ambiguity, they're intuitively comparing you to successful people they've worked with—or unsuccessful ones they want to avoid hiring again.

Consider how these frameworks apply to common questions. When an interviewer asks, "What was the hardest part of that project," they're assessing your scope (what you consider challenging), problem-solving ability, perseverance, and prioritization skills (whether the difficulty was worth solving). Intuitively, they're listening for whether you sound like someone who pushes through hard problems rather than avoiding them and looks for opportunities to "fail fast."

Similarly, "Tell me about a time when you got something from someone who didn't want to give it to you," evaluates conflict resolution, persuasion skills, and relationship management. Intuitively, it tests whether you embrace productive conflict and cut through organizational friction—behaviors that tech culture values. See more about the intuitive framework Big Tech applies in [Big Tech Behaviors](#).

How Interviewers Assess You: Behavioral Questions

To gather signal, interviewers will generally choose from one of three question types:

[1]

“Tell me about a time...” questions are the most common since industrial psychology research has found them most predictive of future job performance. These questions ask for specific examples from your past experience and are designed to extract concrete behavioral evidence. They form the backbone of most behavioral interviews because they’re hardest to fake—you either have relevant experience or you don’t.

Hypothetical questions like “How would you handle a conflict with a designer,” test whether your judgment aligns with the company’s cultural values and the interviewer’s experience of what works. These are also predictive but less so than “Tell me about a time...” questions since anyone can describe an ideal approach. These should be answered with frameworks followed by an exploration of the problem, often involving more back-and-forth with the interviewer to clarify the scenario. If you have an example that fits that scenario, share that as well.

[2]

Values questions like “What is your project management philosophy,” are especially common for leadership roles, again checking cultural alignment and assessing whether you represent the company’s ideal candidate. Examples are still valuable here but the interviewer is looking for frameworks and introspection as well.

As interviewers listen to your responses, they’re collecting both positive and negative signal, according to the areas and values we already mentioned. Strong examples of the behaviors they seek build confidence in hiring you. But they’re also alert for *red flags*—behaviors that suggest you might struggle in their environment.

During the interview, they take notes on both the content of your stories and the signal areas you demonstrate. In a well-run organization, a narrative description of the interview will be produced, expanded from the notes, with the key takeaways called out for readers. Often a breakdown by whatever signal areas or values the organization is targeting is at the top. A final decision is provided, “hire” or “no hire” at a certain level, with justification. Sometimes a confidence level is provided: given what they talked to the candidate about, this is how confident they are in their decision.

[3]

See [Appendix: Behavioral Interview Feedback](#) for an example of written behavioral interview feedback.

Your Role as a Candidate: Decode, Select, Deliver

Behavioral interviews are conversations where the interviewer gathers signal via questions and you choose responses from your career. Every question they ask maps to specific capabilities they're assessing. When you understand what they're really looking for, you can strategically choose and present experiences that demonstrate exactly those capabilities.

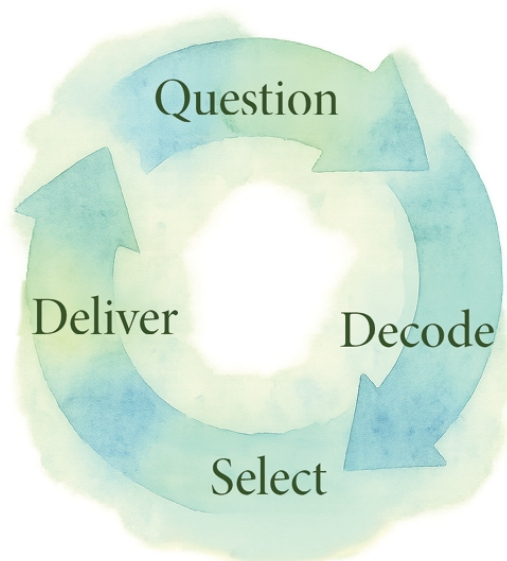
This book teaches you a three-phase approach to every behavioral question, and the rest of Part 1 covers each phase in detail:

Decode the question. Learn about the structured evaluation methods companies use to evaluate you and what they're looking for in your response.

Select the appropriate response based on your understanding of the question, pairing your career stories with the company's needs.

Deliver a compelling and clear narrative that showcases your fitness for the role, avoiding pitfalls, and optimizing for the interviewer's position in the hiring process and signal collection.

Now that you understand the way behavioral interviews work, let's consider each of these steps in turn to learn the principles to master them.



DECODE: UNDERSTANDING THE QUESTION

The first step in the question and answer cycle within a behavioral interview is Decode: to identify what information the interviewer seeks from the question they ask you. This chapter will give you a framework for classifying questions, and a starting point for choosing and crafting the best response.

The Basics: Job Description and Prep Materials

It should (almost) go without saying that if you want to understand what interviewers look for, you should read the job description and heed any prep materials given or conversations had with recruiters. At large companies, job descriptions are often generic and uninformative, but at a startup, you would do well to speak directly to the needs expressed there.

What Interviewers Assess: Signal Areas

Going deeper, typically a company will have what they're looking for in behavioral interviews codified in formal training materials, and possibly even in the interview feedback tool and hiring committee review process.

Signal Areas are what we call the core competencies commonly identified as valuable within our tech economy. Here's a framework for understanding these core competencies, acknowledging that other behavioral interview coaches have curated these areas in different ways. Companies will rarely choose to care about all these things, or even this many things, with notable exceptions like Amazon's 16 Leadership Principles.

[\[4\]](#)

With each signal area, I've included some repeatable behaviors interviewers look for to assess these areas as well as common, related questions they may ask. Of course, a single question can lead to collecting signal across multiple areas: a conflict story is primarily about Conflict Resolution but typically includes Communication and maybe Perseverance, Leadership, etc.

Scope

The overall level of impact, complexity, and seniority you've demonstrated in your work.

Behaviors:

- Managing projects spanning large timescales
- Delivering significant business value, often called “impact”
- Handling technical complexity (system design, performance optimization, architectural decisions)
- Handling organizational complexity (number of team members, number of partners, presence of outside partners, sensitivity of relationships)
- Making decisions with significant consequences
- Showing progression in responsibility over time

Common questions:

- Tell me about your most impactful project.
- What's the most complex technical/organizational problem you've solved?
- Walk me through your career progression.
- Why are you a good fit for this role?

Ownership

Taking initiative to identify problems and drive solutions, demonstrating results-oriented thinking.

Behaviors:

- Proactively identifying problems without being asked
- Driving solutions end-to-end rather than just flagging issues
- Measuring and tracking business outcomes, not just task completion
- Taking responsibility for results beyond your immediate role
- Following up on initiatives to ensure they succeed

Common questions:

- Tell me about a time you identified and solved a problem that wasn't your responsibility.
- Describe a situation where you went above and beyond what was expected.
- Give me an example of how you kept the team focused on delivery.

Ambiguity

The ability to break down large, unclear problems and make progress despite incomplete information.

Behaviors:

- Breaking large, vague problems into concrete, actionable pieces
- Making reasonable assumptions and documenting them
- Gathering information from multiple sources to reduce uncertainty
- Starting work with partial information rather than waiting for complete clarity
- Prioritizing effectively when everything seems important

Common questions:

- Describe a time when you had to work with unclear requirements.
- Tell me about a project where the requirements changed as you progressed.
- How do you prioritize when everything seems important?

Perseverance

Sustained effort through difficulties and maintaining motivation when projects encounter setbacks.

Behaviors:

- Continuing to push forward when projects hit obstacles
- Trying multiple approaches when the first solution doesn't work
- Maintaining team morale and motivation during difficult periods
- Learning new skills or technologies under pressure

- Adapting strategy while maintaining focus on core objectives

Common questions:

- Tell me about a time when a project hit major obstacles.
- What's the most challenging project you've worked on?

Conflict Resolution

Productively navigating disagreements while preserving relationships and driving toward solutions.

Behaviors:

- Initiating difficult conversations rather than avoiding them
- Collecting data or evidence to support your position
- Seeking to understand other perspectives before advocating for your own
- Finding win-win solutions or reasonable compromises
- Maintaining professional relationships after disagreements, including “disagree and commit” behaviors
- Escalating appropriately when direct resolution isn't possible

Common questions:

- Tell me about a time you disagreed with a teammate or manager.
- Tell me about a time when you got something from someone who didn't want to give it to you.
- How do you handle situations where people have different priorities?

Growth

Learning from mistakes, seeking feedback, and helping others develop their skills.

Behaviors:

- Acknowledging mistakes honestly and taking responsibility

- Extracting specific learnings from failures and applying them to future situations
- Actively seeking feedback from others and acting on it
- Mentoring or coaching others to develop their skills
- Sharing knowledge and lessons learned with the broader team
- Demonstrating changed behavior based on past experiences

Common questions:

- What would you do differently next time?
- Tell me about a time you made a mistake and what you learned.
- Describe feedback you've received and how you applied it.

Communication

Clear, effective communication across different audiences and mediums, adapting your approach to context.

Behaviors:

- Adapting your communication style for different audiences (technical vs. non-technical)
- Choosing appropriate communication channels (email, meetings, documentation, etc.)
- Being proactive in sharing information and updates
- Facilitating productive discussions and meetings
- Ensuring alignment across teams and stakeholders

Common questions:

- Describe a time you had to explain something technical to a non-technical audience.
- How do you ensure alignment across multiple teams?
- How did you adapt your communication style to achieve a better outcome in a potentially negative situation?

Leadership

Influencing others and driving outcomes commensurate with your level, regardless of formal authority. Even individual contributors need leadership skills to drive cross-team initiatives and mentor others.

Behaviors:

- Influencing others through expertise, relationships, or compelling vision
- Building consensus around ideas or approaches
- Mentoring or developing other team members
- Taking ownership of team-wide or organization-wide improvements
- Representing your team or project to external stakeholders
- Making decisions that affect others and taking responsibility for outcomes

Common questions:

- Tell me about a time you influenced someone without direct authority.
- Describe how you've mentored or developed others.
- Give me an example of when you drove change in your organization.
- What is your leadership style?
- How do you approach hiring?

Company Values: Signal Areas in Disguise

While the eight signal areas are common across tech companies, many organizations also publish their own company values or leadership principles. These rarely replace the core signal areas, more often repackaging them with company-specific language and priorities.

Understanding how company values relate to universal signal areas helps you prepare more effectively while avoiding the trap of over-rotating on company-specific messaging.

How Company Values Connect to Signal Areas

Most company values are variations on the signal areas you already understand. Amazon’s “Ownership” principle maps directly to our Ownership signal area. Google’s “Focus on the user” connects to both Ownership and Communication. Meta’s “Move fast” relates to both Ownership and Perseverance.

The key insight is that interviewers trained on these company values are still fundamentally assessing the same underlying behaviors but using different vocabulary to describe them.

When Company Values Matter Most

Some companies, particularly Amazon with their 16 Leadership Principles, integrate their values deeply into daily operations. Amazon employees reference these principles in meetings, performance reviews, and decision-making. At companies like this, knowing the specific language matters because interviewers expect you to speak their cultural dialect.

At most other companies, like Meta, Google, or most startups, company values exist on websites and in onboarding materials—and sometimes on the walls—but rarely surface in day-to-day work. Interviewers may occasionally reference them, but they’re primarily looking for the underlying signal areas regardless of specific values language.

How to Research and Apply Company Values

Company values are typically published on the website, in job descriptions, or in recruiter materials. A quick search for “[Company Name] values” usually surfaces them immediately.

As you review a company’s values, mentally map each one to the signal areas you already understand. This helps you see what they’re really assessing beneath the company-specific language.

Look for opportunities to use their specific language when it fits naturally. If Amazon asks about a time you showed ownership, you might say “I took ownership of the customer experience” rather than just “I took initiative.” But don’t force it: the underlying behavior matters more than vocabulary matching.

The Value-Dropping Trap

Many candidates make the mistake of trying to memorize company values and awkwardly insert them into every response. Here’s an example with

Meta's "Living in the Future" company value, in response to a question about choosing a framework:

 *Well, at my previous company, I really tried to "Live in the Future" when we were deciding between React and Vue for our frontend framework. I knew that's important here, so I thought about what the future would look like. Since you created React, I figured that was the future-oriented choice...*

Instead, let company values inform your story selection and language choices, but don't let them override the fundamental principle of demonstrating strong behavioral evidence across all signal areas.

 *Well, at my previous company, we were deciding between React and Vue for our frontend framework. I knew that implementing the current best practices, long-term maintainability, and maturity of the community were important aspects so we chose React...*

SELECT: CHOOSING A RESPONSE

After you decode the question and understand the signal areas requested by the interviewer, it's time to choose a response based on the question type:

[5]

- **“Tell me about a time...” Questions:** choose a story from your past
- **Values Questions:** provide a generalized framework for how you approach that aspect of your role, then relate a story from your past that exemplifies that approach
- **Hypothetical Questions:** clarify the situation with your own questions, provide a framework, walk the interviewer through what you would do, applying the framework, and mention a story from your past where you approached something similar

Because all of these involve eventually telling a story, and because “Tell me about a time...” questions are the most common, let's cover how to choose stories effectively. We'll address the specifics of responding to Values and Hypothetical questions later in this chapter.

How to Choose a Story

When choosing a story from the past, prioritize according to the following four criteria, in order:

1. Scope. Choose the project with the largest scope. Scope can be defined in a number of ways:

- Breadth and variety of actions you took (did you *plan* and *execute* and *resolve conflicts* along the way?)
- Timescale for the project (weeks vs. months vs. years)
- Technical or organizational complexity
- Impact the story had on the business/team/product/etc.

2. Relevance. The story must match the question and deliver the requested signal. Priority should go to stories that naturally demonstrate the signal areas requested without requiring extensive explanation.

[6]

Consider relevance on a spectrum from great to weak matches:

- **Great match:** *Tell me about a time when you failed → My recommendation to rebuild our authentication system from scratch turned out to be wrong...*
- **Decent match:** *Tell me about a time when you failed → I underestimated the complexity of integrating with a legacy system...*
- **Weak match:** *Tell me about a time when you failed → Our sprint velocity was lower than planned...*

Also consider relevance to company, role, and even the profile of the interviewer—circumstances similar to the interviewer’s team will connect more with their needs.

3. Uniqueness. Sometimes you’ve already used a story in response to a previous question that might be a good response to the latest question. Do you go back to that story or try to cover new ground?

To answer that, consider that a great interview is one where you’ve shared *all the core stories* that best represent your past contributions similar to the target role—the stories you’d tell a hiring manager if you only had a few minutes. All else being equal then, opt for the *new* story the listener hasn’t heard.

If you have more than one new story to choose with sufficient scope and relevance, consider uniqueness *across signal areas*. If you’ve already told a technically complex story, you might choose an organizationally complex, for example.

4. Recency. More recent stories carry more weight because they represent your current capabilities and thinking, impacting the interviewer’s perception of your overall scope. A story from last year typically trumps one from five years ago, all else being equal.

For early-career candidates, lean more heavily toward recent work since you’re growing rapidly and your newest projects best represent your capabilities. For senior candidates, you have more flexibility—a standout story from two years ago that demonstrates exceptional leadership or technical depth can outweigh a more recent but less impressive example.

Be careful with stories from 3–5 years ago. You probably should avoid stories from 6+ years ago unless you are very senior (like VP or above) or there are extenuating circumstances (like career breaks).

If you're unsure, use the [*The Menu Technique: Letting Your Interviewer Choose*](#).

Why this priority order matters

This is the right priority order because you have limited time in an interview, but have to balance that with the assessment goals of the interviewer.

Scope over Relevance: The classic example is a question like *Tell me about your favorite project*, which you shouldn't answer according to personal affinity (like your favorite movie) but rather with a project that best represents your largest achievement. Even if another story is slightly more relevant to the literal question asked, your highest-scope story will provide the most valuable signal—the list of repeatable actions—to the interviewer because it gives them the most insight into why to hire you.

For example, if the question was *Tell me about a time when you had a conflict with your manager*, but all your disagreements with your actual manager were minor, you might choose the higher-stakes conflict with the product manager instead that shows your data-driven decision making. You still provide conflict resolution signal with positions of authority, but showcase more of your repeatable actions.

Scope over Recency: A project from two years ago that demonstrated significant technical leadership and business impact is more valuable than last month's straightforward feature implementation. Interviewers are assessing the high-water mark of your capabilities

Relevance over Recency: If you have an older story that directly demonstrates the requested signal area versus a recent story that only tangentially relates, choose the older story.

Consider Uniqueness more as the interview progresses: Early in the interview, focus on scope and relevance. As you move through multiple questions, start factoring in whether you've already told certain stories or demonstrated certain capabilities. Try to work in your most impressive stories and more diverse set of signal areas, possibly stretching relevance in the process.

Common Story Selection Challenges

Picking a most impactful project or conflict story. Because these are so common and take up so much airtime in the average behavioral, I cover these separately in [Step 4: The Big Three Questions](#).

Picking a story about weaknesses or failures. These are also very common, and I cover these in [Questions About Weaknesses, Mistakes, and Regrets](#).

When you only have weak matches. I frequently encounter mentees with only weak matches or with relevant stories but ones that have weaker impact or recency. Mentioning a number of smaller stories can help land a sense of scope and impact even if the primary story you tell lacks depth. See the [The Halo Effect: Getting Mileage Out of Lists](#) in cases like these to convey the impression of accomplishment without relying on only one story.

Whether to pick an older story. Balance story quality with story recency and understand that the more junior you are, the more the interviewer will expect recent stories, especially since your scope and skills are rapidly changing in the early stages of your career.

How to match stories to the audience: The stories you choose when talking with the direct hiring manager may be different from those you choose when interviewed by cross-functional partners. Think about whether the story addresses topics relevant to the listener.

What if I don't have a relevant story at all?

Sometimes you just won't have even a good match. Try to get as close as possible to the signal area and tell that story, similar to the weak match example above. If you really have nothing, not even a weak match, then be honest about that and consider falling back and treating it like a Hypothetical question. Never lie or make up a story (see below).

Junior candidates: when no work-related stories are available

Always prefer a work-related story, but sometimes they simply aren't available if you are a junior candidate.

If this applies to you, then you can choose a story from school, and if there aren't any of those available, then choose a personal story. As a junior

candidate, it is better to have some story, even if it is a personal one, than to have no story at all.

Choosing a story from school

Academic projects can demonstrate the same behaviors as workplace projects. Look for experiences where you took initiative, solved complex problems, or worked with others to achieve meaningful outcomes.

Choosing a personal story

Personal stories should still demonstrate professional behaviors like leadership, problem-solving, or overcoming challenges. Avoid overly personal topics and focus on situations that translate to workplace skills.

Should I lie or make up stories?

No. Never fabricate stories or embellish your role beyond what actually happened.

I understand the temptation. You're in an interview for your dream role and the interviewer asks about leading a major initiative, but you've only contributed to large projects, never led them. Or they ask about conflict resolution and the only disagreement you can remember feels too minor to mention. The pressure to have the "perfect" story for every question can push candidates toward exaggeration, outright invention, or to rely on off-screen LLM-generated responses. Don't do it:

[\[8\]](#)

- **Lying damages your character.** You will lose respect for yourself and create opportunities for impostor syndrome even if you get the job.
- **Lying is hard.** Actors make it look easy but it probably won't be easy for you. You won't have the relevant details about your fabricated projects, and experienced interviewers notice when details get vague, timelines seem unrealistic, or when your confidence drops under probing.
- **Lying is risky.** You could be rejected on principle or damage your reputation in the industry.

Instead of lying, reframe your contributions or be honest that you don't have a story and treat it like a Hypothetical question. Here's how you can reframe:

Tell me about an initiative you led?

✗ *I led the migration from our monolith to microservices, making all the key architectural decisions...*

✓ *I drove the initial proof-of-concept for our microservices migration, which informed the broader team's architectural approach. While the senior engineer made the final decisions on service boundaries, I owned the evaluation of communication patterns and proposed the event-driven architecture we ultimately adopted...*

Tell me about a conflict you resolved.

✗ *I resolved a major conflict between engineering and product by convincing them to adopt my proposed solution...*

✓ *When engineering and product disagreed about our approach, I gathered data on both options and facilitated a discussion where we could evaluate them objectively. This helped us reach consensus on a hybrid approach...*

Should I repeat stories?

A common concern is whether it's acceptable to use the same project example across multiple behavioral questions within an interview or between interviews. In other words, should Scope and Relevance truly take precedence over Uniqueness.

The answer depends on context, but I would encourage you to consider reusing stories (or a different part of the same story) if it means producing the strongest response to the specific question asked, unless you are running out of time and have an important and at least somewhat relevant core story you haven't told yet.

There are some places where repetition is common and acceptable:

Single behavioral interview: In a 45-minute session where you might only tell 2–3 detailed stories, using your best examples matters more than an artificial sense of variety.

When it's genuinely your best example: If your database migration project provides the clearest evidence of conflict resolution skills, use it—even if you already discussed it for technical challenges.

Here are some times when you might consider avoiding repeated stories:

Multiple behavioral sessions: Senior-level loops often include several behavioral rounds, often with sessions focused on specific themes like project deep dives or collaboration. Here, demonstrating breadth of capabilities becomes more valuable, but don't sacrifice quality/quantity of signal (i.e., Scope and Relevance).

Follow-up interviews: If the follow-up stems from concerns about story choice or signal quality, avoid repeating the same examples. See [Follow-Up Interviews](#).

Mitigating the impact of repeating stories

When you need to reuse a story, here are some strategies to mitigate the impact.

Use the Menu Technique when you have multiple relevant examples, presenting options to the interviewer instead of assuming which they'd prefer to hear.

[\[9\]](#)

✓ *I could tell you about two different approaches I've taken to conflict resolution. One involved a disagreement with another engineering team about technical architecture—which I discussed before—and another was about resource allocation with my manager, but might not fit the question as well. Which would be more useful for you to hear?*

List your stories and leverage the Halo Effect to communicate breadth:

[\[10\]](#)

✓ *I've resolved conflicts across multiple roles, and with multiple stakeholders, including other eng partners and PMs, but I think the best example is one I mentioned in another interview. Let me tell you how I resolved a technical architecture dispute with another engineering team...*

Adapt your emphasis to the question when reusing a same story. Don't use identical details every time you reference the same project. Adapt your focus and emphasis based on what each specific question is trying to assess. The same database migration might emphasize technical complexity for one question and stakeholder management for another.

Choosing Responses to Values Questions

Questions like “What is your approach to cross-team communication,” are not the same as “Tell me about a time...” questions. These Values questions require you to demonstrate some generalized framework for addressing a common work situation, and often your maturity in articulating and justifying this approach will weigh heavily on the listener’s view of your scope.

[\[11\]](#)

These frameworks don’t have to be formal or have officially recognized names. For example, you might choose to respond to the cross-team communication question with a three-layer framework:

1. **Layer 1:** a tight-loop communication with teams you directly depend on or who depend on you.
2. **Layer 2:** structured touchpoints with adjacent teams who might be affected by your work or whose work might affect you.
3. **Layer 3:** broadcast communication for the broader organization.

When you provide a framework like this, you immediately answer the question asked but also demonstrate systematic thinking. I would discourage you from making these frameworks too long before pivoting to an example.

Coming up with a framework

Unless it’s a very common question, such as “Tell me your approach to testing” or “How have you conducted hiring,” you’re unlikely to have a prepped framework response. When caught off-guard by a Values question, take 5–10 seconds to quickly construct a credible framework by asking yourself:

[\[12\]](#)

1. **What varies?** Is it the stakeholders, the time, the resources, the risk?
2. **Pick 2–3 variations** but don’t over-complicate it.
3. **Name** your approach for each variation.
4. **Add one insight** about when to use each one.

Common framework patterns

These semi-formal systems come in common flavors:

- For anything about **communication**: Segment by frequency vs. formality (ad-hoc → regular → formal)
- For anything about **prioritization**: Use impact vs. effort or customer value vs. technical risk
- For anything about **conflict**: Scale by stakes (low → medium → bet-the-company)
- For anything about **quality**: Balance by constraints (speed/quality/scope triangle)
- For anything about **learning**: Divide by learning style (observing → doing → teaching)

Always share an example

Immediately back up the framework with an example of when you applied that framework. Do this even if you just came up with the framework on the fly. You're not lying, you just formalized it after you used it. For example, you might select a story about applying this three-layer communication to a recent project, discussing how the framework helped to keep the project on track.

Choosing Responses to Hypothetical Questions

Questions like “What would you do if a project was behind schedule,” are similar to Values questions. The key to handling these Hypotheticals is recognizing that while you may not have faced the exact scenario, you've likely faced the same category of problem.

Choose clarifying questions carefully

Unlike the other questions we've considered, where you launch into a story, Hypotheticals benefit from clarification. Select 1–2 questions based on what would genuinely change your approach to the problem.

Good question selection:

- Shows you understand nuance: “Is this a B2B or B2C product?” (when asked about feature prioritization)[\[13\]](#)

- Reveals critical constraints: “Do I have hiring authority?” (when asked about scaling a team)
- Identifies stakeholder dynamics: “Who are the key decision makers?” (when asked about driving consensus)

Poor question selection:

- Too basic: “What do you mean by ‘difficult’?”
- Too numerous: Asking 5+ questions before providing any value
- Too deflecting: “Well, what would you do?”

Frameworks for Hypothetical Questions

As with Values, look to create a framework and use a pattern-matching approach to do so:

- **Abstract the core challenge:** Project behind schedule → resource/time/scope tension
- **Scan your experience for similar tensions:** Have you dealt with other deadline pressures? Resource constraints? Stakeholder disappointments?
- **Extract the transferable principles:** What worked? What failed? What would you do differently?

As another example, if asked “How would you handle a team member who consistently misses deadlines,” and you’ve never managed someone with that specific issue, consider:

- Have you dealt with unreliable vendors? (similar accountability challenge)
- Have you helped struggling peers? (similar performance issue)
- Have you missed deadlines yourself? (gives you empathy and insight)

To help you build a framework library, here are some building blocks organized by theme:

- **Diagnostic Frameworks** for “something is wrong” scenarios:

- People vs. Process vs. Technology
- Internal root cause vs. External factors
- Acute crisis vs. Chronic issue
- **Decision Frameworks** for “what would you choose” scenarios:
 - Impact vs. Effort
 - Reversible vs. Irreversible
 - Build vs. Buy vs. Partner
- **Communication Frameworks** for “how would you handle” scenarios:
 - Audience × Message × Medium
 - Diagnose → Align → Execute → Follow-up
 - Listen → Empathize → Reframe → Solve

Select and combine these building blocks based on the question. For instance, “How would you turn around a failing project?” might combine diagnostic (what’s wrong?) with decision (what to change?) with communication (how to align stakeholders?).

Select a relevant career story

To ground the conversation, you’ll still want to share a story from your career related to the hypothetical situation. Use the same story selection criteria we’ve discussed for “Tell me about a time...” questions.

DELIVER: TELLING A GOOD STORY

Our brains love stories. Whether their appeal flows from innate neural structure or whether stories are so ubiquitous they've reshaped that structure—or more likely both—the result is the same: we're hardwired to think in and remember stories. When we hear one, specialized brain regions spring into action, performing mental time travel, triggering empathy, and running simulations of events.

[\[14\]](#)

This biological foundation makes stories potent weapons for interviews. Unlike a list of accomplishments or abstract descriptions of your skills, stories capture the interviewer's attention and pull them into your experience. Stories slip past critical defenses and activate our brain's dedicated narrative-processing machinery, making stories easy to remember.

[\[15\]](#) [\[16\]](#)

We can leverage these and other scientific findings to create compelling, structured narratives to convince listeners that you're the right fit for a job. After reading this chapter, you'll learn a recommended structure to respond to Tell Me About at Time, Values, and Hypothetical questions.

What Makes a Good (Interview) Story

Unsurprisingly, there are psychological and physiological reasons why stories from different millennia and across the globe have similar structures. We naturally prefer stories with a problem–action–resolution arc, the more entertaining and aligned with personal interests the better. Considering the kinds of narratives we tell in interviews, stories that explicitly connect actions to outcomes (“because I did X, Y happened”) are more convincing and useful for decision-making.

Putting this all together and combining it with common best-practices, here are the key elements your stories should include to maximize their impact:

[\[17\]](#)

- Clearly presented **actions** that represent **repeatable behaviors** aligned to the signal areas asked about

- Enough **context** to understand these actions and create the problem-action-resolution arc[\[18\]](#)
- Enough **results** to understand whether these actions were effective and to assess whether you understand what success is
- Be **interesting**, which usually means relevant to the role and question, but ideally there are also unique and unfamiliar details
- **Short** enough since the interviewer has other questions to ask, and honestly, career stories are not that entertaining[\[19\]](#)

STAR: A Simple Approach to Story Structure

If you've read anything about behavioral interviews, you've come across the STAR method as framework for achieving these goals and structuring behavioral responses:

- **Situation:** The background and context of the story—what was happening, who was involved, and why it mattered.
- **Task:** Your specific responsibility or challenge within that situation—what you needed to accomplish or resolve.
- **Actions:** The concrete steps you took to address the task—what you did, how you did it, and why you chose that approach.
- **Results:** The outcome of your Actions—what changed, what impact you made on the business.

Created by *Development Dimensions International* in the 1970s, having a fixed structure like this helps you tell a story with a compelling arc without missing any key pieces interviewers need to evaluate you. An easy-to-remember shorthand like “STAR” makes it possible for you to structure your stories on the fly, with just a little practice in advance. Some interviewers are even trained to take notes using this structure, with enforcement from their company's tools.

[\[20\]](#)

This isn't a bad method. It creates a story arc, with a clear beginning, establishing context; a middle with actions taken within that context; and an end, with what happened to the project after those actions.

If you're armed with this format, you will perform much better than someone without any preparation at all. But this format has some

drawbacks, especially for senior candidates, and there's a better one.

Why CARL is Better than STAR

STAR has at least two major drawbacks for most candidates:

1. It leaves out any form of reflection on your experiences, which is a way for the interviewers to assess your scope—more senior people learn from their mistakes and extract wisdom to be reused on future projects.
2. Candidates get too caught up in differentiating between Situation and Task, which are often convolved together in large projects, and they waste prep time trying to tease apart the two.

Instead of STAR, there is another format that solves these problems which I prefer, called CARL:

- **Context:** The overall context of what was happening, including how you were involved—the combination of Situation and Task.
- **Actions:** [same] The concrete steps you took to address the task—what you did, how you did it, and why you chose that approach.
- **Results:** [same] The outcome of your Actions—what changed, what impact you made on the business.
- **Learnings:** What you learned or your reflections on the choices you made.

In CARL, Actions are still the centerpiece but Context can expand or contract, centering around providing the listener with the minimum amount of setup so they can understand the Actions and their motivations.

Learnings are added as a more natural end of the story—think Aesop's fables. And as we mentioned when calling out the challenge with STAR, they provide you a platform for expressing your depth of wisdom, showing that you are self-aware and seek to grow from your experiences.

Story Construction Guidelines

When applying this CARL story format, keep in mind a number of key guidelines:

Actions should be plural. Interviewers are looking for you to list as many relevant actions as possible. Again, they are assessing whether you can repeat your Results in their organization by repeating your Actions. See [*Step 2: Extracting Story Detail*](#) for more on what actions to include.

[21]

Keep the Context and details minimal. Only provide the Context that's required to understand the scope of the story and to justify your Actions. If you're in the middle of the story and you haven't shared an action for a while, you might be providing too much detail.

Provide the requested signal early. Remember that you're telling a story in response to a specific question, seeking some specific signal. Provide that signal promptly or at least make it clear that the story is directly related to the interviewer's question.

Stories should be around 2 minutes long. People are always asking how long responses should be and of course the answer is, "It depends." It depends on the question, on the point in the interview, on the interviewer, etc., but 1–3 minutes is pretty typical. You should know what you're doing if you're going over 3 minutes.

[22]

Results vary based on the type of question. Results can be more than simply the business outcomes from a project. For conflict stories, the Result should include what happened with the project or task, but also what happened in the relationship. Resolving the conflict but leaving the combatants in a poor working relationship is a sign of weak conflict resolution skills. For mentoring stories, the Result includes how the mentee grew—were they able to handle future tasks in a better way, were they promoted? For communication stories, reflect on whether your chosen method, timing, and audience were effective.

Avoid "Fairy Tale Endings." Rarely do projects end with great business results, no tradeoffs, and everyone being happy. Probably something didn't go as planned (part of the Results) and you can learn from it next time (Learnings).

Including Details

“Not enough detail.” “Too much detail.” These are among the most common pieces of feedback interviewers give—and the most frustrating for candidates to receive. How much detail is enough? When does helpful context become unnecessary background? When does demonstrating expertise become unrelatable to the listener?

The appropriate level of detail exists on a spectrum, and where you land on that spectrum depends on your role, your audience, the specific question, and where you are in the conversation. Here are some quick guidelines for you. See [Tuning the Level of Detail](#) for a more, dare I say, detailed discussion.

Use detail to (1) make the story understandable and (2) establish your credibility. Your actions become comprehensible when the listener has enough context to understand them. Details also prove you actually did the work and understood it deeply. As such, once you’ve established your credibility, additional details are wasting precious time. Choose where you want to go deep, go there, then keep the narrative moving.

Consider how the second example here establishes more credibility without taking too much more time:

 *I improved our service’s performance by optimizing the database queries. It made things a lot faster and our users were happier.*

 *I improved our service’s performance by identifying that we were making $N+1$ queries on our product listing page—every product triggered a separate database call for its reviews. I refactored this to use a single JOIN query with eager loading, which reduced our p95 response time from 3.2 seconds to 400ms. Customer support tickets about slow loading dropped by 60% in the following month.*

Provide details mostly in the Actions. Your Actions are where you demonstrate thinking, technical depth, leadership, and decision-making. This is where interviewers learn who you are as a professional. In your Actions, include:

[\[23\]](#)

- The alternatives you considered: *I evaluated three approaches: microservices, a modular monolith, or refactoring our existing*

monolith...

- Your reasoning: *I chose the modular monolith because our team of 8 couldn't support 15 microservices...*
- Specific technical decisions: *I implemented a hexagonal architecture pattern with clear boundaries...*
- How you navigated challenges: *When the VP questioned the timeline, I created a phased rollout plan...*
- Collaboration and influence: *I paired with our Principal Engineer to validate the approach...*

Likewise, any details you provide in the Context section should be relevant—don't provide an organizational structure description or a latency number unless your story has you communicating through that structure or making some technical choices based on that latency.

The 30-second rule. If you haven't moved on to your next action after about 30 seconds of talking, you're probably providing too much detail about that single point.

Feel free to note opportunities for follow-ups. Say "I can talk more about that if you like..." then keep moving.

Examples of Stories

Here are some examples of complete CARL stories for a few common questions. More examples of two common story types—projects and conflicts—are provided in [Step 4: The Big Three Questions](#).

What was the hardest technical problem you've ever solved? [Staff-Level Backend Engineer]

CONTEXT

✓ *Our payment processing system was experiencing intermittent failures affecting about 0.1% of transactions. Rare enough to be hard to reproduce, but frequent enough to cost us \$50K weekly in customer credits. These phantom failures happened during our highest-traffic periods, and customers were getting charged but not receiving their digital goods. After two months of unsuccessful debugging by different engineers, I inherited this as our top P0 issue when our most experienced backend engineer left the company.*

ACTIONS

First, I instrumented our distributed tracing more aggressively, adding correlation IDs across all microservice boundaries and increasing our logging verbosity in production for affected customer segments. This revealed the failures clustered around specific database shards during traffic spikes.

I hypothesized we had a distributed race condition between our payment service and inventory service. To prove this, I built a chaos engineering harness that could inject artificial delays at precise points in our transaction flow. After running thousands of simulations, I could reliably reproduce the issue—when payment confirmations arrived within a 50MS window of inventory updates, our optimistic locking mechanism would silently fail due to a subtle bug in our retry logic.

The fix required redesigning our distributed transaction coordinator. I implemented a saga pattern with explicit compensation logic, ensuring each step could be safely retried or rolled back. I also added circuit breakers to prevent cascade failures and implemented idempotency keys properly across all services.

To validate the solution, I ran it in our staging environment under 10× normal load for 72 hours, then deployed it behind a feature flag to 1% of traffic, gradually ramping up while monitoring our error rates.

RESULTS

The solution eliminated the phantom failures completely, recovering \$200K monthly in lost revenue. The incident documentation and runbooks I created became the template for debugging distributed systems issues across the company, and the chaos engineering framework I built was adopted by three other teams.

LEARNINGS

During this project, I learned that distributed systems debugging requires a different mental model than monolithic applications—you have to think probabilistically rather than deterministically. I also realized that investing in observability infrastructure upfront would have paid massive dividends during crisis debugging. Most importantly, I learned that the hardest bugs often lurk in the spaces between systems, where each component works correctly in isolation but fails in combination. This changed how I approach system design, always considering failure modes at integration boundaries.

This story works particularly well because it shows the candidate operating at the intersection of deep technical expertise and business impact.

- **Demonstrates Senior/Staff-level technical depth.** Shows mastery of distributed systems concepts (race conditions, saga patterns, distributed transactions), uses sophisticated debugging techniques (distributed tracing, chaos engineering, correlation IDs), and implements advanced patterns (circuit breakers, idempotency keys, optimistic locking).
- **High business impact with quantifiable results.** Clear financial impact (\$50K weekly loss → \$200K monthly recovery) and shows an understanding of business consequences beyond just technical issues.
- **Systematic problem-solving approach.** Methodical investigation starting with observability improvements, followed by hypothesis-driven debugging rather than random attempts. Then building custom tooling to reproduce the issue and validating the solution thoroughly before full deployment.
- **Shows leadership without direct authority.** Creating artifacts and tools that later help other teams. Not just fixing the bug but improving the system's overall resilience.
- **Strong reflection shows growth.** The candidate shows an evolution from deterministic to probabilistic thinking, recognizes the ROI of observability investment, and understands that integration points are where complex bugs hide. [\[24\]](#)

What decision did you make that significantly influenced a project's direction? [Mid-level PM]

CONTEXT

Six months into building our B2B analytics dashboard, we were targeting a Q4 launch with 15 enterprise customers in our pilot program. We'd been building based on extensive customer interviews that surfaced 42 "must-have" features. The engineering team of 8 was working nights and weekends, we'd already pushed our launch twice, and our burn rate meant we had 5 months of runway left. The CEO was getting pressure from the board about our delayed revenue targets, and our sales team had pre-sold

the full feature set to several Fortune 500 prospects based on our original timeline.

ACTIONS

✓ I made the difficult decision to radically reduce our launch scope from 42 features to just 6 core capabilities. But I knew I couldn't just announce this—I needed to build consensus and validate this would still deliver value.

First, I analyzed our pilot customer usage data from our beta. Despite having access to 30+ features we'd already built, 80% of all user sessions only touched the same 6 features: data ingestion, basic visualization, automated reporting, user management, API access, and alerting. Everything else had less than 5% adoption.

I then conducted emergency interviews with our 5 most engaged pilot customers, specifically asking: "If you could only have 6 features at launch, which would make or break your purchase decision?" Their answers aligned with the usage data.

Next came the hard part: alignment. I created three scenarios for our leadership team:

- 1. Continue with full scope (launch in 4 months, high risk of quality issues)*
- 2. Pivot to 6-feature MVP (launch in 6 weeks, iterate post-launch) [\[25\]](#).*
- 3. Seek bridge funding to complete original vision (dilution risk)*

For engineering, I proposed that the remaining features wouldn't be canceled but rather "fast-followed" in two-week sprints post-launch. I checked with the EM to see if the product could be designed so these features could be cleanly added later without massive refactoring.

[\[26\]](#)

The sales conversation was the trickiest. I partnered with our head of sales to revisit every deal, identifying which customers truly needed which features by when. We discovered only 2 of our 15 pre-sold customers actually required features beyond our core 6 for their initial rollout. For those two, we negotiated phased deployments with contractual commitments for feature delivery dates.

I also created a public roadmap showing our post-launch velocity, demonstrating that customers would actually get more features faster with this approach since we'd have revenue to hire more engineers.

RESULTS

We launched the MVP in 5 weeks, beating even our aggressive timeline. Thirteen of our fifteen pre-sold customers converted immediately, generating \$2M ARR and extending our runway by 18 months. The two customers requiring additional features signed with delivery milestones.

[\[27\]](#)

More importantly, the faster launch gave us real production feedback that invalidated 12 of our planned features—customers actually wanted different capabilities than they'd originally requested. We would have wasted 3 months building the wrong things.

The engineering team's morale improved dramatically. They went from death march to sustainable pace and could see customer value immediately. We hired 4 more engineers with our new revenue and delivered more features in the 6 months post-launch than we had in the previous year.

LEARNINGS

This experience fundamentally changed how I approach product development. I learned that customer interviews tell you what people think they want, but usage data reveals what they actually need. I now always advocate for launching with a minimal but excellent feature set rather than a comprehensive but mediocre one.

I also learned that reframing isn't just about the decision itself but about creating a narrative that addresses each stakeholder's specific concerns. Engineering cared about technical debt and sustainability, sales cared about customer commitments, and leadership cared about runway and growth. The same decision needed three different stories.

Most importantly, I learned that sometimes the hardest decisions are about what not to build. Every feature we didn't launch was someone's pet project or a customer's specific request. But protecting the team's focus and the product's quality sometimes means saying no to good ideas to preserve great execution.

This example follows the CARL structure while demonstrating the key PM competencies interviewers look for: customer focus, data-driven decision making, stakeholder alignment, and strategic thinking.

- **High stakes decision.** The story demonstrates a pivotal moment where the PM had to choose between competing pressures: customer expectations, team burnout, and company survival.

- **Data-driven approach.** Rather than making a gut decision, the PM used both quantitative (usage data) and qualitative (customer interviews) data to validate the pivot.
- **Stakeholder management.** The story shows sophisticated handling of multiple stakeholder groups with different concerns: engineering, sales, leadership, and customers.
- **Measurable impact.** Clear metrics (\$2M ARR, 18-month runway extension) demonstrate the decision's success.
- **Strategic thinking.** The decision wasn't just tactical (what to build) but strategic (how to position the company for long-term success).
- **Growth mindset:** The learnings show evolution in thinking about product development philosophy, not just tactical lessons from one project.

Describe a project that took much longer than expected. [Mid-level Engineer]

CONTEXT

✓ *I was tasked with migrating our monolithic REST API to GraphQL, estimated at 6 weeks by our tech lead based on the number of endpoints. This was supposed to be a straightforward technical upgrade to improve our mobile app's performance, which was suffering from over-fetching issues and required 15–20 API calls per screen. The mobile team was blocked on several features waiting for this migration, and we'd promised the CEO this would be done by Q2 so we could start on some features Sales needed. What we didn't realize was that our API wasn't just serving our mobile app. It had become a shadow integration platform for 12 internal tools that various teams had built over three years.*

[\[28\]](#)

ACTIONS

The first two weeks went smoothly. I migrated 20 endpoints and was actually ahead of schedule. Then our data analytics dashboard started failing. I discovered that team had been scraping our REST endpoints every hour for their reports, parsing the JSON responses with regex. My GraphQL migration broke their entire pipeline.

I called an emergency meeting and discovered the scope was much bigger: customer success had built Chrome extensions using our APIs, the sales team had Zapier workflows, and QA had automated test suites hitting these

endpoints directly. Of course no one on the engineering side knew about these.

I had to make a choice: force everyone to migrate, risking broken workflows and team friction, maintain both APIs forever, or find a middle path. I proposed building an adapter layer which would expose a REST version of the GraphQL data during a transition period.

But building this adapter revealed deeper problems. Our REST API responses had been “customized” over time with undocumented fields that different teams relied on. The GraphQL schema forced us to formalize these, exposing years of technical debt. What should have been a schema definition exercise turned into many meetings with different stakeholders to understand their actual data needs.

At this point, I needed help from the TL, who met with me and the teams. She helped me create a dependency map and migration tracker, showing which teams used which endpoints and their migration timeline. I ran weekly office hours to help teams update their integrations, wrote migration guides for common patterns, and reviewed code.

[\[29\]](#)

When we hit week 8 with only 40% completion, I had to have a difficult conversation with my manager and the CEO about the timeline. The TL and I met with them and presented three options: rush the migration and break internal tools, do it right and take 6 more weeks, or abandon GraphQL entirely. We advocated for doing it right, showing how the technical debt we were uncovering was already costing us in slow API responses.

RESULTS

The project ultimately took 14 weeks—more than twice as long as estimated. However, the extended timeline led to unexpected benefits. API response times improved by 60%, reducing our AWS costs by \$8K monthly. The mobile app’s network usage dropped similarly, significantly improving user experience in low-bandwidth regions. The formalized GraphQL schema became our source of truth, eliminating the “tribal knowledge” problem.

Most surprisingly, the process of migration uncovered and fixed some critical bugs that had been silently corrupting data for months.

This project taught me that estimates in legacy systems are fantasy without proper discovery. I learned to always ask “Who else might be using this?”

before touching any shared resource. Now I start migrations with a discovery phase, including API traffic analysis and stakeholder interviews.

LEARNINGS

I also learned that sometimes taking longer is actually faster. If we'd forced the 6-week timeline, we would have spent months fixing broken integrations and damaged relationships. The extra 8 weeks upfront saved probably months of technical debt payments.

Most importantly, I learned that communication becomes exponentially more important when projects extend. My weekly updates to stakeholders, showing progress and value delivered even during the delay, maintained trust despite the timeline slip.

This example follows the CARL structure while demonstrating key engineering competencies interviewers look for: technical ownership, stakeholder communication, pragmatic problem-solving, and learning from failure.

- **Realistic scope creep scenario.** The story demonstrates a common real-world problem where initial estimates miss hidden dependencies, showing the candidate understands that software exists in an ecosystem, not isolation.
- **Proactive problem-solving.** Rather than just reporting delays, the engineer identified root causes, created solutions (adapter layer, migration guides, office hours), and presented options with tradeoffs to leadership.
- **Cross-functional collaboration.** The story shows sophisticated handling of multiple stakeholder needs without throwing anyone under the bus.
- **Quantifiable positive outcomes.** Despite the delay, clear metrics (60% faster APIs, \$8K monthly savings) demonstrate that the extended timeline delivered real value.
- **Technical maturity.** The engineer didn't just patch problems but addressed root causes, building proper API governance, formalizing schemas, and creating documentation to prevent future issues.
- **Managing upward effectively.** The story shows transparent communication with management and executives, presenting options rather than just problems, and controlling the narrative around the delay.

- **Growth through adversity.** The learnings show evolution from pure technical execution to understanding systems thinking, stakeholder management, and the true cost of technical debt.
- **Avoids common pitfalls.** The candidate doesn't blame others for poor documentation, doesn't make excuses, and doesn't position the delay as purely negative, showing emotional maturity and professional growth.

Tell me about the biggest mistake you've made and what happened next.

[Manager]

CONTEXT

✓ *Eight months into my first engineering manager role, I was leading a team of 12 engineers split between backend and frontend work. We were consistently missing sprint commitments and cross-functional handoffs were creating bottlenecks. After reading **Team Topologies**, I was convinced that reorganizing into full-stack feature teams would solve our velocity problems. The VP of Engineering gave me autonomy to structure my team however I wanted, and with Q4 planning approaching, I felt pressure to show improved metrics to justify our headcount for next year.*

ACTIONS

I spent a weekend designing what I thought was the perfect reorg: three pods of 4 engineers each, organized around customer segments instead of technical layers. I was proud of my color-coded spreadsheet showing how this would reduce dependencies and hopefully improve our sprint velocity.

I announced the reorg in our Monday stand-up, expecting excitement about the increased ownership and autonomy. Instead, I was met with stone-cold silence. Our senior backend engineer's camera "mysteriously" turned off. The meeting ended with polite but tense acknowledgments.

Within two days, the cracks showed. Our most senior engineer requested a 1:1 and told me bluntly that I'd just destroyed three years of expertise development. The frontend engineers were panicking about suddenly being responsible for database design. The backend engineers felt I was devaluing their specialization.

By the end of the week, I realized I'd made a massive mistake. The Product team got wind of some of the feelings and were asking about Q4 delivery. I had to act fast.

I called an emergency team meeting and I admitted I'd made a mistake. I apologized for not involving them in such a major decision and asked for their help to fix it. The room was quiet until our usually reserved junior engineer said, "Well, at least you're admitting it." I spent some time in that meeting outlining more of the motivations for the reorg, to let them into my mind more, and promised we'd work on it together.

Over the next two weeks, I ran a series of collaborative sessions. We mapped out our actual pain points versus what I'd assumed they were. There was a lot of overlap of course, but it also uncovered that handoff issues were probably more about unclear requirements and velocity issues were being affected by technical debt more than team topology.

Together, we designed a hybrid approach: we kept our functional expertise but created rotating "feature champion" roles where engineers would own features end-to-end while still getting support from their functional peers.

I had to have a humble conversation with my VP, explaining that the reorg I'd confidently announced was being modified based on team feedback. I was certain I'd lose credibility, but I focused on what I'd learned about change management and team dynamics.

RESULTS

The collaborative restructure took 6 weeks to implement fully, much longer than my original weekend plan. But the outcomes were far better. Sprint velocity increased within two months. More importantly, team morale recovered and actually improved beyond pre-reorg levels. The employee survey feedback was positive for that quarter despite my mistake.

The feature champion model became a template for two other teams in the organization.

LEARNINGS

This mistake fundamentally changed how I approach leadership. I learned that no amount of theoretical knowledge replaces actually listening to your team. Reading books and attending conferences gives you tools, but your team tells you which tool to use. I now treat any major change as a collaborative design exercise, not a solo architecture decision.

I also learned that admitting mistakes quickly and publicly as a leader builds more trust than never making mistakes at all. The vulnerability I showed in that emergency meeting became a turning point in my relationship with the team. I think that opened the way for them to bring me problems earlier.

My job wasn't to design the perfect team structure; it was to facilitate the team in designing their own structure. This shift from "manager as architect" to "manager as facilitator" has defined my leadership style ever since.

This example follows the CARL structure while demonstrating key management competencies interviewers look for: self-awareness, humility, team leadership, recovery from failure, and organizational influence.

- **[30]Owns a real, substantial mistake.** The story doesn't dodge with a "weakness that's actually a strength" but admits to a serious error that could have caused team attrition and project failure.
- **Shows the human impact of management decisions.** The story explicitly addresses how the mistake affected real people rather than just metrics, demonstrating emotional intelligence.
- **Quick recognition and course correction.** The manager recognized the mistake within days and took immediate action rather than doubling down or hoping it would resolve itself, showing adaptability and pragmatism.
- **Vulnerable leadership in action.** Admitting the mistake publicly to the team and asking for help demonstrates the kind of psychological safety creation that modern tech companies value in managers.
- **Collaborative problem-solving.** Rather than just reverting the decision or imposing a different solution, the manager facilitated the team in designing their own solution, showing evolution from directive to participative leadership.
- **Deep, transferable learnings.** The reflection moves beyond "I should have communicated better" to fundamental insights about leadership philosophy and change management that apply to many situations.

Telling Longer Stories

The responses in the previous section are medium length stories—I clock them at around 3 minutes of speaking time. If you're given a more open-ended question like, "Tell me about your most impactful project," or you're more senior and therefore have more involved stories, the responses could

be longer, assuming you want to cover all the signal you've identified as valuable.

Long stories are hard to deliver and harder to listen to, so you'll need an *organizational system* to stay on track and hold the interviewer's attention. Of course, the Actions section is responsible for the length in these stories, so structure your Actions into *themes* and list those themes right after the Context in what I call a *Table of Contents*. Your listener will know what's coming and you'll be able to signpost your way through, mentioning each theme as you come to it.

[\[31\]](#)

For example, suppose you completed a large backend refactor that involved the following actions:

- Identifying the need for the refactor, based on bugs and slow development speed
- Aligning on a design approach after resolving a conflict with the other senior engineer on the team over architecture and scope
- Implementing the refactor, including having to build your own custom messaging queue
- Rolling out the changes progressively to shift traffic to the new system, including having to fight some unexpected bugs due to blind spots in the design

Telling this story from start to finish using CARL is doable, but will probably get unwieldy to listen to, with the interviewer being unsure of the bigger picture. They will probably interrupt you with follow-up questions, requiring you to resume your story a couple of times.

To help the interviewer stay focused on your longer story while ensuring they extract the necessary signal, we can bucket these actions by theme and provide a surrounding Context, Results, and Learnings for each of the themes. In this way, your story goes from a single CARL to Context, Table of Contents, a number of mini-CARLs, one for each theme, then Results, and Learnings.

Consider pausing after the Table of Contents. This is a natural place to pause to allow for foundational questions from the interviewer and it also helps to avoid the sense that you're delivering a monologue.

Consider front-loading results. If the story is sufficiently long, you may never get to the Results before the interviewer takes you down many follow-up rabbit holes. You can say the Results and even condensed version of the Learnings right at the top.

✓ *I ended up driving an end-to-end refactoring of this system and there were three parts I think were particularly interesting about it:*

TABLE OF CONTENTS

- 1. How I identified and resolved some conflicts with others around the need for the refactor.*
- 2. The technical difficulties I encountered when I took on the implementation.*
- 3. How we structured the rollout to ensure correctness.*

Theme #1 CONTEXT

Since I joined the team about 3 months before this, I noticed the burdensome on-call experience we all endured, much worse than my previous teams. Likewise, the last feature we added took twice as long as expected and had many more bugs than we expected.

Theme #1

ACTIONS

I analyzed the bugs from the last two on-call rotations and cross-checked that with the parts of the codebase that took so long to modify and sure enough, it's all because the backend was pretty much spaghetti code, with feature upon feature added over many years without thought to coupling or testability.

I decided this needed a complete refactor but when I discussed this with the team in a meeting, the senior engineer was pretty adamant about keeping the team focused on the feature work. Later I went and showed him the analysis I did and that helped him see the need for the refactor but we negotiated back and forth over the amount of time to spend on the refactoring work and what some checkpoints might be.

Once we aligned on that, I went about putting together a design to present to the team...

Theme #1

RESULTS

So after all this, I got the team aligned on spending the time to do the refactor and on the overall technical approach.

Theme #1

LEARNINGS

This methodical process of collecting data and convincing key people on the team, really paid off in not having the project questioned later when we ran into some bumps.

Theme #2

CONTEXT

So that brought us to the second part of the project, the technical implementation...

This kind of signposting and bucketing not only keeps the interviewer engaged during a long story while still communicating a lot of actions you took—similar to how changing slides in a presentation helps keep the audience engaged—it also shows that you have an organized approach to communication.

For examples of longer stories, see [*Examples of Core Stories*](#).

Adjusting Stories to Fit the Question

While every story can't answer every question, most stories contain multiple angles worth exploring. Think about how Hollywood creates different trailers for the same movie. Imagine only the scenes of *Lord of the Rings* with Arwen, Aragorn, Éowyn, Sam, and Rosie—you might think it's a romantic drama. Or just the scenes with Merry, Pippin, Gimli, and Legolas—maybe it's a comedy. Each trailer pulls from the same source material but emphasizes different scenes, music, and pacing to resonate with different audiences.

[\[32\]](#)

Your behavioral stories work the same way. You can choose which parts of the story to foreground for which question. As an example, consider how the response to, “Describe a project that took much longer than expected” [\[⇒\]](#), could be “re-cut” to respond to other questions:

Tell me about a time you influenced without authority. Center the Context around the 12 teams responsible for the internal tools. In the Actions, emphasize creating the dependency map, running the weekly office hours, and building the adapter layer so teams could migrate at their own

pace. And in the Results, focus on how what you built was adopted by other teams.

Describe a technical decision you made with significant tradeoffs. For the Context, discuss how you discovered that REST and GraphQL migrations had conflicting requirements. For Actions, focus on the analysis of the various options, how you quantified the technical debt cost and built a proof-of-concept to ultimately decide to accept the increase in implementation time. In Results, highlight how the adapter pattern became the standard migration approach for legacy systems. In Learnings, reflect that sometimes the “uglier” technical solution is the right business solution.

Tell me about a time you failed. The Context covers how you confidently told the CEO it would be done for the Q2 board meeting based on naive estimation. The Actions cover the incomplete discovery before committing to a timeline and the lack of identifying stakeholders until things started breaking, then how you brought a clear choice and got buy-in to recover from your mistake. In the Results, highlight your takeaways for future estimation phases.

[\[33\]](#)

Hopefully it’s clear how you can use the same story to address different questions and how the work you will do in [Step 2: Extracting Story Detail](#) to correlate stories with relevant signal areas really pays off when you need to remix a story.

Responding to Values Questions

So far we’ve discussed stories in the context of “Tell me about a time...” questions, but questions like “Tell me about your approach to cross-team communication” are different. As we mentioned in [Choosing Responses to Values Questions](#), you should first respond with a framework, then an example of applying that framework using the CARL method:

Tell me about your approach to cross-team communication.

✓ *I think about coordinating and communicating across teams in three layers based on how frequently we need to communicate:*

Layer 1 is tight-loop communication with teams I directly depend on or who depend on me. With these teams, I establish shared Slack channels, attend each other’s sprint reviews, and maintain a living document of

integration points. The goal here is near-real-time awareness of blockers and changes.

Layer 2 is structured touchpoints with adjacent teams who might be affected by our work or whose work might affect us. I schedule monthly syncs, send out concise weekly status emails focusing on upcoming changes, and maintain a public roadmap. The goal is predictability without overwhelming them with detail.

Layer 3 is broadcast communication for the broader organization. This includes quarterly tech talks about what we've built, documentation in our wiki, and architecture decision records. The goal is discoverability—people should be able to find out what we're doing when they need to know.

CONTEXT

As a recent example, we were migrating from Stripe to an in-house payment system to save \$100K annually in processing fees. I was leading the technical execution across 4 engineering teams, with downstream impacts on customer support, finance, and compliance. The migration would touch 15 microservices and needed to handle millions in monthly transactions with zero downtime. The CEO had announced this cost savings to investors, so failure wasn't an option.

ACTIONS

I immediately implemented my three-layer communication framework to avoid coordination failures:

For Layer 1, tight-loop communication, I identified the three teams directly building the migration: payments, order management, and customer data. We created a shared Slack channel with custom alerts from our staging environment, had standups together every morning, and maintained a real-time migration dashboard. When the payments team discovered Stripe webhooks we hadn't documented, the order team knew within 10 minutes through our shared channel, not days later in a sprint review.

For Layer 2, structured touchpoints, I set up weekly 30-minute syncs with finance, customer support, and the mobile team. Finance needed 2 weeks' notice before cutover to adjust reconciliation processes. Customer support needed runbooks for new error messages. I sent Thursday afternoon emails with a consistent format: "This week's progress | Next week's changes |

Risks to watch.” When we had to delay Phase 2 by a week, these teams heard it from me first with mitigation plans, not through the rumor mill.

For Layer 3, broadcast communication, I published bi-weekly architecture decision records that any engineer could read, recorded a tech talk about our migration strategy, and maintained a FAQ in our wiki. This paid off when the data warehouse team discovered our plan and flagged that our new system would break their ETL pipeline, something we’d completely missed.

[\[34\]](#)

I also shifted communication around as needed. I temporarily promoted the mobile team from Layer 2 to Layer 1 for 48 hours while we fixed a race condition we found, since they’d need emergency client updates if we had to rollback. I also sent a Layer 3 broadcast to all engineering about the issue and fix, which prevented two other teams from introducing similar bugs in their own services.

Throughout the migration, I adjusted communication intensity based on risk. During the actual cutover weekend, Layer 1 teams were on a continuous Zoom call, Layer 2 teams got hourly updates, and Layer 3 got a summary email Monday morning.

RESULTS

We completed the migration with only 3 minutes of payment degradation. Zero duplicate charges, zero lost transactions. The clear communication structure meant customer support knew how to handle any questions that came up. Finance was able to reconcile perfectly on day one because they’d had weeks to prepare.

The infrastructure team lead told me it was the smoothest cross-team project she’d ever been part of, specifically because she always knew when and how she’d get updates.

LEARNINGS

I learned that anxiety in large projects often comes from information uncertainty, not technical uncertainty. Teams can handle bad news if they trust they’ll hear it quickly and directly. The Layer 2 teams actually appreciated getting less real-time information than Layer 1—it let them focus on their own work while staying informed.

Responding to Hypothetical Questions

Similar to Values questions, with Hypotheticals it can be difficult to know what the interviewer wants to hear or what an ideal answer might be. All you can do is showcase your problem solving, use structured thinking skills, and back them up with relevant examples from your career.

If the question was, “Suppose you had a project that was behind schedule, what would you do,” here’s a structure you can employ when responding:

1. **Clarify critical context.** You’re unlikely to have all the necessary information just from the prompt. Don’t make assumptions or jump to solutions. Ask targeted questions that help you understand the constraints and stakes. Try questions like:
 - *Is this a customer commitment or an internal deadline?*
 - *Do I have flexibility on scope or resources?*
 - *What’s the impact if we miss this date?*
- Keep this brief, like 30–60 seconds, as too many questions makes you seem indecisive or evasive. The goal is to show you understand that context shapes solutions, not to exhaustively map every detail.
2. **Outline a framework or approach.** Similar to Values questions, structure your approach around what’s important for success. Start with how you’d diagnose the situation: *I’d first need to understand why we’re behind—is it technical complexity, scope creep, or resource constraints?* Then explain your systematic approach: *Within 24 hours, I’d assess three things: the true gap between current state and target, the root cause of the delay, and what tradeoffs are available.* This shows you won’t panic or make assumptions, but instead apply a repeatable methodology.
3. **Present 2–3 likely scenarios and your responses.** Rather than forcing a single answer, demonstrate nuanced thinking by bucketing probable situations. *If it’s underestimated complexity, I’d stop adding features and timebox an exploration. If it’s scope creep, I’d force a prioritization conversation with tradeoffs. If it’s team velocity, I’d diagnose whether we need process improvements or additional*

expertise. This approach shows you understand problems are multifaceted and solutions must match root causes.[\[35\]](#)

4. **Ground it with a real example.** Ideally you have some relevant career story to share in a short CARL format.

5. **Finish strong.** End with forward momentum rather than trailing off.

- Share what you'd do next: *After implementing the immediate fix, I'd run a retrospective to prevent recurrence.*
- Mention common pitfalls: *The biggest mistake I see is throwing more people at late projects without fixing the underlying cause."*
- Or crystallize your approach into a principle: *I've learned that when projects are behind, the highest-leverage move is often to stop and change approach, not just change intensity.*

—

By applying these basic storytelling techniques, your stories will be engaging and relevant, highlighting your repeatable actions connected to the signal areas the interviewer is assessing. There are more advanced storytelling techniques in [Part 3: Refining Responses](#) .

Signal Areas

Scope

The overall level of impact, complexity, and seniority you've demonstrated in your work.

Ownership

Taking initiative to identify problems and drive solutions, demonstrating results-oriented thinking.

Ambiguity

The ability to break down large, unclear problems and make progress despite incomplete information.

Perseverance

Sustained effort through difficulties and maintaining motivation when projects encounter setbacks.

Conflict Resolution

Productively navigating disagreements while preserving relationships and driving toward solutions.

Growth

Learning from mistakes, seeking feedback, and helping others develop their skills.

Communication

Clear, effective communication across different audiences and mediums, adapting your approach to context.

Leadership

Influencing others and driving outcomes commensurate with your level, regardless of formal authority.

[1] *The validity and utility of selection methods in personnel psychology*, Schmidt et al., 1998. [↑](#)

<https://thebehavioral.tech/p/selection-methods-research>

[2] *A Review of Structure in the Selection Interview*, Campion et al., 1997. [↑](#)

<https://thebehavioral.tech/p/structured-interview-research>

[3] We've written all this from the perspective of a thoughtful hiring team that's seeking to be excellent in how they evaluate candidates. Check out [Handling Difficult Interviewers](#). [↑](#)

[4] For more example questions, check out [Appendix: Question Repository and Scripts for Mock Interviews](#). [↑](#)

[5] See [How Interviewers Assess You: Behavioral Questions](#) for the definitions of these question types. [↑](#)

[6] See [Decode: Understanding the Question](#). ↑

[7] *Choose recent stories for wins, older stories for weaknesses*. Brian Kemler, PM Lead (Ex-Google, Ex-Meta) ↑

[8] LLM: Large Language Model ↑

[9] See [\[→\]](#). ↑

[10] See . ↑

[11] See [Responding to Values Questions](#) for an example of how to deliver this framework with additional detail. ↑

[12] Be confident when you present your framework.
Don't start with, "I just came up with this..." etc. ↑

[13] B2B: Business to Business
B2C: Business to Consumer ↑

[14] 20 years of the default mode network: A review and synthesis.
Menon, 2023. ↑

<https://thebehavioral.tech/p/brains-storytelling-dmn>

[15] Narrative Transportation: How Stories Shape How We See
Ourselves and the World. Green & Appel, 2024. ↑

<https://thebehavioral.tech/p/brains-storytelling-attention>

[16] Charting the rise of imaginary worlds in history. Dubourg, et al.,
2025. ↑

<https://thebehavioral.tech/p/brains-storytelling-cultural-history>

[17] *The causal structure and computational value of narratives*. Chen et al., 2024. ↑

<https://thebehavioral.tech/p/brains-storytelling-cause-effect>

[18] *A critique of grammar and a focus on process*. de Beaugrande, 1982. [↑](https://thebehavioral.tech/p/brains-storytelling-grammars)
<https://thebehavioral.tech/p/brains-storytelling-grammars>

[19] *Cognitive Structures in Comprehension and Memory of Narrative Discourse*. Thorndyke, 1977. [↑](https://thebehavioral.tech/p/brains-structures)
<https://thebehavioral.tech/p/brains-structures>

[20] As an interviewer, I don't recommend forcing a certain response structure. See my thoughts in [Appendix: How to Conduct Behavioral Interviews](#). [↑](#)

[21] See [Tuning the Level of Detail](#). [↑](#)

[22] See [Adjust detail according to type of question](#). [↑](#)

[23] See [Step 2: Extracting Story Detail](#). [↑](#)

[24] ROI: Return on Investment [↑](#)

[25] MVP: Minimally Viable Product [↑](#)

[26] EM: Engineering Manager [↑](#)

[27] ARR: Annual Recurring Revenue [↑](#)

[28] REST: Representational State Transfer [↑](#)

[29] TL: Tech Lead [↑](#)

[30] See [Questions About Weaknesses, Mistakes, and Regrets](#). [↑](#)

[31] You'll also be able to bring the interviewer back to your narrative after they get sidetracked with follow-up questions. [↑](#)

[32] Credit to the excellent PM coach Robert Hamilton for this metaphor. [↑](https://thebehavioral.tech/p/robert-hamilton)
<https://thebehavioral.tech/p/robert-hamilton>

[33] In all of these remixes, be sure your listener sees how your

response will answer their question and give them the signal they need early in your story. [↑](#)

[34] ETL: Extract, Transform, Load [↑](#)

[35] Avoid endless hedging. “It really depends...” or “I’d need more information before...” sound like you don’t know what you’re doing. You’ll have to make reasonable assumptions to make progress. [↑](#)

PART 2

PREPARATION ROADMAP

You understand how behavioral interviews work. Here's a step-by-step plan to get ready.

STEP 0: A PLAN FOR PREPARATION

Recall from Part 1 that the flow of a behavioral interview is:

1. Receive a Question: a new question or a follow-up question
2. Decode the question to understand what signal area the interviewer wants to explore
3. Select the appropriate response or story from your career
4. Deliver that response or story in an an engaging way

To effectively perform this Decode, Select, Deliver cycle in a live interview, you need three things:

1. A **story catalog** from your career with enough breadth to provide relevant responses to questions about all signal areas.
2. A prepared handful of **complete responses** to both your most significant stories and a small set of popular questions that are well-structured for listeners to readily extract signal.
3. Enough **practice** with those stories to confidently deliver them and enough experience constructing responses extemporaneously to arbitrary questions, remixing elements from your story catalog in a structured way.

This book takes a progressive approach to helping you achieve this:

[1]

1. Identifying key stories from your career
2. Extracting relevant detail for listeners to derive signal
3. Structuring those details into stories
4. Building responses to three common questions
5. Preparing questions for the interviewer
6. Practicing your stories and responding to arbitrary questions
7. Incorporating reflection and feedback from your actual interviews

It's notable we are not preparing primarily by drilling responses to common interview questions, the way that some software engineers might for coding interviews. Other than the Big Three in Step 4, and some specific

areas related to leadership interviews, it's difficult to accurately predict the questions you'll be asked. Instead, it's better to prepare a catalog that covers the common signal areas, and relevant company values, then use that catalog to select responses.

Even if you think you already have some strong stories identified, I encourage you to read through and follow the process to help you:

- **Clarify which stories are most relevant** from what might be a long list if your career spans decades
- **Extract the most compelling parts** from experiences you've already considered
- **Uncover forgotten details** that strengthen your existing narratives
- **Discover new stories** you hadn't considered interview-worthy
- **Build narrative fluency** by practicing articulating your experiences

STEP 1: IDENTIFYING CAREER STORIES

Let's begin our preparation journey by identifying a working set of stories from your career, building what I call a *Story Catalog*. It's not designed to be exhaustive but to demonstrate the skills, impact, and growth you've experienced.

What We're Building: Your Story Catalog

Your story catalog goes beyond a resume's single-line accomplishments to a structured repository of professional experiences, documented with sufficient depth that you can confidently access the right story for any behavioral interview question and authentically describe your actions and impact—a combination reference and notes for speaking.

I recommend you build a catalog that covers the following:

1. 3–4 *core stories*, representing significant projects—the stories you'd most want to tell a hiring manager—each with a complete story structure you've carefully planned and rehearsed.
2. Planned and rehearsed responses to the Big Three questions: “Tell me about yourself,” “Tell me about your favorite project,” (which would be one of your core stories) and “Tell me about a time when you had a conflict.”
3. *Additional stories* that cover any remaining signal areas not covered by the core stories, or to provide additional examples if asked.

Each cataloged story includes at least the following (see the accompanying table for an idea of where we're headed):

- **Title and Setting:** Story name (mostly for your own recollection purposes), company, timeframe, and your role
- [\[3\]](#) **Context Overview:** The challenges, constraints, and stakeholders involved
- **Action Inventory:** A comprehensive list of what you specifically did
- **Results and Impact:** Measurable outcomes and longer-term effects

- **Learnings:** What you learned and how you'd handle similar situations differently
- **Signal Area Tags:** References to the competencies this story demonstrates, including any company-specific values

[4]

The catalog serves multiple purposes. It ensures you won't get stuck when you need to recall stories about a particular signal area and realize you have three perfect examples—only after the interview ends. It helps you match your strongest evidence to the specific signal areas each company prioritizes. And it becomes a living document you update throughout your career, making future interview preparation infinitely easier.

How We're Building: Progressive Journaling

Instead of jumping directly to this format, it's helpful to *progressively journal* about your stories, starting with brainstorming and moving to deeper excavation of your past. To encourage you to journal, I present some academic results suggesting that structured reflection through writing produces a number of benefits, both in the output and in your mental state:

[5]

Memory enhancement: Research shows that narrative writing helps you remember more details and connections between events, crucial when you need to recall specific actions and outcomes from past projects.

Emotional regulation: Journaling provides mental health benefits by helping process experiences and emotions, making it easier to discuss challenging situations objectively in interviews. Interviewers tend to tune in to, and judge harshly, unprocessed emotions.

[6]

Self-distancing: Expressive writing promotes self-distancing, allowing you to reflect on experiences from a third-person perspective, leading to greater insight and emotional regulation. This objectivity is valuable when analyzing and describing your professional development.

[7]

Professional clarity: Reflection creates clarity about your professional identity, values, and growth trajectory—insights that result in ready-made responses to values questions.

Research suggests that to maximize these benefits, begin by *hand-writing your initial thoughts*. But since having AI-assistance and cross-referenced material is so helpful, *transfer your notes to a digital format* after your initial foray.

Identify Core Stories

Begin by identifying some *core stories*, those experiences that immediately come to mind. These are the projects, transitions, and challenges that have shaped your professional identity. Use these categories to trigger your memory:

High-impact projects: Major launches, significant refactors, system migrations, new product features, architecture decisions that influenced multiple teams.

Challenging situations: Tight deadlines, organizational changes, technical failures, team or interpersonal conflicts, ambiguous requirements, projects where success was uncertain.

Leadership moments: Mentoring others, driving cross-team initiatives, representing your team externally, times when others looked to you for direction.

Learning experiences: Mistakes that led to growth, feedback that changed your approach, skills you developed under pressure, times when you had to completely rethink your approach.

Career transitions: Promotions, job changes, team switches, role expansions, times when your responsibilities significantly evolved.

[\[8\]](#)

Example stories for each level

Just to give you a flavor of the kinds of stories I frequently hear in my interview and coaching sessions, here are common core stories from each level:

Core Stories from L3s (New Grad/Junior)

- Built a small feature from design to deployment
- Fixed a critical production bug that was damaging the business, requiring investigation across multiple services and coordination with senior engineers

- Automated some manual process that was taking the team hours each week
- Led a capstone project in school that involved building an application with a team, managing timelines and technical decisions
- Contributed to an open-source project by implementing a requested feature and navigating the code review process
- Conducted user research interviews and synthesized findings into actionable product requirements for the engineering team [Product Manager]

Core Stories from L4s (Mid-level)

- Initiated, researched, planned, and executed a small project that lasted for a month
- Identified, investigated, and resolved a performance bottleneck that was affecting many users
- Built tooling that reduced deployment time from hours to minutes across multiple engineering teams
- Led the product strategy for a specific user segment, resulting in a new product line that generated significant revenue [Product Manager]
- Established a new development process, reducing time-to-market [Product Manager]

Core Stories from L5s (Senior)

- Led the migration of a legacy system to a modern framework, coordinating with stakeholders and managing rollout risks
- Designed and implemented a new API that improved response times and became widely adopted by other teams
- Resolved a complex technical debt issue that was blocking team velocity, requiring buy-in from team leadership and careful sequencing
- Navigated a complex competitive threat by rapidly pivoting product strategy [Product Manager]
- Established data-driven decision-making processes across the product organization, affecting multiple teams [Product Manager]

Core Stories from L6s (Staff)

- Drove the technical strategy for a product area, influencing roadmap decisions across multiple quarters
- Led the response to a major production incident, coordinating across dozens of engineers and establishing new reliability practices
- Designed and championed adoption of a platform that enabled other teams to ship features 50% faster
- Influenced company-wide architectural decisions, in a medium-sized company
- Drove the development of a new product category that opened up a sizable market [Product Manager]
- Led the product strategy for a major acquisition integration, merging two product lines while retaining users from both platforms [Product Manager]
- Established product management standards and processes across the entire company, scaling the PM organization [Product Manager]

Core Stories from Managers and L7+ (Principal+)

- Led a complete product pivot that required rebuilding team structure and technical architecture
- Scaled the people in an organization significantly while maintaining quality and culture
- Navigated a complex acquisition integration, merging teams and technology stacks
- Established engineering practices and culture that became the standard across the entire company
- Led the complete repositioning of the company's product portfolio in response to market changes, resulting in a successful pivot and renewed growth [Product Manager]
- Drove product strategy for a successful IPO, working with executive leadership to articulate product vision to investors and achieve target valuation [Product Manager][\[9\]](#)

Identify Additional Stories

Core stories will provide coverage for multiple signal areas but often not all the signal areas. To cover any missing areas in your catalog, use the prompting questions below to identify additional experiences. Don't edit yourself or try to follow any kind of story structure during this initial brainstorming. Write down anything that comes to mind, even if it doesn't feel "impressive" enough.

Scope

- What project am I most proud of, and why?
- What's the hardest technical/product/organizational problem I've ever solved?

Ownership

- What's the biggest project I originated and drove myself?
- What decision did I make that significantly influenced a project's direction?

Ambiguity

- When and how did I get started on something without a clear direction?
- When did I have to make a tough prioritization decision?

Perseverance

- When was something delayed or blocked, and how did I handle that?

Conflict Resolution

- When did I most strongly disagree with my manager or teammate?
- When did I have to work with someone difficult, and how did I handle it?

Growth

- What's the biggest mistake I've made, and what happened next?
- When did I have to learn something completely new under pressure?

Communication

- When and how have I convinced others when the stakes were high?
- How have I facilitated conversation between others?

Leadership

- What is the largest success I've experienced influencing others toward business outcomes?
- How have I significantly mentored those around me?

To help you find additional stories, try answering some of the questions in [*Appendix: Question Repository and Scripts for Mock Interviews*](#). Also add questions related to any company values for your target companies; consider using an AI assistant for this.

Ways to Help You Identify Stories

It can be hard to remember stories. Maybe they happened long ago or maybe they were so challenging, you hoped never to think about them again. Here are some ways to extract these stories.

Leveraging past career artifacts

Consider finding whatever artifacts from your career you can access and using those to source stories:

- Performance reviews (especially self-reviews you wrote)
- Resume drafts and LinkedIn profiles from different periods
- Calendars from significant projects
- Old presentations, technical documents, or PRDs/RFCs[\[10\]](#)
- Email threads, Slack messages, etc. from major projects or conflicts
- Code repositories with meaningful commit messages
- Meeting notes or retrospective documents
- Project planning documents or roadmaps

Mapping aspects of the past with others

Existing or former coworkers can be an encouraging resource to help you recall the past. Try interviewing them:

- *What project do you remember me being most involved in?*
- *What do you think I was particularly good at when we worked together?*
- *Can you remind me about [specific project]—what was my role in making it successful?*
- *What's something I did that made a difference for the team?*
- *When you think about my contributions, what stands out?*

Of course you can use similar prompts to the ones you used earlier in the process as well.

Sensory memory triggers

These methods might seem unconventional, but they can unlock forgotten memories:

Environmental Triggers

- Visit past work locations, or whatever similar locations you have access to
- Eat food similar to your past office meals
- Listen to music you had playing during those periods in your life
- Review photos from team events or office spaces

Object Handling

- Look through old conference swag or company merchandise
- Wear the outfits from that period of time
- Page through technical books you referenced during projects
- Review old notebooks or sketchbooks from project planning

What if My Stories Seem Unimpressive?

*Oh, would some Power, the gift give us,
To see ourselves as others see us!*

Robert Burns, *To a Louse*

Many of my coaching clients, especially those from smaller or traditional companies, worry their career stories lack the scale and impact that interviewers at FAANG+ companies expect.

These fears are understandable but generally misplaced. Yes, larger-scale accomplishments do carry weight in behavioral interviews: a story about optimizing a system that serves 500 million users will naturally demonstrate different competencies than one about improving a process that affects a 50-person company. However, this doesn't mean smaller-scale stories are automatically disqualifying.

Hiring managers are looking for employees who can translate ideas and technology into business value, and that means any story where you move through the development lifecycle will be a valuable one. A skilled interviewer evaluates your potential based on the core signal areas—not simply the magnitude of your previous projects.

The real challenge is to articulate the complexity and impact *in terms that resonate with larger-scale interviewers*. Here's how you achieve that:

- Identify repeatable actions that demonstrate strong signal. We'll do this next in Step 2 [\[→\]](#).
- Structure the story so that signal is easy for the interviewer to observe, using the principles from [Deliver: Telling a Good Story](#). We'll do this in Step 3 [\[→\]](#).
- Adjust your language and framing to fit the culture of FAANG+. See [Big Tech Behavioral](#).

The Goal: Raw, Authentic Material

By the end of this identification phase, you should have:

- 3–4 core stories that you're very likely to tell in any behavioral
- 5–7 other stories that cover different signal areas and demonstrate progression in your career[\[11\]](#)
- For each of these, awareness of the business context, results, and stakeholders involved

Remember the goal isn't to find perfect success stories. Often the most compelling behavioral responses emerge from how you navigated

difficulties, learned from setbacks, or handled situations where the outcome was mixed but your approach demonstrated growth.

In the next step, we'll extract the specific actions and details that will form the basis for transforming these experiences into compelling interview responses.

STEP 2: EXTRACTING STORY DETAIL

One of the biggest challenges I see in coaching clients is the lack of expressing relevant details in their stories. Candidates read about the STAR method (Situation, Task, Actions, Results) or CARL (Context, Actions, Results, Learnings) and put together an acceptable story arc but utterly fail at communicating enough valuable signal:

[\[12\]](#)

✗ *We had a tight deadline on a project at my fintech company to add a search filter. I built the filter, after a couple weeks of testing, and shipped it on time. The customers liked it and we hit our metrics.*

The shallow context, quick “I built the feature and shipped it” summary, and vague sense of results tell the interviewer almost nothing about your capabilities. The listener needs to understand your specific contributions, decision-making process, and problem-solving approach.

In this step, let’s return to journaling to extract the relevant details to make stories both meaningful and entertaining. Remember the meat of the journaling you did in Step 1 for each story? You wrote about Context, Actions, Results, and Learnings. Let’s expand on each of these sections as we extract detail.

As you continue journaling, add the takeaways to your story catalog for organization.

Identifying Context

Context serves as the foundation of your story, but most candidates stop at table stakes: company, team, and role. While these basics are necessary, they don’t provide the rich backdrop that helps interviewers understand why your actions mattered or the complexity of the challenges you faced.

The most critical aspect of context is *establishing the stakes*. This is about describing why the work was important to the organization and its customers. A feature that prevents customer churn carries different weight than a nice-to-have enhancement and interviewers need to understand that distinction to properly evaluate the scope and impact of your work.

- [\[13\]](#) What business problem were you solving?

- How would the world be better—or what disaster would be averted—if you completed this project successfully?
- How did this project fit into the broader company strategy?

Context becomes especially crucial for stories with emotional arcs—conflict stories, weaknesses, failures, and high-stakes situations where relationships and judgment are evaluated. In these narratives, seemingly minor details can help interviewers understand the interpersonal dynamics at play.

- When and where did critical conversations happen? Was it a heated exchange in a public meeting or a careful one-on-one discussion in someone's office?
- Who were the key stakeholders involved, what were their motivations, and how did organizational politics influence the situation?

The constraints you operated under often reveal as much about your capabilities as your actions themselves, providing the interviewer with a sense of the difficulty of your accomplishment.

- Were you working with a skeleton crew during a hiring freeze?
- Did you have to navigate complex compliance requirements?
- Was there pressure from a major customer or a competitive threat?

Don't forget to capture what made your situation uniquely challenging or interesting. These details help interviewers understand the sophistication of thinking required to navigate your specific circumstances.

- What alternative approaches were considered or rejected, and why?
- What organizational dynamics or technical complexities made this different from a typical project?

This might seem like a lot of detail and it is—but you won't share all this detail every time you tell the story. The goal is to identify the detail so it's available when the question calls for it.

Identifying Actions

Interviewers are looking for evidence of your capabilities, so they care less about what happened and more about what you did to make it happen. Interviewers are looking for you to list as many relevant actions as possible, assessing whether you can repeat your results in their organization.

Brainstorm the actions you took, including key inflection points that wouldn't have happened without you. Include all the moving parts: ideation, planning, approvals, architecture reviews, technical difficulties, testing, rollout, and follow-up actions.

Here are some actions, grouped by themes, you might have engaged in:

Designing encompasses your product and architectural decisions. System design choices, user experience decisions, process design, go-to-market strategies, and API specifications all fall here. Even choosing between competing product or technical approaches represents a significant action.

- How did you gather the needed information?
- What alternatives did you consider, and how did you choose?

Aligning captures the interpersonal work of building consensus and managing stakeholders. This includes negotiating priorities and tradeoffs, managing conflicting requirements, and coordinating across functions.

- How did you build consensus among stakeholders with different priorities?
- What meetings did you organize or conversations did you initiate?
- How did you handle conflicting requirements from different teams?

Communicating covers both the formal and informal ways you shared information and influenced others. Technical documentation, leadership presentations, difficult conversations, and translating complex concepts for different audiences.

- How did you communicate progress or setbacks to others?
- How did you present technical concepts to non-technical stakeholders?
- What difficult conversations did you initiate or facilitate?

Implementing includes your direct technical execution and project management. Resource allocation, technical implementation, and risk mitigation strategies demonstrate your ability to execute on plans.

- What did you do that another person in your role might not have done?
- What were each of the challenges or turning points and how did you overcome them or influence the outcome?

Iterating shows your ability to improve and adapt based on feedback. Gathering user feedback, measuring results, making course corrections, and driving continuous improvement.

- How did you gather feedback during the project?
- What metrics did you track and how did they influence your decisions?
- What course corrections did you make based on new information?

Testing and Debugging reveals your approach to quality and problem-solving. Quality assurance processes, problem diagnosis, root-cause analysis, and performance optimization.

Releasing demonstrates your end-to-end ownership. Deployment strategies, monitoring and observability, rollback planning, integration with marketing, sales, and customer success, and post-launch support.

- How did you handle the deployment and what safeguards did you put in place?
- What happened in the initial time after launch and how did you respond?

Even **Thinking** and **Deciding** are actions. The cognitive work you did—analyzing options, weighing tradeoffs, making strategic decisions—represents valuable signal for interviewers.

As a final pass, go through the signal areas and highlight places where you demonstrated any related behaviors you missed.

Identifying Results

Results demonstrate the impact of your actions, but many candidates struggle to articulate their achievements, especially if they don't have numbers.

Of course, quantifiable results are preferable and provide concrete evidence of your impact: revenue generated, costs saved, and process efficiency gains all demonstrate business value. Performance improvements in speed, reliability, or efficiency demonstrate technical excellence. User metrics like adoption rates, engagement improvements, or satisfaction scores show customer impact. Team and organizational metrics can be equally compelling. Productivity gains, velocity improvements, or retention increases show your ability to improve how teams work.

- What measurable improvements did your work create?
- How did key metrics change because of your actions?
- What revenue impact (positive or negative) resulted from your work?

Non-quantifiable results can also carry significant weight, especially for senior roles. Increased personal or team scope shows career progression and growing responsibility. Growth in people—new skills developed, responsibilities gained, promotions earned—demonstrates your ability to develop others. Process improvements adopted by other teams, technical debt reduction, knowledge sharing initiatives, and cultural changes all represent systemic impact that extends beyond individual projects.

- Did team members gain new capabilities or responsibilities because of your work?
- What processes did other teams adopt based on your approach?
- How did relationships or team dynamics improve?

When You Don't Have Numbers

First, see if you can find the numbers—look through your career artifacts and ask your former coworkers. If they're really not available, provide a sense of impact through order of magnitude comparisons. “Dozens of customers” versus “hundreds” versus “thousands” gives interviewers a

frame of reference. Relative, even estimated, improvements like “roughly 50% faster than our previous approach” or “reduced the process from weeks to days” show data-driven thinking effectively.

Sometimes the most powerful results come through the eyes of others. Quotes from managers, customers, or colleagues can provide compelling evidence of impact. “My manager said this was the smoothest launch they’d seen,” or “The customer called it a game-changer,” carries weight because it represents external validation.

- What feedback did you receive from stakeholders about the impact?
- How did customers, colleagues, or leadership respond to your work?
- What doors did this project open for future work or opportunities?

Don’t forget downstream effects and longer-term impacts. Work that unblocks other teams, enables future projects, or creates new capabilities often has value that extends far beyond the immediate deliverable.

- What other projects became possible because of your work?
- How did this change the trajectory of your team or organization?
- What capabilities did this create for future initiatives?

 *The project went well and everyone was happy with the results. It made things better for the team.*

 *The automation reduced manual work from hundreds of tickets per week to dozens, freeing up the support team to focus on complex customer issues.*

 *Our VP called it, “The most impactful product improvement in two years,” and used our approach as the template for other teams.*

Identifying Learnings

Learnings demonstrate your ability to reflect, grow, and apply wisdom to future situations. This is particularly important for senior roles where self-awareness and continuous improvement are key differentiators.

Technical learnings show your commitment to craft and continuous improvement. New technologies, methodologies, debugging techniques, or performance optimization strategies all demonstrate your ability to expand your technical toolkit.

- What new technical approaches did you discover or master?
- How did this project change your approach to system design or implementation?
- What tools or techniques did you adopt that you still use today?

Process learnings reveal your ability to improve how work gets done. Better project management approaches, communication strategies, stakeholder management techniques, and risk mitigation methods show operational maturity.

- How did this experience change how you plan or execute projects?
- What did you learn about working with different types of stakeholders?
- What process improvements did you implement on future projects?

People learnings demonstrate emotional intelligence and leadership growth. Insights about team dynamics, motivation strategies, conflict resolution approaches, and influence tactics show your development as a leader and collaborator.

- What did you learn about motivating or managing different personality types?
- How did this change your approach to difficult conversations?
- What insights did you gain about building trust or resolving conflicts?

Strategic learnings show business acumen and systems thinking. Understanding market dynamics, competitive positioning, resource allocation, and organizational behavior demonstrate your ability to operate at higher levels.

- What did you learn about how your decisions impact business outcomes or competitive positioning?
- How did this experience change your understanding of organizational dynamics or decision-making?
- What insights did you gain about balancing competing priorities or resource constraints?

- What did you discover about the hidden dependencies or ripple effects across different parts of the system or organization?

Be careful with mistakes

[\[14\]](#)

When identifying mistakes, be strategic about which ones to share and how to frame them. Choose mistakes that show skill growth rather than character flaws. Frame them as learning opportunities that led to better approaches, demonstrating self-awareness and reflection rather than recurring patterns.

- What assumptions did you have that turned out to be wrong?
- What would you do differently if you could repeat this project?
- How did this experience help you avoid similar problems in the future?

The most compelling learnings show clear application to future situations and it's best to have examples of those future situations. Abstract insights are less valuable than proven changes.

- How did this experience change your approach to similar challenges?
- What advice would you give to someone facing a similar situation?
- How did you apply these learnings to subsequent projects or decisions?

STEP 3: SCAFFOLDING STORIES

You have a list of both core stories and supporting stories from your career, and you're armed with the relevant details to help interviewers see the available signal from your actions. It's time to write complete stories.

A quick note on why I recommend you prepare at least some complete stories:

- Unless you have oratory or debate experience, successfully organizing and delivering an extemporaneous story that covers all the desired signal, under pressure, is non-trivial.
- Complete stories can be shared with others (including LLMs) for feedback and iteration.



Of course you won't read the stories verbatim to the interviewer, but you'll have thought about what to say, about the transitions and key elements, and be much more satisfied with your performance if you scaffold stories in advance.

Build CARLs for Your Core Stories

Now is the time to write complete responses for your core stories. Tackle this in any order you like, but I recommend “inside out,” focusing on Actions first:

1. Organize a **Table of Contents**. If you are senior or above, your core stories will be long, so begin by deciding how to structure your delivery.[\[15\]](#)
2. Write the **mini-CARL stories for each theme** in your contents.
3. Write the minimal **Context** for the overall project, being sure to set the stakes high enough.
4. Finish with the **Results and Learnings**.

These steps are mostly straightforward, but let’s cover organizing a Table of Contents in more depth.

Organizing a Table of Contents

I typically see candidates choose one of two options to structure their Actions into digestible themes:

1. **Chronologically**. Most projects have distinct phases, and if you were deeply involved in the project, you’ll likely have key actions in each phase. A story told from beginning to end is easy to understand.

✓ *There were a number of phases to this project:*

- *How I initiated this project after collecting research data,*
- *How I got alignment from leadership to fund it with a presentation “road show”,*
- *Working with the TL to design around complex constraints*
- *The implementation phase where I had to solve some complex technical issues, and*
- *A careful rollout phase that needed to be coordinated with multiple stakeholder teams...*

2. **Thematically**. Two risks exist with a chronological organization: (1) the most salient signal might be toward the end of the project, so you

risk getting interrupted or derailed before getting to it, and (2) you might want to land well-defined messages about who you are and it's better to group related Actions together, even if they aren't adjacent in time.

✓ *I was involved in a number of inflection points along the course of this project:*

- *Product contributions that ensured the idea was relevant to users and the go-to-market strategy succeeded,*
- *Resolving critical conflicts with leadership, Engineering, and Design, and*
- *Prioritization tradeoffs I made at various stages to ensure velocity and minimize engineering time...*

Include takeaways for your themes

If you look closely at the examples above, you'll see that not only does the candidate list what they're going to talk about, but they also provide the conclusions about themselves they want the listener to take away. Simply listing the themes, while helpful to organize the conversation, misses an opportunity to provide reasons to hire you:

✗ *There were a number of phases to this project:*

- *Its initiation,*
- *Alignment road-show,*
- *Technical design,*
- *Implementation, and*
- *Rollout...*

Check Story Quality

After you complete a story, run a few quality checks:

- Read the story and see if you can identify the key takeaways, referencing the areas from [What Interviewers Assess: Signal Areas](#). Better yet, have someone you know read the story and do the same. Do those map to what you hoped to convey?

- Review the Context and be sure it includes business motivation for the project but is not too long. Are all the details in the Context relevant to the story in some way?
- Does the Table of Contents tell the interviewer what they should conclude about you from each section?

Examples of Core Stories

Let's review some complete core stories at different career points. Note that, as presented, these are pretty long—longer than you would likely talk in an interview. I would recommend you pause after the Table of Contents and leave room in the conversation for follow-ups.

Example: An Early Career Designer

CONTEXT

I was working as a junior product designer at a fintech startup, and our mobile app had a major usability problem—users were abandoning the account setup flow at a 60% rate. The existing flow had seven screens and required users to upload multiple documents before they could even see what the app did. The product team suspected the flow was too long, but we didn't really understand where users were getting stuck or why since collecting all the data and making recommendations was part of the value prop for the app.

TABLE OF CONTENTS

In this project, I got to conduct user research with real customers, collaborate closely with engineering on technical constraints, and advocate for design decisions based on data rather than assumptions.

ACTIONS

I proposed doing user testing to understand the real pain points before redesigning anything. I recruited eight users who had recently abandoned the flow and conducted moderated usability sessions over video calls. What I discovered surprised everyone—users weren't frustrated by the length, but by the lack of clarity about why we needed certain information and what would happen with their documents.

Based on these insights, I redesigned the flow with progressive disclosure and clear explanations at each step. I also had to work closely with our iOS developer to understand what was technically feasible. My initial design assumed we could auto-populate certain fields, but the APIs we were using

didn't support that. We iterated on a solution that balanced user experience with technical constraints.

I also had to advocate for my research findings when the product manager initially wanted to just remove screens to make the flow shorter. I had to point them to the recorded user sessions to demonstrate that explanation, not brevity, was the real issue.

RESULTS

The new flow reduced abandonment from 60% to 45%. Customer support tickets related to account setup dropped significantly, and user feedback scores for the onboarding experience improved from 2.1 to 3.8 out of 5.

LEARNINGS

I learned that assumptions about user behavior are often wrong, even when they seem logical. I also discovered that showing actual user feedback is much more persuasive than design arguments when advocating for solutions. Finally, I realized that involving engineering early in the design process leads to better solutions than trying to hand off perfect mockups.

Example: A Mid-level Product Manager

In this project, I got to lead the development of a new AI feature from concept to launch, build and test early prototypes with real users, and pivot our technical approach based on unexpected user behavior insights.

TABLE OF CONTENTS

CONTEXT

I was the PM for our customer support platform, and our support team was drowning in repetitive questions about account setup, billing, and basic troubleshooting. We were getting over 500 tickets per day, with about 60% being questions that could theoretically be answered by our existing help documentation. The leadership team wanted to explore adding an AI chatbot to deflect some of these routine inquiries and reduce support costs.

ACTIONS

Rather than building a full chatbot immediately, I proposed starting with a lightweight prototype to validate our assumptions about user behavior. I worked with one of our engineers to build a simple rule-based bot using our existing knowledge base, which we deployed to a small segment of users as a proof of concept.

The initial results were disappointing—only 12% of users who interacted with the bot got their questions resolved, and many users bypassed it

entirely to submit tickets anyway. I realized we had assumed users would ask direct questions, but they were actually describing complex scenarios that our simple keyword matching couldn't handle.

I partnered with our ML engineer to rebuild the approach using a more sophisticated NLP model that could understand intent and context. This required me to coordinate among engineering, customer support (who helped us categorize common question types), and our legal team (who had concerns about AI providing incorrect information).

[\[16\]](#) [\[17\]](#)

I also established a feedback loop where unresolved bot conversations were automatically routed to human agents, who could flag when the bot missed something important. This data helped us continuously improve the model.

RESULTS

The final chatbot successfully resolved 45% of user inquiries without human intervention, reducing our support ticket volume by about 30%. Customer satisfaction scores for quick resolution improved, and our support team could focus on more complex issues. The feature became a key selling point for new customers.

Through this project, I learned that AI features require much more iteration than traditional features—the first approach rarely works as expected. I also discovered the importance of building feedback loops into AI products from day one, since the quality depends on continuous learning. Finally, I realized that cross-functional collaboration is especially critical for AI features because they touch so many different areas of expertise.

LEARNINGS

Example: A Staff Engineer

CONTEXT

I was on the Data Platform team at a mid-sized marketing analytics SaaS company. Our customer data processing system handled importing data from various sources, transforming it, and making it available for analysis—about 2 million records per day for 5,000 enterprise customers.

I led the migration of our customer data processing system from a monolithic Rails application to a microservices architecture. The legacy Rails monolith was causing severe business problems. Sales was losing enterprise deals worth \$4M per quarter because prospects couldn't import

their large datasets—the system would either fail or take days. Customer Success was spending 30 hours weekly on data-related escalations, and we'd already lost two customers over reliability issues. Just to top it off, Product couldn't ship the AI-powered features that were our core competitive strategy because the architecture couldn't support real-time data processing.

This project had three interesting parts:

TABLE OF CONTENTS

- 1. Discovery and stakeholder alignment, how I identified the business impact and built consensus across engineering, sales, product, and customer success.*
- 2. Technical execution and architecture decisions, where there were some tradeoffs to navigate.*
- 3. Rollout strategy and risk mitigation, how we migrated without downtime.*

ACTIONS

I'm presenting it like this, but it wasn't that clear to everyone at the time. I was working on infrastructure support for the AI features and running into performance issues with the pipeline, so I decided to assess what other issues the monolith was causing.

I had heard we lost some enterprise deals due to technical limitations so I contacted Sales. They told me they lost four recent deals, which would have been millions in revenue.

I also analyzed six months of support tickets and calculated that data-related escalations consumed 15% of engineering capacity plus direct customer support time—maybe \$400K annually in wasted capacity.

So with this data collected, I went around to some stakeholders to get the needed time and resources. I presented the data to the VP of Engineering in a meeting I called with him—he was easy to convince given the technical limitations.

The VP of Product was harder to convince. She had a full feature roadmap, but I showed her that three of her top Q3 priorities were technically impossible without this migration, including the AI features she'd committed to the CEO. We negotiated a 60/40 capacity split—60%

migration, 40% features—and I committed to unblocking one priority feature by month 3, not making her wait until the end.

All these people wanted to see a timeline, which made sense. I structured the project to deliver incremental business value monthly rather than a big-bang cutover:

Months 1–2: Migrate import pipeline, to enable enterprise POCs

[\[18\]](#)

Month 3: Enable real-time processing, to unblock Product feature

Month 4: Migrate transformations, to reduce support load

Month 5: Migrate exports, to improve performance

Month 6: Complete cutover

This meant Sales could start testing with enterprise prospects by month 3, and Product would get a feature unblocked at the halfway point.

I sent this to the stakeholders, they thought it was reasonable, and we set up an every-other-week meeting to keep them informed on the progress.

Then came the technical design and implementation. I led the design with two other senior engineers. We designed six microservices (ingestion, validation, transformation, storage, export, monitoring) with requirements driven by business needs: handle 100M+ record imports, fail independently so one customer can't impact others, and expose real-time data streams for AI features.

The biggest debate was around data consistency. I proposed event sourcing with Kafka for consistency and replay capability. One senior engineer pushed back since our SRE team had limited Kafka experience. I thought we could build POCs of both approaches and test with a simulated 50M record import. The event sourcing approach handled failures much more gracefully and gave us the debugging capabilities we'd need for customer issues. To address operational concerns, I talked to the SRE manager to get some Kafka training.

[\[19\]](#)

The legacy system had a single PostgreSQL database with 200+ tables. I proposed a “strangler fig” pattern where new services would initially read from the monolith's database while writing to their own, then gradually shift reads over. I worked with our DBA to sequence the migration based on foreign key dependencies. The critical constraint was zero service degradation during business hours, so I scheduled all database migrations for the middle of the night with automated rollback procedures.

[20]

I'd committed to unblocking a Product feature by month 3. I worked with a product engineer to build "real-time audience segmentation" on the new event-streaming architecture, time-boxing it to 2 weeks. The feature launched to beta and generated immediate positive feedback. The VP of Product shared this with the CEO as proof the migration was delivering value, which built credibility for the rest of the project.

Four months in, we discovered a subtle bug in event ordering that caused data corruption for three customers. I immediately rolled back those customers to the legacy system and spent a week debugging—the issue was using application timestamps instead of Kafka's internal ordering. I fixed it, built a reconciliation process, and personally called each affected customer to apologize. Two escalated to our VP of Engineering.

I turned this into an improvement opportunity by building chaos testing to catch race conditions earlier and adding automated integrity checks that would alert within 15 minutes of any corruption. I also pointed out that the new architecture gave us much better debugging and recovery tools than the legacy system.

After five months, we had six microservices running in production handling 40% of customer load, fully instrumented with monitoring, feature flags, and automated reconciliation.

It was time for the rollout.

I worked with Customer Service to plan communications. They were nervous given past incidents so I created a tiered rollout starting with internal accounts, then technical customers who could provide feedback, then progressively larger segments. For each tier, I worked with Customer Service to draft communications explaining what was changing.

During rollout to our second tier, one of our largest customers reported exports taking 3× longer. This escalated immediately to executives. I jumped on a call with their technical team, our account manager, and our VP of Engineering. Through debugging, I found the new export service was optimized for median dataset sizes but this customer had 10× the volume.

I implemented a hot fix and deployed it the same-day, and brought export times to 50% of the original legacy system. I apologized directly to the customer and explained the long-term benefits. They weren't happy, obviously, but appreciated the honesty.

For the final phase, I coordinated with Product, SRE, and Customer Support, ran a pre-mortem exercise, which helped us make sure we felt confident to pull the trigger.

We successfully migrated 100% of traffic. The legacy system was decommissioned after a 30-day observation period.

RESULTS

System latency dropped multiple seconds. Production incidents decreased 85%. Infrastructure costs decreased 30%. Engineering velocity increased measurably—features that took 6 weeks now shipped in 2–3 weeks.

For the business, Sales closed two enterprise deals worth a bit over \$1M in the following quarter that would have been impossible before. Customer escalations dropped 70%, saving approximately \$280K annually. The architecture enabled us to ship three major AI features that became key competitive differentiators.

LEARNINGS

This was a pretty long project and I learned a lot through it. Building stakeholder alignment was as important as technical execution. Framing the project in terms of business value—lost revenue, support costs, blocked features—rather than technical elegance was crucial to getting buy-in and maintaining support when we hit bumps.

Incremental migration beats big-bang rewrites. The phased approach let us deliver value continuously and catch issues when they impacted fewer customers.

Cross-functional relationships matter during crises. The trust I built with Sales, Product, and Customer Service by involving them early made it much easier to handle the data corruption incident and the customer escalation. They gave me the benefit of the doubt because they understood the project and saw the progress.

Because of the scope and impact here, I was promoted to Staff Engineer.

Example: A Story from School

CONTEXT

For my senior capstone, our team of four was tasked with building a web application for the campus food pantry to help students find and reserve food items. The existing system was entirely paper-based, causing long wait times and inventory confusion.

TABLE OF CONTENTS

In this project, I got to do user research with real stakeholders, resolve a conflict between team members over technical decisions, and make architecture tradeoffs while leading the technical implementation.

ACTIONS

I took the lead on requirements gathering by interviewing food pantry volunteers and students to understand their pain points. Based on these conversations, I discovered our initial plan for a simple inventory display wouldn't handle the real problem—students would still show up for items that were already taken.

I proposed we build a reservation system using React for the frontend and Node.js with PostgreSQL for the backend. This created a conflict with two teammates who wanted to use Django since that's what they'd used before. I suggested we spend a week prototyping both approaches—I built a quick React demo while they worked on Django. When we compared the results, the React version handled real-time updates more smoothly, which was crucial for inventory tracking.

We then collaborated to write up a PRD for this project, including some light technical design ideas.

For the technical implementation, I architected the database schema and built the real-time inventory system using WebSockets. I also set up our deployment pipeline using Heroku and handled the integration with the food pantry's existing Google Sheets workflow. When we ran into performance issues during testing with multiple concurrent users, I optimized our database queries and added basic caching.

RESULT

We delivered the system on time, and food pantry staff reported shorter wait times and better inventory accuracy. Students could now see real-time availability and reserve items ahead of time. Our professor selected our project to demo to incoming students.

LEARNINGS

I learned that talking to real users early prevents building the wrong thing. I also discovered that technical conflicts are often best resolved through prototyping rather than debate, and that I enjoy the challenge of making architectural decisions under constraints.

Example: A Personal Story

Please review [Junior candidates: when no work-related stories are available](#) before choosing to tell a personal story. Namely, you should only do this if you are a junior candidate and can't identify a work-related story.

CONTEXT

I'd been living in my apartment complex for about a year and noticed that neighbors barely knew each other, even though we all shared the same courtyard space. After a few package thefts and some tension over parking, I thought bringing people together might help build community and address these issues naturally.

ACTION

I started by talking to a few neighbors to gauge interest and quickly realized people wanted to connect but no one wanted to take the lead on organizing something. I volunteered to coordinate a block party, even though I'd never planned an event before.

I began by going door-to-door to understand what people would actually want—some families with kids wanted activities for children, while others were more interested in just having a chance to chat.

When we hit a roadblock with the city permit process, I had to research alternative approaches and discovered we could use the complex's community room instead of blocking the street.

RESULT

About 40 neighbors showed up, including several families who'd lived there for years but had never met. We ended up having regular quarterly gatherings after that, and the package theft issues largely disappeared. People started looking out for each other's apartments and helping with things like pet-sitting.

LEARNINGS

I learned that most people want to participate in community building but need someone to take the first step and handle the logistics. I also discovered that starting small and simple is better than trying to plan the perfect event—we could always improve future gatherings based on what we learned.

Identifying Follow-Up Questions

Now that you have the stories written, it's a good opportunity to prepare answers to common follow-up questions. Spend some time journaling the

answers to these:

About the Challenge

- What was the hardest part?
- What constraints were you working under?

About Your Role

- What were your specific contributions?
- How much of this was your idea vs. assigned work?
- Who else was involved and what did they do?

About Decisions

- How did you make that decision?
- What alternatives did you consider?
- Who else was involved in the decision?

About Results

- How did you measure success?
- What was the long-term impact?
- What would you do differently?

About Relationships

- How did others respond to your approach?
- What feedback did you receive?
- How did this affect working relationships?

About Learning

- What did you learn from this experience?
- How has this influenced your future approach or changed the organization's other teams?
- What would you do differently next time?

STEP 4: THE BIG THREE QUESTIONS

I've said you can't predict the behavioral questions you'll encounter and that's true, but there are three questions that appear in virtually every interview loop:

- Tell me about yourself
- Tell me about your favorite project/most impactful project or otherwise a chance to tell one large core story
- Tell me about a conflict you resolved

Since you know they're coming, you have an opportunity to prepare polished, practiced responses you can deliver confidently even when you're nervous. Not only are you very likely to encounter these questions, but they give you a chance to cover a wide variety of signal areas, establishing yourself as a good fit for the role.

Let's dive into how to craft compelling responses to each of these essential questions.

Tell Me About Yourself

[\[21\]](#)

This question pops up everywhere. It starts in casual networking conversations, shows up with recruiters, hiring managers, and of course behavioral interviewers, and is so ubiquitous it has its own acronym: TMAY (pronounced tee-may).

What to achieve in a TMAY

More than just a nicety, answering TMAY effectively can set the tone for the entire session.

Break the ice and leave a great first impression. Candidates and interviewers alike are nervous at the beginning of an interview. Take this first few minutes to make a lasting impression when the interviewer is most attentive.

Set the context and your scope. Honestly, many interviewers have not reviewed your resume—they might be coming straight from another meeting and not even have looked to see even which open role they're

interviewing to fill. Your TMAY anchors the conversation and helps both of you.

Steer the rest of the conversation. A strong TMAY lets you guide the interviewer toward topics that showcase the best of your career. This ensures the rest of the interview focuses on your repeatable behaviors, which is how the interviewer will judge you.

Express passion for your role and for the company.

Paradoxically, the way to achieve all of this is to keep your TMAY short—about two minutes. Interviewers want enough context to orient themselves but are also eager to dive into their more meaty questions.

A simple structure for TMAY

[\[22\]](#)

To accomplish all this, here's what I recommend as a structure for your TMAY:

1. Personal Summary
2. Accomplishments
3. Forward-Looking Statement

Personal Summary

Start with a couple of sentences introducing yourself. Think of it as a condensed LinkedIn About section. Mention your role (e.g., backend engineer, full-stack developer), years of experience and title, and a unique trait or focus that sets you apart.

I'm a Senior Engineer, focused on backend with five years of experience building scalable systems and solving tough performance challenges. I'm passionate about crafting solutions that are not only efficient but also highly maintainable without documentation.

Tailor this summary to the job. If the role is for a frontend engineer, don't introduce yourself as a backend specialist.

Accomplishments

In the middle, highlight key achievements, similar to how you would on a resume. Choose two to three accomplishments, ideally recent ones, that:

- Showcase core skills and business impact—remember, the manager is hiring you to accomplish something in the real world (revenue, retention, performance, etc.) so prove to them that you care about outcomes.
- Align with the role you’re applying for.
- Can be expanded upon in later questions. Ideally you’re mentioning whatever you’ll cover in your “Favorite Project” response.

Each accomplishment should be concise, typically one sentence, with a dabble of technical detail and business impact.

✓ *At TechNova, I led a project to optimize our Redis-based caching strategy, which reduced page load times by 40% and improved customer retention.*

This isn’t the time to dive into STAR/CARL stories—keep it high level and intriguing. Your goal is to pique interest, not exhaust it.

Forward Looking Statement

End with a sentence or two connecting your past to the role you’re pursuing and what you’re hoping for in the future. This wraps up your TMAY and hands the conversation back to the interviewer and avoids awkwardly stopping.

✓ *I’m excited to bring my experience in distributed systems to a team tackling large-scale challenges and I’m looking to become a TL because I really enjoy the mentoring and technical vision parts of my job.*

Examples and counter examples

Let’s review some complete TMAYs for candidates at different points in their career.

Example: New Grad

As a new grad, prioritize work experience, even if it was short, over personal projects and school projects. Since you’ll likely need to draw from some of those and they may not have real-world impacts, highlight key features as the “business impact” part of the accomplishment.

✓ *I recently graduated from State U with a degree in Computer Science, and I’m passionate about solving challenging problems with code. My focus*

has been on backend development, particularly designing APIs and working with cloud infrastructure.

When I interned at RetailRise, I worked on optimizing database queries, reducing query times. And during my final year, I built a project management tool for remote teams as part of a capstone project, using Python and Flask. This tool included real-time collaboration features and a custom notification system.

I'm excited to bring my learning and enthusiasm to Bloom, given its need for large, scalable systems.

Example: Mid-Level Engineer

✓ *I'm a software engineer with four years of experience, specializing in frontend development and creating intuitive user interfaces. I'm really passionate about making experiences users love.*

At Acme Co, I led the implementation of a design system that reduced UI development time by 30% and ensured consistency across the product. Before that, I worked on a customer feedback platform where I integrated a machine-learning model into the frontend to provide real-time insights.

I'm looking forward to contributing my skills in building user-centric applications to a team solving exciting challenges in the healthcare space.

Example: Senior Engineer

I'm a senior software engineer with over 10 years of experience building distributed systems and scaling complex applications. My expertise lies in designing architectures that work across a large array of teams and usecases.

✓ *At Flux, I spearheaded the migration of our monolithic application to a microservices architecture, reducing downtime by 50% and improving team productivity. Previously, I built a real-time analytics platform handling millions of daily transactions, using a combination of Kafka and Elasticsearch. I also enjoy mentoring junior engineers, helping them grow into confident contributors.*

I'm eager to bring my deep technical experience and collaborative approach to solving the large-scale challenges your team is working on.

Example: People Manager

✓ *I'm an engineering manager with 12 years of experience leading high-performing teams across multiple organizations. I really love building engineering cultures and helping people grow.*

At RideLink, I led a team of 20 engineers to deliver a mission-critical platform that enabled a 40% increase in customer onboarding speed. I also introduced a biannual hackathon that resulted in several ideas being turned into production features. Across my career, I've worked on scaling teams from five to 50 while maintaining a strong engineering culture.

In my conversations so far with folks at Bloom, I've been intrigued by the products and the connections you've built between the teams and your customers. Sounds like an interesting place for me to make an impact.

Example: Proactively Handling Complex Situations

If your career history includes layoffs, gaps, or short stints, address these proactively and concisely in your TMAY. Set the narrative yourself to avoid misinterpretation.

[\[23\]](#)

✓ *Gap: After Sparkly shut down, I took six months to upskill in cloud architecture and completed a certification program. I'm excited to bring this expertise into a new role.*

✓ *Layoff: Due to the changing economics with their real estate customers, HousingTech was under some financial pressure, ultimately deciding to cut the entire Growth team.*

✓ *Short Stints: I've worked across a number of early stage startups, which really accelerated my growth as an engineer but ultimately none of them has panned out from a product-market fit perspective.*

Counterexample: Overly Personal, Personal Summaries

While the interviewer is interested in unique aspects of you, make sure you share aspects relevant to the job. I've heard a number of (especially junior) engineers begin their TMAY with a story like:

✗ *I've been interested in computing since I was 6, when I worked on my first Scratch program in school...*

Counterexample: The History Lesson

Avoid giving a chronological walkthrough of every job you've had. It's tempting to say:

✗ *I started as a frontend engineer at Company X, then moved to Company Y, and now I'm at Company Z...*

This approach is tedious and low-value. Focus instead on the highlights and impact you want the listener to walk away with. For example, if you want the interviewer to hear about how you've worked at large and small companies, say that; if you want them to hear about how you've grown from a junior engineer to a senior engineer quickly, then say that.

Counterexample: Less is Not More

✗ *I'm a senior software engineer at Momentum, with 10 years' experience [full stop]*

Some candidates omit accomplishments of any kind, thinking they shouldn't repeat what's on their resume—they probably got this advice from a well meaning but inexperienced job coach. Bad idea. Many interviewers haven't read your resume or at best they've skimmed it. Use your TMAP to guide them toward the stories that matter, taking care not to produce a History Lesson.

Counterexample: Don't Be Negative About Your Former Employer

When describing the motivation for something new, be positive about what you're headed toward, not negative in describing what you're leaving behind.

✗ *I'm looking for some place I can grow more technically. My current company has really poor engineering, with the codebase kind of a mess. My manager isn't interested in addressing the technical debt at all. The product roadmap changes every week, so we're constantly context-switching and never actually shipping anything meaningful. Plus, there's no real career growth path—I've been stuck at the same level for two years.*

This response makes the listener wonder if you are the problem, and certainly exposes your lack of leadership ability to drive technical and cultural change in your current role.

You can be honest about your motivations without criticizing your current employer:

✓ *As I've grown technically, I've become increasingly interested in working on developer tools and infrastructure that can impact how entire engineering teams work. That's what draws me to this role—the opportunity to build in a culture that knows how to ship and is known for technical excellence.*

Tell Me About Your Favorite Project

Early in the interview, often right after TMAY, you'll be given the opportunity to tell the story of one of your core projects. It might come in the form of an explicit question like, *Tell me about your favorite project* or *Tell me about your most impactful project*. Or it might be a question like, *Tell me about a time when you solved an ambiguous problem*.

Any of these give you the opportunity to choose one of your core stories to tell first, to create a strong impression of your impact, and to set the stage for the rest of the interview. In this section, we'll discuss why it's important to carefully choose this first project to discuss.

Interviewers are seeking to understand the scope you've operated at and how you go about accomplishing something end to end—in other words, a detailed description of your daily work over a long period of time. If the company is like any other company, where they are hiring you to deliver business results using some skillset, then because of the nature of the question, the answer alone is enough to predict the interview's outcome most of the time. The remaining signal to acquire is cultural in nature (conflict resolution, growth, etc.) and not simply “can you deliver on the core parts of the job.”

[\[24\]](#)

It doesn't matter how this question is phrased—“favorite project” or “most impactful project” or whatever—respond with the same story. The interviewer is not actually interested in your affinity for the project, as in “What is your favorite movie,” but rather in hearing what presents the most complete picture of your career.

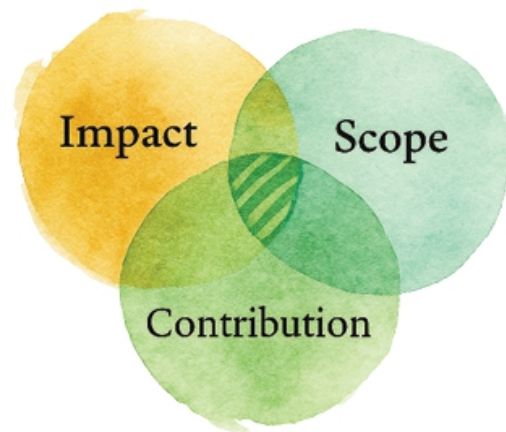
[\[25\]](#)

Choose a project that has high impact, scope, and personal contribution

When selecting your favorite project, optimize for three key dimensions:

Impact: Choose work that moved meaningful business metrics. This could be revenue growth, user engagement improvements, system performance gains, or cost savings—anything that demonstrably made the organization better. Quantifiable results with specific numbers are ideal, but qualitative improvements that clearly benefited users or the business also work.

Scope: Look for projects with substantial breadth or duration that gave you opportunities to demonstrate multiple signal areas. Complex projects spanning weeks or months, involving cross-functional collaboration, or requiring technical and non-technical problem-solving provide richer material for showcasing your full range of capabilities. Also ensure the project relates to your target role in its primary focus.



Personal Contribution: You need to have been a primary driver, not just a participant. The interviewer wants to understand what you specifically did to achieve the results. Being part of a high-impact team isn't enough; you need clear ownership of significant actions and decisions that influenced the outcome.

The sweet spot is finding a project where all three intersect: substantial business results that you personally drove through complex, multi-faceted work. If you can't find a project that scores highly on all three dimensions, prioritize personal contribution over the others—a smaller project where you led everything is better than a massive initiative where your role was peripheral.

Structuring your favorite project story

Since you're talking about a project of meaningful size, you'll find it helpful to structure your response not simply as a single CARL but by

organizing it into themes, or groups of Actions, then signposting those themes throughout your response.

See [Telling Longer Stories](#).

Preparing and responding to follow-ups

As one of your core stories, be sure to prepare common follow-up questions like, *Were there any conflicts you encountered*, *What was the hardest part*, etc. Refer back to [Identifying Follow-Up Questions](#). Also see the discussion on how to respond to various kinds of follow-up questions in [Adjust detail according to type of question](#).

Thoughts on project stories in leadership and specialized interviews

Some interview processes have dedicated sessions for project discussions. See [Deep Dives or Project Retrospectives](#).

Tell Me About a Time You Resolved a Conflict



The authors of *Beyond Cracking the Coding Interview*, citing data from their own *interviewing.io* archive, claim that *Tell me about a time when you had a conflict* is the number one question responsible for no-hire decisions and downlevels. Saying it's number one seems extreme to me—I would have picked “Favorite Project”—but their point is well-taken: understanding the types of conflicts and how you've resolved them is a key indicator of seniority and helps to understand whether you fit within conflict-makes-us-better philosophy of modern tech companies.

[\[26\]](#)

How tech companies view conflicts

I don't have conflicts with coworkers is a frequent response from either junior candidates or those not familiar with how tech companies work. This is a red flag, because it doesn't fit with modern perceptions of workplace dynamics in Silicon Valley-style companies.

[\[27\]](#)

Here are some best practices companies look for related to conflict resolution:

Be assertive: you disregard power structures when the greater good is at stake. If you think you're right, you speak up.

Go directly to the source: you do not triangulate around conflicts or hide behind management (at least not prematurely).

Remain emotionally in control: even in the face of others' anger, you stay calm and professional, venturing to care about the people involved even if there's disagreement.

Stay focused on outcomes: you prioritize the organization/product/user, and not personal gain. Of course, this might only be how they present yourself to others; inside you may have many motives.

Make data-driven, rational decisions: you believe in the meritocracy of ideas and use fact-based persuasion to advocate for your position. Data and logic are sufficient to change your mind.

Sometimes your stories won't fit these best practices. When they don't, check out [When your stories don't fit tech company culture](#).

Let's look at how you can choose the right story, describe it in the right way using CARL, hitting the right notes throughout.

[\[28\]](#)

Choose a story with high stakes, high involvement, and where you were right

When picking a conflict story, pick ones where:

The stakes were high: scope of your conflicts is a reliable indicator of your level. Conflicts over formatting a codebase or which features to include in a release are low stakes versus conflicts over long-lived system design or team charters.

You were deeply involved: being a central player in a conflict is the only way for a listener to evaluate your conflict resolution skills. Make sure you did something to bring about a resolution.

You ended up being right: it is less risky to tell a story where you were right, at least partially. Humility is valuable but not the signal the interviewer seeks and you risk looking like someone who makes poor judgments.

Common conflict stories

Candidates often think they don't have a conflict story, that the interviewer is asking for an emotional drama or when a coworker was being overtly combative with them. But that's not the case. Workplaces are filled with the kinds of conflicts that make great stories:

- You or a coworker are unsatisfied with an implementation or process choice.
 - *My teammate wanted to use a NoSQL database for our user management system, but I pushed for PostgreSQL due to our need for ACID transactions.*
 - *Our team was doing daily standups that ran 45 minutes, so I proposed switching to async updates with only blockers discussed synchronously.*
- Stakeholder wants some work item and you push back for time or cost reasons.
 - *Product wanted three new features in the sprint, but I advocated for spending half the time on technical debt that was causing daily on-call issues.*
 - *Marketing requested a real-time analytics dashboard by month-end, but I knew that a less complex solution would meet their core needs.*
- You want time for some work item, but stakeholders push back for time or cost reasons.
 - *Leadership wanted to ship the feature immediately, but I insisted on adding input validation after showing them potential security vulnerabilities.*
 - *Manager planned for only 3 days of quality work in the next quarter, but I knew the confusing information architecture of*

our site was causing us to lose customers.

- Stakeholder is unhappy with implementation quality and demands changes.
 - *The client kept requesting “small changes” that were doubling our development time, so I created a change request process with clear time estimates.*
- You need something on a certain timeline from a partner team but they are resistant.
 - *The infrastructure team couldn’t provision our new services until Q2, so I worked with them to identify a temporary solution using existing capacity.*
- You or another coworker wants to adopt a new process/technology and there’s some resistance.
 - *I wanted to introduce TypeScript to reduce our production bugs, but senior engineers were concerned about the migration overhead.*

For these, stakeholders could be managers, product managers, designers, senior engineers, customers, etc.

Choosing a non-work story

If possible, choose a story from your work experience, preferring those over stories from school or from personal interactions. If you really don’t have a disagreement story from your work experience, choose one from school first, and then personal if you must. Any of these are more acceptable if you are a student yourself or have recently graduated, of course.

An Example from School

✓ *During my senior capstone project, our team of five was building a mobile app for campus dining. Three weeks before the deadline, my teammates wanted to completely redesign the user interface because they thought it looked “too basic,” but I pushed back because we hadn’t finished core functionality like payment processing and order tracking. The stakes were high since this project determined our final grade and served as our portfolio piece for job interviews.*

An Example from Personal Life

✓ *When my roommate and I were planning to move apartments together, she wanted to sign a lease for a place that was \$400 more per month than our budget because it had granite countertops and a gym. We had both agreed on a spending limit to save money for post-graduation expenses, and I knew the extra cost would put financial strain on both of us for the next year.*

Avoid misconduct stories

I've been asked a number of times—always by women—whether a story about a coworker's inappropriate advances or other harassing behavior is a productive story to tell. While the candidate may have demonstrated real courage and even strong conflict navigation, I recommend choosing a different story. First, this is better categorized as addressing misconduct or even criminal behavior, not interpersonal conflict. Second, these stories tend toward legal escalation or eventual separation, certainly not collaboration or compromise. Third, and perhaps most frustratingly, interviewers may distance themselves from the situation, thinking, “That wouldn't happen here,” rather than focusing on your skills.

You shouldn't have to carry these experiences silently, but in this narrow context with limited time, it's wiser to select a story that's more easily mapped to experiences the listener thinks you'll have in their organization.

Common elements of successful conflict resolution stories

As a way to help you remember Actions you took during the conflict resolution process and to give you a sense of what conflict resolution best practices look like in a tech company, here are elements of successful conflict resolution stories I commonly hear:

Be proactive in raising concerns: if you are headed toward a conflict, be the one who directly confronts it with your coworkers. If you don't bring it up, then at least be receptive when it's brought to your attention.

Only start constructive conflicts: no one likes a cantankerous coworker. What you care about reflects on your maturity and fitness for a role.

[\[29\]](#)

Seek to understand before being understood: often more information is needed before jumping into a conflict. Perhaps you seek it from the main party or collect corroborating evidence from others.

Choose the right communication channels: you move from one communication channel to another based on the state of the conflict. Comments on a PR or shared document are rarely effective conflict resolution mediums. Sometimes casual conversations are best.

[\[30\]](#)

Demonstrate empathy: expressing an understanding and an affinity for the other party's position greases next steps and compromises.

Demonstrate maturity: even in the face of emotional circumstances, you maintain your composure, and act in a professional and laudable way.

Use data to make decisions: where data or proof-of-concepts are possible, you collect or build them.

Involve the right coworkers at the right time: if other people are needed, you pull them in when needed, not before to create drama and not after when the project has suffered a stall.

Don't take too long: invest time and energy relative to the business value of the conflict, perhaps timeboxing any data collection or deliberation steps.

Come to a clear resolution: even if the result is not what you desired, there is clarity and a clear path forward for all participants.

Preserve relationships: coworkers still want to work with you after the conflict is resolved, even if they didn't get their preferred outcome.

If your story involves a number of these actions, it's probably a good story to tell.

How CARL naturally turns into a three-act story

Using the CARL format is still a good way to respond to a question about a conflict. When you tell the story this way, it naturally forms a narrative in three acts, familiar to any ancient Greek playwright or modern movie-goer:

- Act I: Setup (Context): setting the stage for the narrative
- Act II: Confrontation (Action): the conflict itself, followed by key turning points
- Act III: Resolution (Result & Learnings): the conclusion and end of the narrative.

This build-up of tension and subsequent release, ending with narrative closure, presents a familiar emotional arc, similar to how musical cadences progress and resolve. Moviemakers would dedicate 50% of their time to Act II and an equal split of time for Act I and III, but I would suggest more of a 10% allotment for Context, 60% to Action, and 30% to Result and Learnings.

Here are some thoughts on how best to structure your CARL story for conflict resolution:

- Move quickly through the Context section; get to describing the conflict quickly. Don't go so quick that you fail to land the high-stakes nature of the conflict though. The listener needs to understand what happens if the conflict is not resolved.
- Focus on Actions you took that moved the conflict forward. Things like bringing in others, proposing compromises, collecting data, writing proposals and messages.
- Consider adding more detail to human interactions than you normally would in a CARL story. Waiting until Monday to have that difficult meeting rather than right before everyone leaves on Friday is a sign that you're aware of how to manage human relationships.

Full examples of conflict stories

Data Quality vs. Timeline as an ML Engineer

CONTEXT

✓ *Our fintech startup was under intense pressure to launch a fraud detection model before our Series B funding round. The CEO had promised investors we'd have ML-powered fraud prevention live by month-end, and the product team had already committed this timeline to our biggest enterprise prospect. We'd been working with a dataset of transaction labels that were created by our customer service team over the past year, but I'd noticed significant inconsistencies when I started model development. About 30% of the labels seemed questionable - transactions marked as fraudulent that looked legitimate to me, and vice versa.*

When I raised this concern, the product manager was visibly frustrated. She told me we "couldn't afford to be perfectionists" and that we needed to ship with what we had. The sales team was breathing down her neck

because this prospect represented 40% of our projected revenue for the quarter.

ACTION

I understood the business pressure, but I also knew that shipping a model with poor precision would be catastrophic - false positives would block legitimate customer transactions, potentially losing us existing users and damaging our reputation with the enterprise prospect we were trying to win.

First, I ran a quick analysis to quantify the impact. I trained a baseline model on the existing data and found it had a 23% false positive rate on our validation set. I calculated that this would block roughly \$2M in legitimate transactions monthly for our current customer base.

I scheduled a meeting with the product manager and presented this data, but she remained firm on the timeline. She said the business risk of delaying was greater than the technical risk of shipping an imperfect model. At this point, I could see she was stressed and feeling caught between competing pressures from leadership.

I then proposed a compromise: give me two weeks to clean the most critical mislabeled examples while the engineering team built the model serving infrastructure. I offered to work with our customer service team to review a sample of transactions during business hours and create a more reliable training set. I also suggested we could launch with more conservative thresholds initially to reduce false positives, even if it meant catching fewer fraud cases at first.

When she still seemed hesitant, I brought in our head of customer success, who confirmed that transaction blocking issues were already our second-highest support ticket category. This helped illustrate that launching with high false positives would create immediate operational problems beyond just user experience issues.

After some back-and-forth, we agreed on a modified timeline: I'd spend 10 days on data cleaning while working closely with the customer service team to establish a labeling process, and we'd launch with conservative settings that we could tune more aggressively once we had cleaner training data.

RESULT

The extra time investment paid off significantly. After cleaning the data and retraining, our false positive rate dropped to 8%, and we actually caught

15% more fraudulent transactions than the original model. We launched one week later than originally planned, but the product manager was able to present much stronger metrics to both our investors and the enterprise prospect.

The enterprise deal closed two weeks after launch, partly because our fraud detection numbers were substantially better than what they were seeing from competitors. More importantly, we avoided what could have been a customer service nightmare and potential churn from blocking legitimate transactions.

LEARNINGS

This experience taught me the importance of quantifying technical risks in business terms that stakeholders can understand. Rather than just saying “the data quality is poor,” showing the direct impact on revenue and customer experience made the case much stronger. I also learned that proposing solutions alongside problems is crucial. If I had just pushed back without offering the compromise timeline and conservative threshold approach, we might have reached an impasse.

The product manager later told me she appreciated that I understood the business pressure she was under while still advocating for technical quality. We’ve since established a regular data quality review process for all ML projects, which has prevented similar conflicts and actually sped up our overall development cycle.

Platform vs. Feature Investment as a Product Manager

CONTEXT

✓ *Our B2B SaaS company was in a heated competitive battle with two well-funded startups that had launched similar workflow automation tools. Our engineering manager, David, came to me in Q3 planning with a proposal to dedicate the entire quarter to refactoring our microservices architecture and paying down technical debt. He argued that our deployment times had grown to 45 minutes, we were having weekly production incidents, and developer velocity was slowing dramatically.*

I understood his concerns, but we were hemorrhaging potential customers to competitors who were shipping flashy AI-powered features every month. Our latest user research showed that 60% of prospects were choosing competitors specifically because of feature gaps around smart notifications and workflow templates. David became visibly agitated when I suggested

we needed to prioritize these competitive features. He said I was being “short-sighted” and that we’d “regret ignoring technical fundamentals.”

ACTION

I realized this conflict was bigger than just Q3 planning---it was about our entire product strategy and how we balanced short-term competitive needs with long-term sustainability. I needed to find a way forward that addressed both concerns without completely sacrificing either.

First, I did some back of the envelope calculation on lost revenue. Ideally I would have worked with the marketing and customer success teams on this but I didn’t have time. I arrived at losing roughly \$400K in ARR due to feature gaps. I also asked David to detail the operational costs of our technical debt---he showed me that engineering productivity had dropped 35% over six months, and our incident response was costing us about 20 hours of engineering time weekly.

I proposed a hybrid approach: dedicate 60% of engineering capacity to the most critical competitive features (smart notifications and basic workflow templates), while using 40% for the highest-impact infrastructure improvements. David pushed back hard, saying that “half-measures” wouldn’t solve the underlying architecture problems and would just create more technical debt.

This led to a heated discussion where David accused me of not understanding the long-term consequences, and I’ll admit I got defensive and said he wasn’t considering the business reality. The conversation ended with both of us frustrated, and I could see it was affecting the broader team’s confidence in our leadership.

I decided to bring in our CTO for a three-way discussion. I presented both perspectives with data, and we worked through various scenarios. After significant back-and-forth, we agreed to a modified version of my proposal: ship a minimal version of smart notifications in month one, dedicate month two entirely to infrastructure work, and use month three for workflow templates with any remaining infrastructure improvements.

RESULTS

The compromise had mixed results, but ultimately taught me valuable lessons about balancing competing priorities. We did ship smart notifications and won back two enterprise deals worth \$180K ARR. The

infrastructure work in month two addressed some critical issues, though not the deeper architectural problems David had identified.

By Q4, we did need to invest more heavily in infrastructure than I'd originally anticipated, which confirmed that David's concerns were well-founded. However, the features we'd shipped in Q3 kept us competitive during that infrastructure work, and we retained several key customers who might have otherwise churned to competitors.

David and I had some tense moments during the quarter, but we gradually rebuilt our working relationship. He appreciated that I acknowledged his technical expertise had been correct, and I valued that he remained committed to finding solutions even when he disagreed with the approach.

LEARNINGS

This experience taught me to better distinguish between technical debt that can be managed incrementally versus fundamental architectural issues that require dedicated investment. I learned that experienced engineering leaders often have insights into technical risks that aren't immediately visible from a product perspective, and I need to create space for those concerns even when they conflict with business timeline pressures.

Moving forward, I've established quarterly "technical health" reviews where engineering leadership can present data-driven cases for infrastructure investment, and I've become better at advocating upward for technical needs. I also learned that conflict resolution sometimes means accepting that you can't make everyone completely happy, but you can preserve relationships by acknowledging different perspectives and being transparent about tradeoffs. The key is following through on commitments and adjusting course when new information emerges.

People and Time Allocation as an Engineering Manager

CONTEXT

✓ *Our engineering organization was under significant pressure during Q2. My team was three weeks into a critical security compliance project that needed to be completed by end of quarter to meet SOC 2 audit requirements - a prerequisite for several enterprise deals worth \$2M in pipeline. At the same time, our payments team was struggling with a complex migration to a new payment processor that had hit multiple technical roadblocks.*

During our weekly org leaders meeting, my director announced that she needed to "borrow" my senior engineer for six weeks to help rescue the

payments project. He was not only our most experienced engineer, but he was also the technical lead on our security implementation and the only person on my team who fully understood our authentication architecture.

I immediately expressed concern about this reassignment in the meeting. Sarah explained how the payments migration was “mission-critical” during this crunch period. She emphasized that the entire company’s payment processing would be at risk if the migration failed, affecting all revenue streams. Other engineering managers in the room stayed silent, and I could feel the pressure to just agree and move on.

ACTION

I understood the gravity of the payments situation, but I also knew that losing this engineer would put our SOC 2 compliance in serious jeopardy. I asked for time to assess the impact and propose alternatives and the director agreed.

After the meeting, I immediately ran through our project timeline with the engineer. Without him, we’d miss the compliance deadline by at least 3-4 weeks, which would push our enterprise deals into Q3 and potentially Q4. I documented this impact and also quickly messaged our Head of Sales to understand the revenue implications of delayed enterprise deals versus potential payment processing issues.

I then spent time understanding the payments team’s specific technical challenges by grabbing a quick meeting with their EM. I discovered they were struggling with API integration complexity and database migration scripts---areas where the engineer’s expertise would help, but not uniquely so. I think the director was just trying to make sure the work got done to a high quality bar.

I messaged the director outlining the data and I proposed three alternatives: temporarily reassigning our mid-level engineer (who had payments experience from her previous company), bringing in a contractor with payment processing expertise, or having the senior engineer provide consulting support to the payments team for 4-6 hours per week while remaining on our project.

She initially pushed back, saying this other engineer wasn’t senior enough and contractors would take too long to onboard. We ended up jumping on a quick call between meetings later that afternoon.

I could tell she was worried about the seniority of this other engineer so I sought some middle ground and to timebox: assign the mid-level engineer,

have the senior engineer do twice daily check-ins for two weeks and see how it's going. Because this would take time away from the SOC2 compliance, I asked if we could inform the leadership team---this was risky for my team so I wanted to make sure the leadership team heard the risks. I drafted an email that the director sent to leadership and me (after some edits of course).

RESULT

So the mid-level engineer was reassigned to the payments team with the senior eng providing daily consultation calls.

The approach worked better than expected. Maria's, the mid-level engineer's, previous payments experience proved valuable. Alex, the senior engineer, was able to step back after a short oversight period and maintain momentum on our security project, hitting our SOC 2 compliance deadline with two days to spare.

The payments migration completed one week later than originally planned, but within acceptable business parameters. More importantly, the director appreciated that I had come with solutions rather than just resistance, and she later acknowledged that my pushback had led to a better outcome for the organization.

RESULT

This experience taught me the importance of understanding stakeholder motivations and framing resource conflicts in terms of organizational outcomes rather than team ownership, and to always come prepared with data and alternatives when pushing back on leadership decisions.

I also realized that effective pushback requires making the tradeoffs visible to decision-makers rather than trying to resolve them at my level.

Moving forward, I've established better communication channels with peer managers about resource needs early in planning cycles, and I've become more proactive about cross-training team members so we're less vulnerable to key-person dependencies.

Be empathetic and positive

It's common for a conflict story to have unpleasant parts, or more pointedly, unpleasant people. In those cases, there is an uncharitable way to describe it, a framing that might be accurate but ultimately negative.

Resist the temptation to tell the story this way, with a villainous foil to your heroic righteousness. Instead, be diplomatic, positive and generous in your framing and express empathy toward those involved, even if at the time you felt anything but empathetic.

✗ *Our product manager was completely unrealistic about timelines and kept demanding we add more features without understanding the technical complexity. She would interrupt our planning meetings, dismiss our concerns about code quality, and basically treated the engineering team like code monkeys who should just implement whatever she dreamed up. When I tried to explain why her timeline was impossible, she got defensive and started micromanaging our daily standups.*

Let's reframe this so that it acknowledges the same underlying tension but:

- Recognizes the PM's external pressures and positive intentions
- Frames her behavior as coming from passion rather than ignorance
- Takes partial ownership of the communication breakdown
- Focuses on systemic issues rather than personal character flaws
- Sets you up as someone who seeks to understand and solve problems collaboratively

✓ *Our product manager was under significant pressure from leadership to deliver key features for an important client demo. She was passionate about the user experience and had great instincts for what would delight customers, but we weren't aligned on how much engineering effort certain features would require. I could see she was frustrated when technical constraints seemed to block her vision, and I realized we needed better communication about the tradeoffs between speed, quality, and scope.*

But don't hide the emotional content

In your quest to be empathetic and positive, you'll be tempted to purge your story of any negative emotional content. Don't do this either. If someone yelled at you, say that. If they were angry and vindictive, say that. If the executive was threatening the department with dissolution, say how that was affecting you and your coworkers. Do all this in an even and professional, journalistic tone, focusing on facts, and show that your conflict resolution skills extend to handling emotionally charged situations.

A common sign that you're hiding emotional content is a leaden story that sounds like no conflict at all:

✗ *Our lead designer had some thoughts about the user onboarding flow that differed from my product roadmap. She's incredibly talented and really cares about user experience, so I understood her perspective. We had a productive conversation about it and worked together to find a solution that balanced user needs with business requirements. We ended up iterating on the design and launched something that made everyone happy.*

The listener is simply confused by this. Where is the conflict they asked about? Put the emotional content back in to make the response relevant to the question:

✓ *Our lead designer was upset when she learned I had committed to shipping user onboarding in two weeks without consulting her. She interrupted our standup to say that my timeline was "completely unrealistic" and that I was "setting the team up for failure." The engineering team went silent, and I could see they were uncomfortable being caught between us. I was frustrated too because the CEO had been asking daily about onboarding metrics and I felt pressure to deliver quickly. After the meeting, the designer sent me a blunt Slack message saying she wouldn't sign off on shipping "half-baked UX" that would hurt our conversion rates. I realized this conflict was affecting team morale and our ability to work effectively, so I scheduled a face-to-face meeting where we could both express our concerns openly and work out a realistic timeline that prioritized the most critical user experience elements.*

The Result includes the relationship

Achieving forward motion on a project after a conflict is progress, but if your relationships are damaged, endangering future progress, then your story casts doubt on your conflict resolution skills.

When presenting the Result part of your CARL response for a conflict, include the state of the relationships after the conflict was resolved. Consider stating it explicitly—*Jenny was satisfied with this compromise and expressed it in the follow-up meeting*—and citing evidence of future successful collaborations.

If relationships were damaged in some way, tread carefully and provide some reason why the interviewer should trust you to resolve conflicts. I see

this most often when the conflict is zero-sum or the candidate's response was constrained by outside forces, leaving no room for win-win outcomes, and the candidate did what they could to bridge the personal trust gap but was ultimately unsuccessful.

When your stories don't fit tech company culture

If you were in a hierarchical organization, perhaps a traditional business, or even a tech company in parts of the world where conflict is avoided and obedience to management is expected, telling your story without additional framing can reflect negatively on your ability to resolve conflicts.

If this applies to you then consider these approaches:

Acknowledge the constraints of culture: describe what was expected of you in that organization and why it differed from a more conflict-embracing culture.

Describe what you would have liked to do: if you were in a more permissive culture, what would you have done. It's even better if you tried to do these things but were hampered.

Consider using the Halo Effect: if you have a string of smaller stories to mention where you did demonstrate assertiveness and willingness to rock the boat to make things around you better.

[\[31\]](#)

✓ *At my previous company in Tokyo, we had a very hierarchical structure where directly challenging senior management decisions was considered disrespectful. When our director announced we would launch our mobile app without proper testing because a competitor had just released theirs, I knew this could damage our reputation with enterprise clients.*

In that culture, the expectation was to simply execute the directive without question. However, I felt strongly that we needed more time for quality assurance, so I worked within the cultural constraints by approaching my immediate manager privately and presenting data on potential user impact. I also prepared a detailed risk assessment document that outlined the technical concerns without directly contradicting the director's timeline.

Ideally, I would have liked to bring this up directly in the planning meeting with all stakeholders present, but I knew that would have been seen as insubordination and could have damaged working relationships across the team.

Even with my indirect approach, it took three separate conversations and bringing in our most senior engineer as an ally before we secured an additional week for testing.

Telling a story where you were wrong

[\[32\]](#)

I will reiterate that telling a story where you were wrong is risky. Yes it shows humility and a growth mindset, but you risk looking like someone who has poor judgment. A company like Amazon even encodes being someone who is “Right, A Lot” as part of their formal values.

I would only voluntarily choose a story when you were wrong if you’ve already established yourself as competent and are seeking to demonstrate humility, flexibility, and a growth mindset. This is more likely if you are interviewing for a leadership role and you are an experienced behavioral candidate; in other words, you know what you’re doing.

Sometimes you’ll be asked explicitly about when you were wrong. Preferably you have a story where you were not obviously wrong in advance—or at least the interviewer cannot find immediate fault with your position. Note this may be difficult for you to assess since the interviewer is likely more experienced than you.

✗ *At our early-stage startup with 4 engineers, I pushed hard for a microservices architecture when we were rebuilding our core platform. I argued that we needed to think long-term about scalability and team ownership, and that starting with microservices would prevent painful migrations later... We ended up going back and forth but ultimately the team voted for the monolithic architecture.*

After six months, we ended up pivoting pretty hard on product direction and rebuilt most of the platform from scratch. Having it all in one place meant that process was easier.

I learned that architectural decisions need to match current organizational reality, not future aspirations. While my technical reasoning about scalability was sound, I ignored the practical constraints of team size and product maturity. The “right” technical solution isn’t always the right business solution. Now I advocate for evolutionary architecture---starting simple and splitting services only when team size and complexity genuinely demand it. This taught me to weight immediate team productivity more heavily when making technology choices at early-stage companies.

This seemed pretty clear in the beginning and even though they call it out as a learning, this story comes across as a net-negative on signal.

As a contrast, here's an example of where the outcome was not clear in advance. Notice how the candidate justifies their initial position, making it sound defensible. Then they lean into how they came around to the corrected view and provide evidence that they've internalized their learnings:

✓ *We were planning the next quarter's roadmap and ended up at odds on which feature to prioritize...Our user research consistently showed that customers wanted more customization options in our dashboard. Every survey and interview reinforced this, so I was able to win support for building 15+ customization features including widget arrangement, color themes, and data filtering options.*

Post-launch analytics revealed that less than 10% of users engaged with any customization features, and our overall dashboard engagement actually decreased. Users were overwhelmed by the options and many complained that the interface felt "cluttered" compared to our simpler previous version. We quickly spent some time removing most customization options and redesigning around a cleaner, more opinionated experience.

This taught me the critical difference between what users say they want and what they actually use. People often express preferences for flexibility and control in research settings, but their actual behavior shows they prefer curated, streamlined experiences. I learned to validate stated preferences with behavioral data and prototype testing before committing to major feature investments. Now I'm much more skeptical of feature requests that add complexity, even when they come directly from user feedback, and I always look for ways to test actual usage patterns rather than just stated preferences.

STEP 5: PREPARE QUESTIONS FOR THE INTERVIEWER

The five minutes at the end of your interview isn't just courtesy time. It's your opportunity to demonstrate your interest in the role, assess the fit for you, and communicate what's important to you as a candidate.

Why Your Questions Matter

It may be a surprise to you, but interviewers evaluate you based on what you choose to ask. No questions can indicate disengagement or that you're treating this as transactional. Thoughtful questions demonstrate:

- **Preparation:** You've researched the company and role
- **Strategic thinking:** You understand what matters for success
- **Self-awareness:** You know what you need to thrive
- **Genuine interest:** You care about this specific opportunity
- **Maturity:** You're thinking beyond just getting the job

Your questions also serve a critical practical purpose: gathering information to determine if this role, team, and company align with your career goals and working style.

Avoid Poor Questions

This is somewhat controversial, but I recommend you avoid typically bland questions:

- *Tell me about a day in the life on the team.* Builders everywhere attend meetings, do planning, coders code, etc. There's probably something more specific you want to know.
- *What's the tech stack?* This should be in the job description or easily researched.
- *What are the growth opportunities here?* Too vague and focused only on what you get.
- *Do you like working here?* Invites a predictably positive response with little substance.

Also avoid red flags:

- *How much is the pay?* This is not the time for negotiation.
- *How many vacation days/free food/...?* Asking about perks seems petty.
- *Are you happy with the pay? Do you like the boss? What are you doing later tonight?* Don't ask personal questions.

Choose Thoughtful Questions

Instead, ask high-quality questions tailored specifically to the audience and what you care about.

Tailor to the audience. Different interviewers can answer different questions.

- Hiring Manager
 - Team dynamics and priorities
 - How success is measured
 - Management philosophy
 - Team challenges and opportunities
 - How they support growth
- Team Members
 - Day-to-day realities
 - Technical challenges
 - Team collaboration patterns
 - Honest assessment of culture
 - What they wish they'd known
- Cross-Functional Partners
 - How teams collaborate
 - Decision-making processes
 - Communication patterns
 - Vision and strategy for their org
- Recruiters
 - Timeline and next steps
 - Structural/logistical questions
 - General company information
- Senior Leadership / Executives

- Company vision and strategy
- Organization priorities
- How they think about culture
- Long-term technical/product direction

Tailor to you. Before the interview, identify what's most important to you. This helps you ask questions that actually inform your decision:

- Skill growth
- Impact
- Work-life balance (but be specific)
- Culture and collaboration
- Career development

Be specific rather than generic. Instead of *What is your culture like?* ask, *Give me an example of how you've seen [company value] play out on the team?*

Experience-based rather than hypothetical. Instead of *How does the team handle conflicts?* ask, *Can you tell me about a recent technical disagreement and how it was resolved?*

Include forward-looking statements. Not just asking about past projects but also about what they expect to happen in the future.

You only need one or two. You won't get a lot of time for questions so choose judiciously.

Example Questions

Here's a set of topics you could ask about. I've collected a lot more questions for you in [Appendix: Questions to Ask the Interviewer](#).

- **Team:** *What is the focus of the team in the near term and why? How does planning work for project roadmaps?*
- **Role:** *What led to this role being open? What does success look like in this role?*
- **Company:** *How does this team's work connect to the company's broader mission and strategy? What major bets are the company making?*

- **Manager:** *How would your team describe your management style?
Can you give me an example of how you've helped someone grow?*
- **Partners:** *How does this team typically collaborate with product management/design/other engineering teams?*

STEP 6: PRACTICE

Your Story Catalog is just raw material until practice transforms it into fluent interview responses. Some candidates resist this kind of practice, fearing it will make them sound robotic or rehearsed. But the cognitive load of real-time storytelling—selecting details, organizing coherently, adapting to your listener—is large in my opinion and practice closes the gap between what you mean to communicate and what actually comes out under pressure.

Progressive Practice to Optimize Results, Time, and Expense

The key to efficient preparation is progressive practice: start with controlled, low-pressure environments where you can focus purely on content and structure, then gradually introduce the complexity and pressure of real interview conditions.

1. **Practice by yourself:** Master your core stories and basic delivery in private
2. **Practice with AI:** Test yourself with unpredictable questions and get immediate feedback
3. **Practice with peer mock interviewers:** Experience real conversation dynamics and human feedback
4. **Polish with professional mock interviewers:** Calibrate against expert standards and company expectations[\[33\]](#)

Begin Practicing by Yourself

Solo practice might feel awkward, but it's where you build the muscle memory that makes live interviews feel natural. You're creating neural pathways that will activate automatically when you're under pressure, freeing your cognitive resources to focus on reading the interviewer and adapting your responses.

Record yourself delivering your core stories

Start with your Story Catalog and select 3–5 stories that provide the best coverage across signal areas and company values. These are the stories

you'll tell most frequently in interviews.

Set up your phone or laptop to record video of yourself, then deliver each story in CARL format as if speaking to an interviewer. Don't worry about perfection on your first take. The goal is capturing a baseline. Here's what to watch for:

[\[34\]](#)

- **Pacing.** Are you rushing through actions or lingering too long on context? Aim for spending about 10% of your time on Context, 60% on Actions, and 30% on Results and Learnings.
- **Clarity of ownership.** Count how many times you say "I" versus "we." Are your specific actions clear, or do they disappear into team accomplishments?
- **Verbal fillers.** Notice patterns in your filler words—"um," "like," "you know," "basically." Everyone has them, but excessive fillers signal nervousness and reduce credibility.
- **Energy and engagement.** Do you sound genuinely interested in your own story? If you're boring yourself, you'll definitely bore an interviewer.
- **Organization.** Can you follow your own narrative? If you're jumping around chronologically or between different aspects of the project, your listener can't follow.

Record yourself 2–3 times for each core story, applying lessons from each review. You'll notice dramatic improvement by the third take as you build fluency.

Master the Big Three

Your responses to "Tell Me About Yourself," "Tell me about your favorite project," and "Tell me about a conflict" deserve extra attention since they appear in virtually every interview loop. See [Step 4: The Big Three Questions](#)

Practice these until you can deliver them conversationally without notes. Record yourself, watch the playback, refine, and repeat. Aim for delivering each one 3–5 times until it feels natural rather than memorized. Pay special attention to:

- **Your TMAY opening.** The first 20 seconds establish your presence. Do you sound confident and engaging?
- **Transitions in your project story.** Are you clearly delivering the most important contributions to the project? Do you need to consider organizational techniques like [Table of Contents](#)?
- **Emotional tone in your conflict story.** Are you balancing professionalism with authentic emotional content?

Practice Decode, Select, Deliver with real questions

Once your core stories and Big Three feel solid, it's time to practice the pattern-matching you'll do in real interviews: hearing a question and quickly selecting the best story to answer it. You're not trying to cover all the possible questions, but test your ability to apply the Decode, Select, Deliver framework quickly leveraging your Story Catalog.

Use the question scripts in the appendix [\[→\]](#) for this exercise. For each question, understand what it's asking, make your selection, then deliver the full CARL response.

Look for signal areas where you don't have ready-made stories. You may need to return to story identification [\[→\]](#).

[\[35\]](#)

Leverage AI to Extend your Practice

Large language models are of course available all the time and never get tired of practicing with you.

Use AI to test yourself with unfamiliar questions. After you've practiced with the scripts from this book, prompt an AI to identify additional questions based around your target company's values.

Generate common follow-up questions for your stories. Share your stories with the AI and ask about common follow-up questions then practice responding.

Recognize AI's limitations. AI can't replicate things like nonverbal feedback (facial expressions and body language), conversational flow (interruptions and tangents), pressure, nor can it evaluate you on how you're coming across emotionally. That's why you also need mock interviews with a human.

[\[36\]](#)

Bringing in People: Peer Mock Interviews

Good against remotes is one thing. Good against the living: that's something else.

Han Solo

Unlike coding and perhaps even system design, it is much harder for you to know if your behavioral responses are sufficient for a hire decision—it's fundamentally subjective and depends heavily on the company to which you're applying.

Leveraging a peer mock interviewer can help you wade through that subjectivity. A mock interviewer provides:

Pressure: telling stories on the spot, to a real human, is better than writing them down, going over them in your head, or even responding to an AI.

Follow-ups: interviewers will push you to answer follow-ups relevant to your stories, which will help you anticipate common reactions from interviewers. This is especially important if your stories involve you making mistakes.

Coaching: a good mock interviewer will give you actionable feedback to improve your stories.

Accountability: if you have more than one mock scheduled, then you'll be held accountable to make the necessary changes to your stories between mocks. Preparing for behavioral interviews is not easy, so accountability is always welcome.

Encouragement: preparing for interviews can be a slog and having a human who cares about you express interest in your process and encouragement along the way can be truly beneficial for your mental health.

Confidence: likewise, having a skilled interviewer tell you that you're ready will give you confidence, which is part of the battle in the interview itself.

Finding peer interviewers

There are a few ways to find someone to help you conduct a peer mock interview:

- **Leverage your network:** Reach out to friends or former colleagues who work at your target companies. They often have insight into the

company culture and interview process and might be willing to do a mock interview with you.

- **Mock interview companies:** Many online communities have peer networks designed to pair you with other preppers for free.
- **Phone a friend:** Even a non-technical friend could ask you questions using the scripts in the appendix [\[→\]](#). They might not do great at follow-ups or provide amazing feedback, but the pressure of answering on the spot will help.

Prepare your peer interviewer

Give your peer some guidance on what you're looking for before you begin.

Cover the signal areas: A comprehensive mock interview should touch on all the key signal areas from [What Interviewers Assess: Signal Areas](#). The scripts provided in [Appendix: Question Repository and Scripts for Mock Interviews](#) cover these areas.

Create a feedback framework: Give your peer a simple rubric to evaluate each response. If you have the evaluation framework for the company you're interviewing with, use that. Otherwise, something like a 1–5 scale across dimensions such as:

- Story selection relevant to role
- Demonstration of relevant skills
- Communication clarity and structure
- Evidence of growth/learning

Have them ask follow-ups: Train your peer interviewer to ask follow-up questions like those in [Identifying Follow-Up Questions](#).

Collect and incorporate feedback

Be sure to leverage your peer for honest feedback about your performance after your mock:

- **Record and review your sessions:** One advantage of peer mocks is that you can easily record them (with permission). Watching yourself answer questions can reveal nervous habits, verbal pauses, or areas where you tend to ramble or lose focus.

- **Feedback on structure:** Were the stories easy to follow? Did you use CARL effectively?
- **Assess your story choice:** Did your examples demonstrate the right level of complexity for your target role?
- **Assess your story depth:** Did you provide enough specific, repeatable actions and quantifiable results?

Calibrating and Polishing with Professional Mock Interviews

[\[37\]](#)

While peers are valuable, I maintain that a professional mock interviewer—at least someone who works as a hiring manager in tech—is your best resource if you can find and afford one for these reasons:

- **Calibration:** a professional mock interviewer will help ensure you've picked the best stories that represent your strengths relative to the company's values.
- **Follow-ups:** an experienced manager knows what other managers think when they hear stories and will probe in the right places.
- **Coaching:** an experienced mock interviewer has probably conducted 10s or even 100s of sessions with candidates and will coach you in ways a peer just can't.
- **Confidence:** as hard as you prepare with peers, you might have some doubts as to your readiness unless you conduct a mock with a professional.

While they represent an investment of time and money, the returns are substantial. You should come out with stronger stories, better delivery, and increased confidence going into the real thing.

Here's how you can sequence your preparation and mock interviews to maximize the time you have with them:

Set clear goals: Discuss your objectives with the interviewer before you begin. You might want to work on connection to company values or get feedback on whether the scope is right for the target level.

Plan for two sessions: Everyone's situation is different but for the average candidate, I recommend you have two mocks:

- In the first mock, cover the Big Three and get feedback on overall storytelling structure and technique.
- Apply that feedback to your other stories and ensure you have your stories aligned with signal areas and company values. Schedule your mocks with enough time between them, at least a couple of days, so you can focus on implementing the feedback.
- In the second mock, practice questions related to the company values or the signal areas. The goal is not to prepare for specific questions but rather to ensure you can pair the question with the best project on the fly.

Record and review: Ask your interviewer if you can record the session and get a written summary of their feedback from them, if that's not something they're already planning to do. Apply the same feedback reflection you did during the peer mock interviews.

STEP 7: WHAT TO DO AFTER AN INTERVIEW

If you made it through the interview, congratulations. There are a few things to consider to maximize the value you can gain from all your effort.

Capture details while they're fresh

Within 30 minutes of your interview ending, open a document and brain-dump everything you remember:

- Every question you were asked
- Which stories you told and why you chose them
- Follow-up questions the interviewer asked
- Moments where you felt confident vs. uncertain
- Topics you wish you'd covered differently
- The interviewer's reactions and engagement level
- Any specific feedback they provided

Your memory of the interview degrades rapidly. Details that feel crystal clear immediately after will become fuzzy within hours. This raw capture is your foundation for productive reflection.

Assess your performance

This is challenging, since most interviewers won't provide you any feedback or even reactions, but you can still learn from what signal is available. Spend some time reflecting on these questions and jot down notes.

Did they ask follow-up questions?

Follow-ups or interruptions are to be expected and might be positive or negative signs:

- **Clarifying follow-ups (neutral to positive):** *How big was your team?* or *What was the timeline?* suggest the interviewer is engaged and building a mental model of your situation. They want more details because they're interested.
- **Probing follow-ups (positive):** *How did you make that decision?* or *What alternatives did you consider?* indicate the interviewer is assessing your judgment and decision-making framework. They're going deeper because they see potential.

- **Skeptical follow-ups (concerning):** *Wait, you did all of that yourself?* or *How much of this was actually your idea?* suggests the interviewer doubts your claims or suspects you're overstating your role.
- **Redirecting follow-ups (negative):** *Let's move on to another example?* or frequent interruptions indicate your response isn't providing the signal they need.

Were they engaged?

Interview demeanor varies widely by individual and culture, but you might be able to glean some insights from assessing how engaged they were. Recall their nonverbal behavior:

- **Positive signals:** Taking notes, nodding, leaning forward, making eye contact, smiling at appropriate moments
- **Neutral signals:** Professional but reserved demeanor, consistent note-taking without much reaction
- **Negative signals:** Checking the time, looking distracted, minimal note-taking, cutting you off frequently

Some interviewers maintain poker faces throughout, but dramatic shifts in engagement—starting strong then dropping off, or vice versa—provide useful data.

How well did you manage time?

From your perspective, an effective behavioral conversation would cover substantial portions of your story catalog, at least those parts that showcase the strongest examples from your career. Reflect on the pacing of your interview:

Red flags:

- You only discussed 1–2 stories in 45 minutes (probably too much detail)
- The interviewer seemed rushed at the end, skipping questions (you took too long on early responses)
- You finished 10+ minutes early (responses were too shallow or the interviewer gave up)

Good signs:

- You covered 5–7 different signal areas with appropriate depth
- The interviewer had time for their questions but not excessive awkward silence

- The last 5 minutes felt purposeful, not rushed or dragged out

Am I satisfied with story choice?

Evaluate your story choices. For each story:

- **Did I choose the best story from my catalog?** Sometimes in the moment, you default to a familiar story when a different one would have provided stronger signal. Review your Story Catalog and identify whether a better option existed.
- **Did I emphasize the right aspects?** Perhaps you chose the right story but focused too much on technical details when the interviewer wanted to hear about stakeholder management, or vice versa.
- **Did I answer the question asked?** It's surprisingly easy to pivot to a story you're comfortable with rather than one that directly addresses the question. If the interviewer asked about ambiguous projects and you talked about large scope but straightforward ones, then you missed the opportunity.

Identify next steps and implement them

Take your observations and translate them into concrete changes in your Decode, Select, Deliver cycle.

Update your story catalog

Based on the interview experience, refine your catalog:

- **Add clarifying details.** If you received a clarifying follow-up question you struggled to answer (*How long did this project take?* or *How many people were involved?*), add that information to your Story Catalog so it's readily available next time.
- **Adjust story emphasis.** If you realize you spent too much time on technical details and not enough on business impact, add a note to your catalog entry: "Remember to front-load the 40% revenue increase when telling this story."
- **Flag stories that didn't work.** If a story fell flat or didn't provide the signal you expected, note why: "Datacenter migration story: seems impressive but too infrastructure-focused for product team interviews. Save for platform team roles."
- **Identify gaps.** If you struggled to answer a question because you lacked a relevant story, note this gap and consider whether you have experiences

you’ve overlooked or whether you need to acknowledge this as a genuine weakness.

Practice the stories that struggled

If you stumbled through a story or realized afterward that you chose poorly, add it to your next practice session. Record yourself telling it again with the improvements you’ve identified. Get feedback from a peer or professional mock interviewer specifically on that story.

Learn from rejections

If you receive a *No Hire* decision, the sting of rejection can make reflection difficult. But failed interviews are often more instructive than successful ones, revealing blind spots you can’t see when things go well.

- **Request feedback from your recruiter.** Most companies won’t tell you anything for legal reasons, but it’s always worth asking. Even vague feedback like “the committee wanted to see more evidence of leadership” or “concerns about scope” gives you direction.
- **Consider whether your Story Catalog needs work.** Repeated rejections at a particular level suggest either your stories aren’t demonstrating the expected scope, or you need to adjust how you’re telling them. This might be the time to invest in a professional mock interview for calibration.

This kind of reflection and action transforms you from someone who “does interviews” to someone who systematically improves at interviewing. The difference compounds over your career, both in landing your next role and in the perpetual low-level interviewing that happens in performance reviews, promotion discussions, and internal mobility conversations.

The most successful candidates keep these principles in mind even when not interviewing, updating their Story Catalog after completing major projects, for example. They reflect on what made them successful so they can articulate it later. They request feedback regularly so they’re never surprised by how others perceive their work.

Whether your last interview resulted in an offer or a rejection, you now have more information than you had before. Don’t waste it.

Table: Story Catalog Example

<i>Story</i>	<i>Context</i>	<i>Actions</i>	<i>Results</i>	<i>Learnings</i>	<i>Signal Areas</i>
API Performanc	Customer complaint	Identified bottlenecks,	60% faster	Early communicatio	Ownership, Perseverance,

e Optimization	s about slow response times	documented them; implemente d caching, collaborated with infrastructur e team via regular meetings, ...	response s, reduced customer churn by 15%	n with partners key to success	Communicatio n
-------------------	--------------------------------------	---	--	--------------------------------------	-------------------

PM Prioritization Conflict	Payment processin g 75% failure rate; PM wanted new features	Gathered failure data; projected business impact; scheduled 1:1; proposed timeline for fix and later feature developmen t	PM agreed to 2-week infra sprint; saved \$10k monthly revenue loss	Importance of understanding business context, data- driven decision making	Conflict Resolution, Communicatio n, Ownership
----------------------------------	---	---	---	--	---

[1] Much of the practical inspiration for this part comes from *The Artist's Way* by Julia Cameron. [↑](#)

[3] Astute readers will see the embedded CARL story framework here. See [Why CARL is Better than STAR](#). [↑](#)

[4] Get a story catalog template. [↑](#)
<https://thebehavioral.tech/p/story-catalog>

[5] *Paper Notebooks vs. Mobile Devices: Brain Activation Differences During Memory Retrieval*, Umejima et al., 2021. [↑](#)
<https://thebehavioral.tech/p/paper-vs-mobile-journaling>

[6] *Stepping back to move forward: Expressive writing promotes self-distancing*,

Park et al., 2015. [↑](#)

<https://thebehavioral.tech/p/writing-promotes-self-distancing>

[7] *Learning by Doing: A guide to teaching and learning methods*, Gibbs, 1988.

[↑](#)

<https://thebehavioral.tech/p/professional-clarity-research>

[8] Reference [Appendix: Leveling at Tech Companies](#) for expectations at each level. [↑](#)

[9] IPO: Initial Public Offering [↑](#)

[10] PRD: Product Requirement Document

RFC: Request for Comment [↑](#)

[11] Don't get too focused on these numbers. Early career candidates will have fewer stories. [↑](#)

[12] Review [Deliver: Telling a Good Story](#). [↑](#)

[13] I encourage my coaching clients to think about how Steve Jobs would describe this project. [↑](#)

[14] See [Questions About Weaknesses, Mistakes, and Regrets](#). [↑](#)

[15] See [Telling Longer Stories](#) for more about the Table of Contents. [↑](#)

[16] ML: Machine Learning [↑](#)

[17] NLP: Natural Language Processing [↑](#)

[18] POC: Proof of Concept [↑](#)

[19] SRE: Site Reliability Engineering [↑](#)

[20] DBA: Database Administrator [↑](#)

[21] Death, Taxes, and TMAY [↑](#)

<https://thebehavioral.tech/p/tmay>

[22] Don't use flattery like, "It's been my life's dream to interview for this company," or otherwise delay the proceedings. The interviewer has 45+ minutes of questions to ask you. [↑](#)

[23] See a further discussion in [Handling Sensitive Conversations](#) [↑](#)

[24] This statement is true if you've had at least a year of experience. Evaluating students requires a more holistic and potential-based approach vs. a results-based approach. [↑](#)

[25] It can be hard to choose a single story. Consider the Menu Technique to enlist the interviewer's help in choosing. [↑](#)

[26] *Beyond Cracking the Coding Interview* by McDowell, Mroczka, Lerner, and Mamano, 2025. [↑](#)

[27] Much of these conflict resolution and feedback delivery best practices was codified in Kim Scott's *Radical Candor*, which is well-known and respected in Silicon Valley's management circles. [↑](#)

[28] CARL: Context, Actions, Result, Learnings. See [\[→\]](#). [↑](#)

[29] This is Covey's Habit #5 from *The 7 Habits of Highly Effective People*. [↑](#)

[30] PR: Pull Request [↑](#)

[31] See [The Halo Effect: Getting Mileage Out of Lists](#). [↑](#)

[32] See [Questions About Weaknesses, Mistakes, and Regrets](#). [↑](#)

[33] Beginning paid mocks too early can waste your time and money. Build a solid foundation first. See tips for paid mocks [\[→\]](#). [↑](#)

[34] CARL: Context, Actions, Results, Learnings. See [Why CARL is Better than STAR](#). [↑](#)

[35] This is a book. Go explore the latest in AI interview practice tools. I will not attempt to keep up. [↑](#)

[36] *Behavioral Prep on a Budget: Peer Mock Interviews* [↑](https://thebehavioral.tech/p/behavioral-prep-on-a-budget-peer)
<https://thebehavioral.tech/p/behavioral-prep-on-a-budget-peer>

[37] *Getting the Most Out of Mock Behavioral Interviews* [↑](https://thebehavioral.tech/p/getting-the-most-out-of-mock-behavioral)
<https://thebehavioral.tech/p/getting-the-most-out-of-mock-behavioral>

PART 3

REFINING RESPONSES

Beyond the basics, toward mastery of storytelling and communication skills.

PITFALLS BY SIGNAL AREA

There are common concerns an interviewer may have while listening to your stories. These could turn into red flags damaging your candidacy so review them after you prepare your core stories.

Scope

- When describing projects, do you articulate the complexity and scale of the work? Consider using measures like number of components, time, number of teams or people involved.
- Are the projects you're choosing representative of your level?[\[1\]](#)
- Are you claiming what you're doing or just saying what happened?

Initiative

- How did you realize the need for action?
- Does this demonstrate self-starting behavior or just task completion?
- What would have happened if you didn't act?
- Did you show willingness to go beyond your assigned role? Was this your job to begin with?
- How did you communicate what needed to be done?
- How did you arrive at alignment to complete the work?

Perseverance

- Did you articulate the challenge and why it was difficult, selling the scope completely?
- How did the challenge get discovered?
- What was the impact if this wasn't resolved?
- What did you do to overcome the challenge?
- Are you showing sustained effort over time rather than achieving quick wins?
- Did you take follow-up actions after the challenge?
- Are you explaining how you kept yourself and others motivated?

Ambiguity

- Did you land the scope of the ambiguity? How could you quantify this?
- What methods did you utilize to tame the ambiguity? Are you demonstrating a structured thinking and approach?
- Did you have to communicate and clarify for other people?
- Are you biasing toward action, taking action when things are unclear?

Communication

- When describing your projects, did you highlight the methods and reasons why you communicate with others?
- Are you highlighting both written and verbal communication?
- Are you demonstrating communication at different levels of the organization (across, down, up)?

Leadership

[\[2\]](#)

- Did you show influence over others? How was that related to or not related to the position you held?

Growth

- If you're mentioning a mistake, was it a real mistake?
- Was it too obvious in the beginning to make that mistake?
- If you were receiving feedback, how did you take it?
- Did you notice the mistake or growth area or did you need someone to point it out for you?
- Did you internalize what you learned and make the most of the feedback or failure opportunity?

- Did you actually overcome the challenge? Did you apply the learning in future opportunities?

Conflict Resolution

- Was it a conflict reflective of your level?
- Did you seek to understand before being understood? Did you show empathy?
- Were you directly involved in resolving the conflict? What action did you take?
- Did you need help from others to resolve the issue?
- How did it resolve? Was it a win-win resolution?
- What was the state of the relationship after the conflict was resolved?

IMPROVING YOUR COMMUNICATION

The questions in a behavioral interview are simply a frame around which you can build. Answering the questions is not the goal, but instead delivering a compelling case for your candidacy while not looking like a salesman.

This chapter offers a collection of advanced techniques, mental models, and tactical approaches for candidates who've mastered the fundamentals and are ready to take more control of their interviews. If you're comfortable with basic story structure and understand what signal means, these concepts will help you move from reactive to purposeful.

The Golden Rule: Guide the Interviewer

You and the interviewer share a time problem. You have 45 minutes to convey the most relevant parts of a career you know intimately. They have 45 minutes to extract enough signal to make a confident hiring decision.

Most candidates approach this as a one-way street: the interviewer asks questions, you answer them. But this puts your success entirely in the interviewer's hands. What if their questions don't naturally lead to your best examples? What if they spend time on less relevant areas?

Instead, treat the interview as *mutual exploration*. Yes, the interviewer is directing the conversation, but you have agency too. Here are some tactics and mental models you can employ to guide them there.

Feel free to redirect the interviewer

Sometimes the question you're asked isn't the best path to your strongest signal. That's fine, you can redirect.

Did you have any conflicts in this project?

✓ *There were some small disagreements in this project, but the more interesting conflicts are from another project. Can I talk about those?*

Be careful not to overuse this. One or two strategic redirections in an interview is good; constant redirection feels evasive. Use it when you genuinely have a better example that provides richer signal for the competency they're evaluating.

Presenting your most compelling Actions: the Merchant and the Miner

When presenting your experience, you can be either a merchant or a miner. Both have valuable gems to offer, but they display them differently.

The merchant arranges their gems in beautiful settings, under perfect lighting, in easy-to-view glass cases. Everything is organized and accessible. When a customer asks to see something, the merchant knows exactly where the most impressive pieces are and can guide them directly there.

The miner has equally valuable gems, but they're embedded in the mountainside. The customer has to dig for them, chipping away at rock, hoping they find the good stuff. Maybe they will, maybe they won't.

Be the merchant:

- **Know your best material.** For each story, identify the most compelling actions and inflection points before the interview. Don't make the interviewer excavate them.
- **Emphasize your agency.** Frame things as *I decided...* or *I proposed...* rather than *The team did...* or *It happened that...* Your actions are the gems—display them prominently.
- **Make your value obvious.** Don't bury your most impressive work in the middle of a long story. Put it where the interviewer can't miss it.



Telling the interviewer what's available: the Project Mural

Imagine trying to understand a detailed mural on a wall—but you can only view it through a straw. You see tiny fragments at a time. You're not sure where to look. You might spend all your time examining an unimportant corner while missing the action in the center.

This is what an interviewer experiences trying to understand your complex projects through verbal communication, which is inherently low bandwidth. They're seeing fragments, trying to piece together what matters.

[3]

Guide their vision to the right places:

- **Show them the structure upfront.** Use techniques like Table of Contents: *This project had three phases. The most challenging was the third, where we had to...* You're giving them a map.
- **Know what matters most.** Before the interview, identify which parts of each story contain the richest signal for this role. When you're telling the story, nudge them toward those areas.
- **Zoom in on high-signal moments.** Don't treat every part of your story as equally important. Spend time on the decisions, challenges,

and actions that best demonstrate your capabilities.

Adapt to Your Interviewer's Style

In my experience, there are generally two styles of interviewers. You should be able to determine in the first few minutes which interviewer you're working with and adapt accordingly:

The conversationalist

They ask fewer, broader questions and let you talk. They listen carefully, ask thoughtful follow-ups, and try to extract multiple signal areas from a single story. They create a comfortable atmosphere and give you space to share your thinking. The interview feels more like a discussion between colleagues than an interrogation.

This works for them because they believe the best signal comes from understanding how you think through complex situations. They want to hear your reasoning, your tradeoffs, your learnings. To them, one rich story can reveal more than five surface-level ones.

Here's how to be successful with these interviewers:

- **Manage your time ruthlessly.** The biggest trap with conversational interviewers is they'll let you talk too much. They won't interrupt you, even when you're telling the wrong story or belaboring signal you've already covered.
- **Watch for cues that you've given them enough on a particular signal area.** If they've stopped taking notes or asking follow-ups about a specific aspect, you've probably established that signal. Move on.
- **Front-load your most important stories.** Don't save your best example for question five—you might not get there. If they ask about leadership early, use your strongest leadership story.

The examiner

They rapidly fire questions, seeking quick responses. They often interrupt to move to the next question. They may work from a checklist or rubric, methodically ticking through competencies. The pace feels brisk, maybe even transactional.

This works for them since they need to assess lots of signal areas in a tight window and they've learned that letting candidates ramble wastes time. They'd rather hear 30 seconds about ten different situations than ten minutes about one project. Their goal is breadth of evidence, not depth.

Here's how to be successful with these interviewers:

- **Stay calm.** Being interrupted doesn't mean you're failing—it means they got what they needed from that part of your answer. Don't get flustered or feel like you did something wrong. This is just their style.
- **Front-load value immediately.** With conversational interviewers, you can ease into a story. With examiners, you can't afford that luxury. Lead with the headline:

Tell me about a time you dealt with ambiguity.

✗ *So this was back in 2022 when I was working at TechCorp on the platform team. We had this project that was starting up and the requirements weren't very clear...*

✓ *I led a platform migration with no clear requirements or success metrics. Here's how I created structure: First, I...*

- **Compress your context.** Get to your actions faster.
- **Feel free to redirect back.** If they interrupt you before you've shared a critical action or result, redirect them:

Before we move on—one key thing from that project was [important action/decision]. That became the template for how we handled similar situations later.

- **Use the Menu Technique aggressively.** When they ask a broad question, give them options so they get exactly what they want to assess.

Tuning the Level of Detail

[4]

So much of communication is deciding what information to include and exclude. Returning to our discussion in [Part 1](#) on details, here are frameworks for making better decisions about what details to include and where to include them.

What details actually accomplish

In addition to providing enough context to understand your Actions and establishing your credibility as we discussed in Part 1, details also:

- **Create opportunities for follow-up questions.** Details are natural places to dig deeper, letting you guide the conversation toward your strongest areas. When you mention *I refactored our authentication flow to use JWT tokens*, you're creating an opening and an interviewer interested in security might ask about token expiration and refresh strategies. One focused on system design might ask about stateless vs. stateful approaches. This is particularly valuable when you're less confident about certain aspects of a story. By emphasizing the details you're strongest on, you make those the most natural follow-up targets.
- **Make your story memorable.** Entertaining or vivid details help your story stick in the interviewer's mind. Consider how a listener might remember you as someone who made an impression versus simply accomplished a task. For example: [\[5\]](#)

✓ *When I first proposed the migration plan in our architecture review, our Principal Engineer—who'd been at the company for twelve years—said it was the most thorough technical design doc he'd seen. That feedback meant a lot, but it also made me realize I'd set a new bar for myself that I'd need to maintain.*

Handling technical detail

Calibrating technical detail is one of the hardest things to get right. Go too shallow and you seem non-technical. Go too deep and you lose your audience or seem unable to communicate clearly.

Adjust depth to the role. The technical depth should match what matters most for your role's impact. Infrastructure engineers typically need to go

deeper into system-level concerns, while product engineers might focus more on user-facing impact and tradeoffs.

Consider “Going Deep” just once per response. Your mileage may vary on this one, but if you include technical detail in one particular portion of your response, you’ve struck a balance between the benefit of establishing your credibility, and the time and possible confusion additional detail could bring.

Explain complexity simply. For senior roles, being able to explain complex topics simply to an intelligent person is one of the hallmarks of excellent communication. If you can’t explain your technical work to someone with a solid background in tech, you probably don’t understand it well enough yourself.

Assuming your audience is not an expert, the following might be too complex, establishing your technical credibility but not demonstrating strong communication skills.

 *I implemented a CRDT-based conflict resolution strategy using a vector clock mechanism with causal ordering to ensure eventual consistency across our replicas while maintaining semilattice properties for our merge function.*

Try something like this, with appropriate similes:

 *I implemented a conflict resolution system that lets multiple users edit simultaneously without losing anyone’s work. Think of it like Google Docs—everyone can type at once, and the system intelligently merges changes. I used CRDTs, which are data structures designed specifically for this “eventual consistency” problem. The key insight is that merge operations need to be commutative—meaning the order doesn’t matter—so I designed our data model to have that mathematical property.*

How to handle jargon and acronyms

Every field has jargon and acronyms, and you’ll need to decide what requires explanation and what doesn’t. This is more art than science.

Some examples to calibrate you. These are just my opinions, but I consider myself a reasonably well-informed, technical person.

- **No explanation needed:**

- AWS, Docker, Kubernetes, SQL, REST APIs, microservices (for engineering roles)
- A/B testing, conversion rate (for product/growth roles)
- Figma, design system (for design roles)
- **Brief explanation helpful:**
 - AWS IoT Greengrass: *which extends AWS services to edge devices*
 - CRDT: *conflict-free replicated data types, which are data structures designed for distributed systems*
 - Eventual consistency: *where all servers converge on the same state, but might temporarily show different values*
- **Longer explanation needed:**
 - Highly specialized protocols or tools specific to your domain
 - Internal company systems or frameworks[\[6\]](#)
 - Academic or research concepts not widely used in industry

This line evolves over time. For example, five years ago, “vector database” probably needed explanation outside of ML circles, but is likely considered common knowledge today.

Have mercy, they are only managers. Your interviewer is likely a people manager, which means it might have been a while since they had their hands on a modern technical system. Think about where a reasonably intelligent person with a solid technical background would need help, but err on the side of providing brief explanations.

Give a “One-Breath” explanation

When you do need to explain jargon, keep it to one breath, like a subordinate clause, and emphasize the takeaways for the listener:

✓ *I implemented our feature flags using LaunchDarkly—a third-party service that manages feature toggles—which let us deploy to production but control rollout percentage.*

The explanation “a third-party service that manages feature toggles” takes three seconds and provides just enough context. Then you immediately return to your main point about why this mattered.

Another example:

✗ *I implemented a canary deployment strategy leveraging our Spinnaker CD pipeline with K8s HPA configured for our pods, integrating Prometheus metrics via our service mesh sidecar proxies to trigger automated rollbacks based on our SLO burn rate calculated from our SLI thresholds.”*

Even for infrastructure engineers, this is too dense. You’re name-dropping tools without explaining the actual problem or solution. Better would be:

✓ *I implemented a canary deployment strategy—where we gradually roll out changes to small percentages of users—using our existing deployment pipeline. I configured it to automatically monitor error rates and latency. If the new version showed problems compared to the old version, it would automatically roll back. This meant we could deploy confidently even during business hours, knowing that any issues would affect at most 5% of users and only for a few minutes.*

This version explains the concept (canary deployment), describes the monitoring simply (error rates and latency), and focuses on the outcome (safe deployments during business hours). Tools are de-emphasized; the approach and impact are centered.

Leverage the interviewer’s previous experience

Tech workplaces are more similar than they are different and teams experience the same kinds of challenges—technical, product, and people challenges—all over the world. To save time and improve the signal per minute ratio, you can and should leverage this fact to avoid “tech-splaining” a common situation like technical debt to a behavioral interviewer:

✗ *I was working with an engineering team that had accumulated a number of shortcuts taken in the code that needed to be fixed later. The engineers were saying they needed more time to address this technical debt before we could ship new features. Technical debt happens when you build things quickly and don’t have time to go back and clean up the code, so it becomes harder to maintain over time...*

✓ *Engineering pushed back on our Q3 roadmap because of accumulated technical debt. Rather than debate priorities...*

Recognizing these patterns allows you to skip past the setup and get straight to what made your approach distinctive. You should still provide

context when your specific situation had unusual constraints, when the industry norm doesn't apply (e.g., highly regulated environments), or when the way the problem manifested was atypical.

If you are early in your career, it can be hard to know if a situation is so common it can be assumed to be understood by the interviewer. Here's a (far from exhaustive) list to help you calibrate:

Common project situations:

- Project behind schedule due to scope creep or resource availability change
- Launch delayed because of unexpected dependency or bug
- Feature had to be cut or descoped to meet a hard deadline
- Deciding whether to build, buy, or integrate a third-party solution
- Migrating to a new system/architecture while maintaining the existing one
- Balancing new feature work against maintenance and reliability improvements

Common technical challenges:

- Production outage or performance degradation affecting users
- System scaling issues as usage grows
- Post-launch bug or incident requiring immediate response
- Balancing speed of delivery with code quality and testing
- Legacy system causing problems but too risky/expensive to replace immediately
- Security vulnerability discovered that requires urgent remediation

Common conflicts (listed in Part 2 [\[⇒\]](#)):

- You or a stakeholder are unsatisfied with an implementation or process choice
- You or a stakeholder wants some work item and the other pushes back for time or cost reasons
- Stakeholder is unhappy with implementation quality and demands changes
- You need something on a certain timeline from a partner team but they are resistant

- You or another coworker wants to adopt a new process/technology and there's some resistance

Common leadership situations:

- Team member underperforming or missing deadlines
- Team member burned out or struggling with work-life balance
- Having to deliver negative feedback or a disappointing performance review
- Managing team morale during a difficult project or organizational change
- Team member wants to work on different projects than assigned
- Disagreement within the team about technical approach or priorities
- Onboarding a new team member or ramping someone up on a complex system
- Advocating for your team's needs (headcount, resources, recognition) to leadership
- Two team members have interpersonal conflict affecting collaboration
- Deciding how to distribute high-visibility work among team members

Common organizational/cultural challenges:

- Insufficient documentation or knowledge transfer when someone leaves
- Communication breakdowns between distributed or cross-functional teams
- Misalignment between team goals and company priorities
- Process is slowing down the team but there's resistance to changing it
- Decision-making authority unclear, leading to delays or conflicts
- Team gets reorganized or leadership changes, creating uncertainty

For further calibration, here's a list of concepts *not* generally understood. Be sure to explain these unless you know your audience is a domain expert:

- **Domain-specific regulations and compliance:** Details of HIPAA, SOC 2, FDA approval processes, PCI
- **Industry-specific terminology:** Ad tech (DSP, programmatic bidding), fintech (ACH, reconciliation), healthcare (HL7, FHIR), hardware (yield rates, fab capacity)
- **Company-specific tools and processes:** Internal code names, proprietary frameworks, custom-built infrastructure, non-standard role titles, or unique organizational structures
- **Unusual business models:** Marketplace dynamics (unless widely known), specialized enterprise sales cycles, government contracting, non-profit constraints, or niche monetization approaches
- **Highly specialized technical areas:** Specific ML model architectures, blockchain mechanisms, or embedded systems constraints
- **Geographic or market-specific context:** Country-specific payment methods, regional regulatory environments, or cultural behaviors that affect product decisions

Adjust detail according to type of question



Tailor your responses to what your interviewer needs to hear *at that moment in the discussion*. When you do this well, you'll avoid over-explaining, keep the conversation engaging, and ensure the interviewer gets the right signals to assess your fit for the role. The key is following the lead of your interviewer and recognizing the *type of question* they're asking.

Trunk, Branch, and Leaf Questions

Think of interview questions proceeding like the parts of a tree:

Trunk Questions: These introduce a new topic and set the foundation for a series of follow-up questions.

Branch Questions: These dive deeper into specific aspects of your answer.

Leaf Questions: These clarify minor details or seek very specific information.

If you know where you are in this “tree”, you can tailor your response accordingly—giving just the right amount of detail.

Here's a sequence of questions you might encounter in an interview:

TRUNK

Tell me about the project you're most proud of?

[You respond with a CARL story about how you built a threaded comments feature into your discussion platform]

BRANCH

Were there any interesting conflicts you encountered during this project?

[You respond with a story about a conflict you resolved by collecting data from a small user group. The interviewer is concerned that you made a hasty decision based on limited data and they ask...]

LEAF

So based just on this small user group, you made a decision? How big was this group?

[You answer this question with a reference to additional beta testing you did]

The interviewer is drilling down to gather more specific information with each subsequent question and is looking for more and more condensed answers as they progress.

Handling follow-ups: STAR/CARL is not always your friend

[\[7\]](#)

Many candidates default to the STAR or CARL method for every question. While this works well for trunk questions, it's not always the best approach for branch, and almost never the best approach for leaf questions:

- **Trunk questions** → Use CARL to structure your response [\[8\]](#)
- **Branch questions** → Answer directly, then provide a structured story if needed
- **Leaf questions** → Give a concise, direct answer

Let's return to the earlier example to see how this works in practice, picking up at the branch question:

Were there any interesting conflicts you encountered during this project?

BRANCH

✓ Yes, I had a conflict with the PM over whether to allow arbitrarily nested comments that we resolved by gathering some UX feedback.

Since the beginning of the threaded comments work, there had been an open question as to whether to support arbitrarily nested comments, like Reddit, or collapse the context to just two levels, like Facebook. For our application, I felt like it would be better to have just two levels, since it simplifies the experience for our less technical users.

About one week into the work, we really needed to make a decision since it affected the backend data schema. I asked the PM and they felt strongly that users would want the additional flexibility of more nesting depth.

I was thinking of just giving in but it also came with a lot of implementation complexity, so I proposed to run a quick user experience test and bring the data back to the team and the PM agreed. I used the set of users we've talked with before and worked with the designer to mock up both options and asked for opinions. It was overwhelmingly in favor of the simpler version.

[9]

Once I had this result, I messaged the PM about it to give them a heads up, then called a quick meeting including the PM and showed the team the results. We all agreed to go with the simpler approach. Usage of the feature seems strong and no one has asked for more layers of comments so I think that was the right decision, plus the PM appreciated how I messaged them early instead of surprising them with the result during the meeting.

Looking back, this should have been something we did earlier in the process, instead of leaving this kind of conflict to linger until the implementation phase, but I was glad we had the user experience group within easy reach to help us decide.

In that example, you got immediately to the point, answering the interviewer's question but also highlighting an action you took (collecting data) before continuing with the CARL story about the conflict. You knew to respond to this branch question with a full CARL story because Conflict Resolution is one of the core signal areas interviewers assess.

For leaf questions, the interviewer just wants a direct response, either to clarify something or collect additional signal on an aspect of your skills:

So based just on this small user group, you made a decision? How big was this group?

LEAF

✓ *It was small, like 5 people, but they had been right before, plus we also ran a survey during the Beta phase which also included a question about comment depth.*

You realized the interviewer was concerned about how you may make rushed decisions, so you added the additional validation you performed—but it was quick, no CARL story.

At every stage of an interview, your goal is to give the interviewer the right signal with minimal fluff. If they ask a broad question, structure your answer clearly. If they ask for specifics, give them exactly what they need—nothing more.

How Fast Should You Speak?

The question of speaking pace comes up frequently—both from candidates wondering if they speak too fast or too slow, and from interviewers complaining about one or the other. While admittedly tactical, it does matter: speaking quickly can signal intelligence and confidence, while speaking too slowly can undermine your credibility.

Check out the video in the accompanying QR code for examples of what I personally consider the slowest and fastest speeds for candidates. My assessment assumes conversations between native English speakers. Interviewers generally make allowances for candidates speaking in their second language.

[\[10\]](#)

TOWARD BETTER STORIES

After you're comfortable with basic CARL stories, it's time to make your stories more compelling, strategic, and entertaining, if that were possible for career stories.

This chapter has techniques and discussions for this—not tricks or manipulation but helping you deliver the strongest story in the limited time you have.

Avoid Rigid Storytelling

Ultimately the CARL response format is just a guide and like any other framework, you should seek to understand why it works then employ those principles instead of blindly following the format.

Following a method like STAR or CARL gives your story a natural arc, which is pleasing and engaging to the listener, but sometimes the question format lends itself to different response structures.

[\[11\]](#)

For example, *Tell me about your experience mentoring engineers*, is drifting away from a “Tell me about a time...” format and an ideal response would get straight to the point:

I've mentored engineers in three different contexts:

- 1. Directly as a people manager,*
- 2. As a peer, but being assigned as a long-term mentor, and*
- 3. In an ad-hoc way, to peers I've engaged with.*

✓ *In each of these cases, I've primarily focused first on building rapport and trust through vulnerability, and an interest in their personal and professional lives. Then, I understand where I can add value via something akin to the Socratic method—I've found this works best to defuse resistance, especially in senior folks who have a lot of personal value tied up in their performance.*

With this kind of approach, I've promoted a number of people to staff-level positions, made some crucial adjustments to the trajectory of peers' careers, and improved the team culture around me, while maintaining trust.

For example, ...

This is direct and front-loads the value delivered, then backs it up with examples—you could almost describe it as RLCA (Results, Learnings, Context, Actions).

The point is that once you practice CARL a number of times, be flexible how you respond to interviewers' questions based on what signal they're trying to acquire from you.

Raise the Stakes Early

“What would Steve Jobs say about your project?” This question has helped many of my coaching clients reframe how they introduce their story Context. It forces them to articulate why the project mattered to the product, the users, the company.

People want to work on things that matter. Interviewers want to hire people who've worked on things that matter. If you open your story with flat, procedural context, you're missing an opportunity to establish that what you did was significant.

[\[12\]](#)

Kurt Vonnegut famously mapped common story arcs on simple line plots, tracking a character's fortune over time. Your behavioral interview stories follow what he called *Man in a Hole*: the story opens with trouble (the character falls into a hole), and the narrative is about how they climb out.

[\[13\]](#)

The deeper the hole, the more impressive the climb.

The Context is where the listener learns about the hole. How difficult was the challenge? What was at risk if you failed? Then your Actions become the record of the climb out.

✗ *I was working on a data pipeline project at my previous company. We needed to migrate from our legacy system to a new cloud-based infrastructure. There were some technical challenges and timeline pressure. The project took about six months and involved coordination across several teams.*

There's no hole here. It's just a description of tasks.

✓ *Our data pipeline was failing 2–3 times per week, costing us real customer trust. Each failure meant delayed insights for our largest enterprise clients—we were literally watching deals slip away in sales calls. We had six months before our biggest customer's contract renewal, and*

they'd made it clear: fix the reliability or they'd move to a competitor. The legacy system was so fragile that the team was afraid to touch it, but the new cloud infrastructure had never been proven at our scale.

In this example, we've added a number of elements to deepen the hole:

[\[14\]](#)

- The stakes were high (customer trust, losing a major customer, revenue at risk)
- The timeline was tight (contract renewal deadline)
- The challenge was technical (fragile legacy system, unproven new system)

The Menu Technique: Letting Your Interviewer Choose

[\[15\]](#)

Sometimes the best way to ensure you're delivering the signal an interviewer is looking for is to let them choose from your available stories. The *Menu Technique* is simply offering a choice of 2–3 relevant stories, describing their key takeaways, then asking the interviewer which they'd like to hear in detail.



Time is short in an interview and this approach transforms you from someone guessing at what the interviewer wants, potentially spending precious minutes on sub-optimal stories, into someone who adapts in real-time to deliver maximum signal.

When to use the menu technique

Interviewers don't want options for every question they ask. If there's an obvious story choice go with that one. But there are some situations where the Menu Technique will be well received:

You have stories with very different takeaways. When your available stories demonstrate different aspects of your capabilities, offering options ensures you showcase the skills the interviewer wants to see. For example, one conflict story might demonstrate technical leadership while another shows cross-functional relationship management.

The interviewer has shown an inclination for specificity. If an interviewer has already asked for “another example” or “something different,” they’re signaling that they want particular types of evidence. Rather than guessing again, offer them choices.

You’re in a time-sensitive part of the interview. When the interview is running short and you want to maximize signal delivery in the remaining time, the Menu Technique ensures you don’t spend precious minutes on a story that doesn’t address their core concerns.

You don’t understand what the interviewer really wants. Sometimes interview questions are ambiguous or could be interpreted multiple ways. Instead of picking the wrong angle, present options that cover different interpretations.

Follow-up interviews or targeted signal gathering. When you know an interviewer has been specifically tasked with assessing certain competencies, offering a menu helps them efficiently gather the exact signal they need.

How to structure a menu

When presenting options, briefly describe each story and **explicitly state what signal areas or insights the interviewer will gain.** This helps them make an informed choice rather than just picking randomly.

✗ *Tell me about a time when you resolved a conflict.*

Well, I resolved a conflict while working at the fintech startup and a conflict I had in my most recent job. Which one would you like to hear?

The interviewer has no context on which one would provide the signal they’d like to collect. Instead try this:

✓ *I have a couple of examples I could share that demonstrate different aspects of conflict resolution.*

First, I have a story about resolving a technical disagreement with another senior engineer over our microservices architecture—this would show you how I handle peer-level conflicts using data and prototyping to drive decisions.

Second, I could tell you about a conflict I had with a product manager over feature prioritization during a tight deadline—this one demonstrates how I navigate cross-functional disagreements while balancing business and technical constraints.

Which one would you like to hear?



The Halo Effect: Getting Mileage Out of Lists

[\[16\]](#)

Sometimes you have relevant stories for a question but none of them are that great to tell individually. At other times, you might want to land a message of scope—perhaps the breadth of your experience or the depth of complexity of a project—in as little time as possible. In these cases, you can turn to lists and leverage the *Halo Effect* to make an impression.

The Halo Effect is a psychological principle where a positive impression in one area influences perceptions in other areas. Coined by psychologist Edward Thorndike in the context of military leadership, this phenomenon also affects how people evaluate each other in the workplace, influencing decisions about hiring, promotions, and collaborations. For example, someone perceived as organized might also be seen as more competent, even if the evidence doesn't fully support that.

[\[17\]](#)

By listing accomplishments or steps succinctly in a behavioral interview, you can harness the Halo Effect. Communicating a breadth of achievements fosters the impression that you're highly capable, which can positively

influence the interviewer's perception of your overall competence and communication.

The Halo Effect in a TMAY

A common place you can employ the Halo Effect is in the Tell Me About Yourself (TMAY) [\[→\]](#). Here's the beginning of a TMAY we can work to improve:

✗ *In my last role, I led the implementation of a design system that reduced UI development time by 30% and ensured consistency across the product.*

Before that, I worked on a customer feedback platform where I integrated a machine-learning model into the frontend to provide real-time insights.

[\[18\]](#)

I also improved the CI/CD pipeline by identifying a set of needed tests after talking with the senior engineers and stakeholders from different departments. I also developed front-end features, like search filters, ads management features, an upsell for new customers. And I refactored and improved backend APIs, helping us move from a monolith to microservices in our NodeJS codebase. Oh and I collaborated with PM, design, and the CEO to identify new requirements for a series of holiday sales over the Black Friday weekend...

This is simply too detailed. It's good that you're opening with the two big projects with solid business impact, but the laundry list of smaller projects is distracting—the interviewer is likely to lose focus, awash in minutiae.

Why not simply remove the smaller projects? We could but then we're missing out on communicating the breadth of our experience.

Instead of removing them, let's employ the Halo Effect to communicate our scope using lists:

✓ *In my last role, I led the implementation of a design system that reduced UI development time by 30% and ensured consistency across the product.*

Before that, I worked on a customer feedback platform where I integrated a machine-learning model into the frontend to provide real-time insights.

I did a wide variety of other projects too: improving the CI/CD pipeline, a number of front-end features, a backend refactor, and a decent amount of stakeholder management.

This concise summary communicates a lot of scope without burdening the listener and creates the impression that you are a versatile engineer. It

also “sign posts” for the interviewer, allowing them to choose from a wide variety of areas to probe further, if they need to gather more signal to make a hiring decision.

The Halo Effect during Actions

If a deep dive into your actions isn’t appropriate—perhaps due to time constraints or the relative simplicity of the project—a list can tell a story while giving you a halo of organizational and execution skills:

✓ *To build the search filters, I started by consulting with the product owner, collaborated with the designer for initial ideas, discussed the technical approach with my senior engineer, developed a first version, gathered user feedback, added tests, and launched the final version.*

Even if all these steps took a total of a week in a small organization, listing them gives the impression of a structured and robust process.

The Halo Effect when you don’t have great stories

Sometimes, you won’t have a single standout story that fits the interviewer’s question. However, you might have several related or smaller stories. A list can help you leverage these collectively to create a positive impression.

For example, if asked about times you resolved a conflict and you have a number of shallower examples, you could try a list approach before selecting one of the stories to tell:

✓ *I’ve dealt with conflicts in several contexts, including with product leaders, other engineers, and my own manager. Let me tell you about the time I navigated a conflict with a product leader...*

This signals that you have a breadth of experience while inviting follow-up questions if the interviewer is unsatisfied with the longer story you chose to tell. It’s an application of Merchant vs. Miner.

[\[19\]](#)

HANDLING SENSITIVE CONVERSATIONS

Some questions demand careful handling, where a single misstep can derail your candidacy. Some career situations require thoughtful framing to avoid rejection. Some interviewers will interpret your responses in the least charitable light possible. This chapter shows you how to navigate all three: discussing weaknesses and mistakes authentically, addressing sticky career situations, such as layoffs or gaps, and thinking defensively to protect yourself from uncharitable interpretations.

Questions About Weaknesses, Mistakes, and Regrets

As we discussed when we introduced CARL, the ability to reflect on and improve from past experiences is a key indicator of seniority and emotional maturity, so it's natural that behavioral interviewers would ask questions about weaknesses, mistakes, and regrets. Interviewers ask these questions to assess your self-awareness, growth mindset, and ability to learn from setbacks—all critical qualities, especially for senior roles.

Admit to a real mistake or weakness

Please don't answer such a question with a disguised strength. Avoid these classic non-examples, that signal to the interviewer that you're either not self-aware enough to identify real areas for improvement, or you're being deliberately evasive:

- *I work too hard and sometimes burn myself out*
- *I care too much about details and can be a perfectionist*
- *I'm too passionate about the product and sometimes push my team too hard*
- *I have trouble saying no to additional projects*

But equally problematic are responses that highlight fundamental job-relevant deficiencies:

- *I'm not detail-oriented and often miss important requirements*

- *I struggle with time management and frequently miss deadlines*
- *I struggle to connect with the business impact of my work*
- *I have trouble working with others and prefer to work alone*

Instead, choose real weaknesses or mistakes that demonstrate self-awareness while framing them in a way that shows you're actively working to improve:

- *I am learning to balance speed and quality...*
- *Learning when to apply new technologies vs. living with legacy technology...*
- *Getting alignment across teams sometimes takes longer than expected...*
- *Landing direct feedback in a way that makes it palatable...*
- *Balancing the need for delegation with the right structures to ensure accountability...*

If you're discussing past mistakes, ideally, select examples where the “right” approach wasn't obviously clear in advance—this shows good judgment rather than poor decision-making. See the examples in [Telling a story where you were wrong](#) for how to frame situations where your initial position was reasonable given the information available.

Try to provide evidence of change

The most compelling weakness or mistake responses include concrete evidence that you've applied your learnings since you learned them. If you have such evidence, tack it on to the end of the story to show strong Growth signal.

✓ *As a product manager at my previous company, I prioritized building a comprehensive user analytics dashboard because our largest enterprise client had been requesting detailed reporting capabilities for months. They were paying us \$200K annually and had threatened to churn if we didn't provide better visibility into their team's usage patterns. I felt confident this was the right call—we had a clear customer need, significant revenue at risk, and detailed requirements.*

[\[20\]](#)

However, after spending two months building the dashboard, we discovered that only the original requesting client actually used it regularly. Other enterprise customers found it overwhelming and continued using our basic reporting. Worse, we had delayed working on mobile optimization, which user research later showed was the top priority for 70% of our user base, including several enterprise prospects we lost during this period.

I learned that even when you have a vocal, high-value customer making specific requests, you need to validate whether their needs represent broader market demand. I also realized I had been too focused on preventing churn rather than understanding what would drive growth across our entire customer base.

Six months later, I faced a similar situation where our second-largest client requested custom SSO integration. Instead of immediately prioritizing it, I surveyed our entire enterprise customer base and discovered that 80% were also struggling with user onboarding complexity—they wanted their employees to access our platform without creating separate accounts.

[\[21\]](#)

Rather than building custom SSO just for one client, I worked with engineering to implement Google Workspace and Microsoft 365 integrations that would allow single sign-on for most enterprise customers. This addressed the original client's core need (seamless access for their team) while solving the broader onboarding friction that other customers were experiencing. This took slightly longer to build but resulted in much higher adoption and actually helped us close three new enterprise deals because prospects could see we had enterprise-ready authentication.

When to be vulnerable

Most of the time, you gain nothing from exposing vulnerabilities when you aren't explicitly asked for them.

Tell me about a change you made to the team's composition.

✗ *Well, I was leading a team that was somewhat disengaged and I was receiving some negative feedback from stakeholders on the team's performance. So I had to make some tough calls about who to move off the team...*

You've volunteered that you were leading an underperforming team, which begs questions about how the team got that way. Unless there's some

reason you provide, the interviewer will assume you created the mess yourself, and no matter how good your story about changing the team's composition, you'll still warrant a red flag for poor leadership.

Tell me about a time you had to influence without authority.

✗ *At my last company, I struggled to get buy-in from the design team because, honestly, I don't have a design background and they didn't respect my opinion on UX decisions...*

You confessed to lacking credibility with designers, but the question didn't require any such vulnerability.

Avoid these kinds of unforced errors.

There are some very limited cases when vulnerability does serve you, but I encourage you to tread softly:

When the question directly asks for it. Like we talked about, dodging these with humble-brags destroys credibility. The interviewer is testing whether you can be honest about imperfection while showing how you've grown.

When it humanizes your leadership approach. Sharing moments of uncertainty, fear, or doubt can make your leadership stories more relatable—especially for senior roles. *I was honestly scared we'd miss the launch deadline or I didn't know the right answer, so I brought the team together to figure it out* shows authenticity without undermining your competence.

When it showcases a significant learning. If a mistake led to a major shift in your approach that you still apply today, the vulnerability is worth it because it demonstrates exceptional growth. The learning becomes more credible when you admit the failure was real and painful.

Thinking Defensively

[\[22\]](#)

Sometimes the best defense is the best defense.

Gary Kasparov

Behavioral interviewers are necessarily making a subjective judgment. With only 45–60 minutes of conversation, it will be a hasty judgment, full of “reading between the lines” and “jumping to conclusions.” This isn't unfair; it's the nature of the process. Interviewers are often senior employees

who've seen their share of hiring mistakes, making them naturally cautious. And for leadership roles, this wariness intensifies since hiring a problematic manager can have cascading effects across the organization.



Given this sensitive environment, even when what you share in your stories isn't "wrong," you can still leave yourself open to uncharitable interpretations. Any place your stories can be construed to reflect negatively in some signal area creates space for doubt. Interviewers fill gaps in your narrative with assumptions and those assumptions probably won't favor you.

Here are some counter examples:

✗ *[As a Mid-Level Engineer] I came to the sprint meeting and the manager assigned me this ticket...*

- You aren't demonstrating strong Ownership or interest in the business outcomes, delivering more junior-level signal.

✗ *[As a Senior Engineer] The codebase lacked test coverage so it took me a while to put together the changes for this feature...*

[As a Product Manager] The executive didn't have any visibility into this project so it was under-resourced during the whole of its lifecycle...

- You're admitting that you notice pervasive issues around you but don't seek to remedy them.

✗ *[As a Product Manager] The stakeholders had different opinions about the feature scope, so we scheduled several alignment meetings to figure out the direction...*

- Rather than driving decision-making, you're letting ambiguity persist through multiple meetings.

✗ *[As an Engineering Manager] The team documented the API changes in our internal wiki, but some teams still had integration issues...*

- You're treating communication as a one-way broadcast rather than ensuring understanding, thus weak on Communication.

These examples aren't necessarily "wrong" actions but you can see how, absent other framing, they can be interpreted negatively. In this environment you need to think defensively, proactively addressing concerns and carefully framing stories.

Becoming aware of weak story components

[\[23\]](#)

It's hard to act defensively when you're not sure what to defend against. Here are some tips in identifying weak spots both before and during an interview:

Review your stories for negative signal: Spend some time journaling about potential weaknesses in your stories. You've probably already done that when you looked for potential follow-up questions in Step 3 [\[⇒\]](#).

Be suspicious of follow-up questions: A follow-up question is frequently aimed at solidifying the interviewer's understanding of a potential weakness. Pause for a moment and see if there's an uncharitable reason why they may have asked you that.

Leverage mock interviewers: It can be challenging to spot the weaknesses in your own stories. A professional interviewer in particular can point them out for you.

Counteracting negative interpretations

Elide difficult parts of stories: I never advocate for lying but not sharing or sidestepping problematic parts is fine. For example, instead of offering up the fact that your manager assigned you the ticket, say this:

[As a Mid-Level Engineer] I came out of the sprint meeting with this high impact task...

Proactively frame ones you can't elide: Explaining the rationale behind what you did often helps avoid uncharitable conclusions. For example, clarifying why so many meetings were required while landing some data-driven decision making signal:

✓ *[As a Product Manager] The stakeholders had different opinions about the feature scope, so we scheduled several alignment meetings to figure out the direction. We strategized extensively as a team and came in with a clear position, but the executives were at odds about whether the feature should prioritize enterprise customers or consumer growth.*

I realized this wasn't a product decision but a strategic business question, so I prepared data on user engagement and revenue impact from both segments, then facilitated a decision-making session where we could resolve the strategic conflict with concrete evidence rather than just opinions.

Acknowledge mistakes: It is much better to acknowledge a mistake you made than simply communicate it and leave it hanging there. Ideally you share how you avoided that mistake the next time.

[\[24\]](#)

[As an Engineering Manager] The team documented the API changes in our internal wiki, but some teams still had integration issues. I realized we were hasty in launching this without more coordination between us and the

customer teams. I led some changes to our launch planning process, having the tech lead add Change Management to the planning docs.

✓ **Address the concern within the follow-up:** If you can identify what the interviewer is driving at with their follow-up question, you can address that right away. Consider this exchange:

[As a Senior Engineer] The legacy system was causing performance issues, so I spent three months doing a complete rewrite to modernize the architecture...

How did you use those three months?

Why are they asking this? Could be that they're concerned it's taking too long, or that business needs were disregarded, or maybe that the rewrite is technically risky. Hard to tell, but we need to guess and take a stab at justifying our time:

✓ *I broke it into six two-week phases with feature flags, migrating one component at a time while maintaining backward compatibility. Each phase delivered measurable performance improvements, so if we'd needed to stop at any point, we would have still captured significant value. The three months was the total timeline, but we were shipping improvements to users every sprint.*

With some thought, practice, and feedback from others, you can avoid providing the interviewer opportunities to form unfavorable judgments.

Handling Difficult Career Situations

[\[25\]](#)

How should you handle sticky situations like getting laid off? Do you bring it up yourself or wait to be asked? Do you lie? Find some way to spin it? Here are some thoughts.

What are sensitive situations?

Through the course of your career, you may find yourself navigating sticky HR-related situations:

- Layoffs
- Getting fired
- Being out of work for a while

- Taking a mental health break
- Job hopping/short stints
- A failed startup

These need to be handled with care, sometimes proactively and sometimes avoided if you can.

Principles for handling sticky situations

[\[26\]](#)

Be honest: I don't know of any situation where you should lie in a behavioral interview. Explain? Yes. Adjust framing? Sure. Spin? Yeah, even that. But never lie. For one thing it compromises yourself and for another, most people can't lie very well so you'll just end up heightening the sense of concern the hiring manager may have about you.

But you don't have to confess: Honesty is not the same thing as actively divulging compromising details. If you were laid off and your manager cited performance reasons, you don't have to offer that to a hiring manager.

Be proactive: Hiring managers want to see that you've taken control of your situation. Demonstrate forward momentum in your career despite these setbacks.

Be positive: Don't dwell on the past or badmouth former employers. Accept mistakes made by yourself or others since you can't change them.

Be forward looking: Emphasize how your experiences, both before the sticky situation and since, have prepared you for the role you're applying for now.

Layoffs

Address proactively? No. Wait to be asked.

Interviewer's concern: Were you laid off due to performance issues that could resurface?

- [\[27\]](#) Ethan Evans has a great post about discussing layoffs with hiring managers on LinkedIn and here's a summary of his framework.
- If your entire team was let go, tell that story—it's less about you and more about a business choice.

- If it was not the entire team or you suspect you were let go for performance reasons, then start by framing it as part of a larger layoff, then give a plausible reason for why the layoff occurred, like your project was no longer needed or the team was overstaffed.

Getting fired

Address proactively? No.

Interviewer's concern: Were you fired for performance reasons or another red flag (like unhealthy interactions with others)?

- You're not obligated to share that you were fired.
- When talking about leaving a company, frame your leaving as a part of a career transition and that the previous role was not a fit for some work-related reason, not because you were fired.
- You can keep descriptions brief and neutral: *There was a misalignment between my role expectations and the company's evolving needs. Since then, I've taken steps to...*
- As always, stay forward looking, focused on growth: *That experience helped me realize that I thrive in [a different kind of environment]. Since then, I've honed [skill] by [taking action] and I'm excited to bring that to my next role.*

Career break

Address proactively? Yes if it's obvious from your resume and it's been "a long time," which is probably 6+ months.

Interviewer's concern: Are you out of touch with the latest technology? Are other companies passing on you because of a weakness?

- Use the response to "Tell me about yourself" to proactively frame your time out of work in case the interviewer is suspicious but doesn't ask.
- Blaming it on the "current market" (as I'm writing this in 2025) is not a bad idea, as long as you can back that up with a solid description of the things you have been doing in the meantime.
- Try an intentional or personal framing: *I took time to focus on [upskilling, family responsibilities, etc.], and during that period, I...*

- Then highlight proactive steps you've taken during your time off: ... *completed [a certification, contributed to open-source projects, and deepened my expertise in XYZ technology through personal projects, etc.]...*
- Even framing how you're going about your job search can be helpful and demonstrate strong career-ownership skills: *I've been researching B2B companies since I feel that's the best places for AI to make it's first world-changing impact, and looking for a company with a mid-size team where my end-to-end ownership and relationship building skills will really shine...*

Mental health breaks

Address proactively? No. It's hard to predict how empathetic an interviewer will be.

Interviewer's concern: Will this impact job performance?

- These are not usually visible from your resume and even if a mental health reason is why you're out of work, which is the visible evidence on your resume, it's best not to mention this. Some interviewers may be empathetic but some will be concerned—there's really no upside to discussing it.
- You can mention “personal reasons” but better would be to reframe as a career shift, employing growth language as we have done throughout this playbook: *I wanted to work in a different field than company X, so I took some time off to improve my Y skills...*

Job hopping/short stints

Address proactively? Yes.

Interviewer's concern: Are you a flight risk or opportunist?

- A hiring manager may not ask you directly but might have concerns after seeing your resume, so best to briefly mention this during your “Tell me about yourself.”
- Don't give a history lesson like, *First I was at X, then I was at Y, ...* Instead, summarize the hops in a way that makes your career directional: *I spent some time wearing different hats at different*

places but found that I enjoyed product-focused work the most and I'm looking forward to embedding myself on a team to really make a difference for users.

Failed startup

Address proactively? Maybe.

Interviewer's concern: Can you contribute to/lead a successful team?

- Often times this looks like a recent, possibly short stint, or the interviewer is concerned you might be fired.
- If you do discuss it, properly diagnose the reason the startup failed and demonstrate learning from it: *Honestly we didn't focus early enough on arriving at product-market fit and got too focused on serving the first few customers...*
- Highlight applicable skills you learned during the startup: *Wearing multiple hats helped me strengthen my ability to prioritize and drive a roadmap forward, which I know is valuable in this new role.*

Sticky situations don't have to sink your behavioral interview. When handled properly, they can showcase resilience, problem-solving, and self-awareness—all of which make you a stronger candidate.

MANAGING THE HUMAN ELEMENT

We've covered a lot of mechanics and frameworks, but let's remember, you're having a conversation with another human. Human factors affect both your performance and their evaluation of you.

[\[28\]](#)

Content remains most important, since a gracious interviewer might overlook poor delivery, but few will hire someone with confident delivery but weak stories. The ideas in this section support your delivery while staying secondary to substance.

Connecting Authentically

[\[29\]](#)

Connecting genuinely with your interviewer transcends empty “be yourself” platitudes. Research identifies several key factors that significantly influence how interviewers evaluate candidates, and understanding these can help you present your authentic self more effectively.

[\[30\]](#)

Present with confident energy. Your energy level and vocal modulation signal engagement and enthusiasm about both the conversation and the role. This doesn't mean being artificially animated, but letting your genuine interest in the work come through in your voice and demeanor. Monotone delivery or flat affect can bore interviewers, while natural variation in your speaking pace and tone keeps interviewers engaged.

Maintain natural eye contact. Consistent eye contact builds trust and demonstrates confidence without being aggressive or uncomfortable. In video interviews, this means looking at the camera rather than the screen, which requires some practice but creates the impression of direct engagement. For in-person interviews, natural eye contact during conversation—the same level you'd maintain with a colleague during a work discussion—establishes credibility and connection.

Practice fluency between thought and speech. This isn't about language proficiency but your ability to translate your ideas into clear, organized verbal communication. When you've rehearsed your key narratives, you

can focus on the conversation itself rather than struggling to find the right words under pressure.

[\[31\]](#)

Look professional. Your appearance should align with the company's culture while erring slightly toward more professional than necessary.

Create genuine moments of connection. Warm greetings, authentic smiles, and personalized elements in your “Tell Me About Yourself” response help establish rapport. Show genuine curiosity about their work and the company's challenges. When discussing your learning experiences, demonstrate real introspection rather than rehearsed humility. Take moments to truly listen to their questions and respond thoughtfully—this creates a conversational dynamic rather than a mechanical Q&A session.

If you want a lot more nuance and research about building genuine connections with people, I encourage you to check out content from Vanessa Van Edwards.

[\[32\]](#)

Handling Difficult Interviewers

I've written this entire book from the perspective of a thoughtful hiring team that's seeking to be excellent in how they evaluate candidates: seeking consistency, reducing bias, and relating the hiring process as closely as possible to the job requirements.

Not all organizations or interviewers are thoughtful or competent. Some still ask brain teaser questions or demand responses in rigid formats.

[\[33\]](#)

Here are some common situations you might encounter and how to handle them.

Interviewers that interrupt you

Some interviewers will frequently interrupt you, often to ask follow-up questions. Don't get flustered: answer their question and keep going with your story. Take stock of what they asked you about and try to work similar details into the rest of your stories since you know what they're looking for.

If they stop you for additional business context as you dive into the actions, now you know to include more of that next time. If they question

your reasoning for a technical choice, now you know to provide more of your thinking next time.

Interviewers that switch between questions quickly

As interviewing has evolved and organizations have sought to standardize their evaluation processes, the result is sometimes interviewers who bounce from question to question, clearly moving through a prepared script. This can be jarring and, of course, non-conversational.

The advice is the same as interviews who interrupt: keep your head, practice your Decode, Select, Deliver loop and know that since you've prepared your stories in advance, you know how to guide the interviewer to the signal they're looking for in each question.

Unfortunately there's not always much you can do with these interviewers. Guide them toward the stories you've prepared and hope for the best. Experiencing a poor quality interviewer is perhaps a signal not to work there anyway.

Confidence

[\[34\]](#)

Anxious nonverbal behavior affects how interviewers rate candidates, independent of the actual content being discussed. When you display nervous habits like fidgeting, avoiding eye contact, or speaking in a shaky voice, interviewers perceive you as less competent, even when your stories demonstrate strong capabilities.

Of course, interviews create anxiety so let's talk through some ways to both increase your actual confidence as well as increase the perception of your confidence.

Address impostor syndrome. Many candidates, especially those transitioning between companies or career levels, struggle with feeling like they don't belong in the target company. This manifests as minimizing achievements, attributing success to luck rather than skill, or reluctance to position yourself as an expert. Recognize these self-dismissive thoughts for what they are: cognitive distortions that don't reflect reality.

Combat this by documenting your actual track record. Create evidence-based affirmations grounded in past successes: "I successfully led three cross-functional projects that delivered measurable business impact." Notice recurring themes in positive feedback you've received throughout

your career. Identify consistent strengths that appear across different roles and challenges.

Use permission exercises to unlock confidence. Give yourself explicit permission to own your expertise: “I give myself permission to speak confidently about my technical leadership” or “I deserve to present my professional journey with pride.” These aren’t empty self-help mantras but cognitive tools that help you access the confident professional persona you already embody in your daily work.

[\[35\]](#)

Prepare your body. Physical preparation affects mental state more than most people realize. Exercise regularly leading up to interviews—physical activity reduces stress hormones and improves mood. Eat well-balanced meals and avoid excessive caffeine, which can increase anxiety and jittery behavior. Try power posing before the interview: spend two minutes in a confident stance (hands on hips, chest open, taking up space).

[\[36\]](#)

Project confidence through deliberate nonverbal choices. Maintain steady eye contact, use your hands, and avoid touching your face. If you make a mistake during your response, don’t apologize or draw attention to it. Simply continue with your story. Interviewers often don’t notice small stumbles unless you highlight them.

Respond to clarifying questions calmly. When interviewers ask follow-up questions, don’t interpret this as criticism of your initial response. They’re gathering more information, which is normal and expected. Answer their question directly and return to your narrative.

Maintain momentum even when things feel off-track. If you sense the interview isn’t going well, resist the urge to panic or dramatically change your approach. Stick to your prepared stories and continue delivering them with the same energy and structure. Often, your perception of how things are going differs significantly from the interviewer’s actual assessment.

In most cases, the interviewer wants you to succeed, hoping you’ll demonstrate that you’re the right person for the role. If you approach the conversation with this belief, seeking to collaborate with the interviewer to deliver the signal they’re looking for, you’ll build confidence.

Remote Video Interviews

Video interviews create unique communication dynamics that don't align with traditional in-person interactions. Academic research has some insights for us on how to adjust to remote interviews:

[\[37\]](#)

Compensate for reduced nonverbal communication. When your facial expressions and body language are harder to read, your listeners will rely more on your verbal content. This means your story structure becomes even more critical. Luckily, you own a book with a lot to say about that.

Work harder to establish initial rapport. Building connection virtually requires more intentional effort. Warm greetings become more important—smile genuinely when you first connect, use the interviewer's name, and express enthusiasm about the conversation.

Consider your technical setup. Your camera position, lighting, and audio quality now affect how your content is received. Position your camera at eye level—laptop cameras sit too low, creating an unflattering angle. Ensure lighting comes from in front of you, not behind; a window or lamp facing you provides much better visibility than backlighting from behind you. Test your audio beforehand and consider using earbuds to reduce echo.

[\[38\]](#) [\[39\]](#)

Mind your background. What's visible on camera now communicates information about you. A clean, simple background works best—either a plain wall or a subtle blurred background. Avoid anything distracting, controversial, or overly personal in your frame. If you're interviewing from home, ensure your space looks organized and professional.

Maintain natural eye contact through the camera. Look directly at your camera lens, not at the interviewer's image on your screen. This creates the impression of eye contact for them, though it feels unnatural at first. Practice this beforehand.

Prepare for technical difficulties gracefully. Have a backup plan for connectivity issues and know how to quickly troubleshoot common problems. If technical issues arise during the interview, address them

calmly and professionally—your composure during unexpected challenges becomes part of the assessment.

Minimize Humor

Energy, engagement, and positivity are essential but humor is hard to pull off: it's highly subjective, culturally dependent, and therefore risky to employ humor during your core storytelling time. There is a case to be made for humor during the opening, rapport-building phase of the interview.

✗ *So when our database crashed at 2AM, I guess you could say I had a crash course in incident response!*

You can be memorable and likable with authentic enthusiasm about your past work, genuine curiosity about the company's challenges, and natural warmth in your interactions instead of humor.

[1] See [Appendix: Leveling at Tech Companies](#). ↑

[2] This chapter is aimed at ICs. See [Common Pitfalls for Senior Candidates and Leaders](#). ↑

[3] Have empathy for the interviewer. Conducting a good behavioral is difficult. The interviewer must ask relevant questions, follow complex technical stories, probe for signal, take notes, and create space for you to succeed—all simultaneously. See [Appendix: How to Conduct Behavioral Interviews](#). ↑

[4] *Avoiding Too Much Detail in Behavioral Interviews*. ↑
<https://thebehavioral.tech/p/avoiding-too-much-detail>

[5] Don't over do this. ↑

[6] References to internal systems are probably the biggest offender of the bunch. ↑

[7] They probably got this advice from an interview coach like me. ↑

[8] See [Why CARL is Better than STAR](#). ↑

[9] You highlight the interpersonal communication details during conflict stories as it shows your understanding of people dynamics. See [Tell Me About a Time You Resolved a Conflict](#). ↑

[10] *How fast should you speak in tech interviews?* ↑
<https://thebehavioral.tech/p/speaking-pace>

[11] See [Why CARL is Better than STAR](#). ↑

[12] See [Change the World](#). ↑

[13] The original Vonnegut *Shapes of Stories* talk. ↑
<https://thebehavioral.tech/p/vonnegut-stories>

[14] I got this metaphor from Steve Huynh's video *Don't Get Down-Leveled or How to Tell a Good Story*. ↑
<https://thebehavioral.tech/p/dont-get-downleveled>

[15] *The Menu Technique* ↑
<https://thebehavioral.tech/p/menu-technique>

[16] *The Halo Effect* ↑
<https://thebehavioral.tech/p/the-halo-effect>

[17] *A Constant Error in Psychological Ratings*, Thorndike 1920. ↑
<https://thebehavioral.tech/p/thorndike-halo-effect>

[18] Use the Halo Effect as a New Grad: work done as an intern or projects completed in coursework often aren't very meaty in themselves, but if you string them together in a list, they give the impression of competence. ↑

[19] See [\[→\]](#). ↑

[20] Here you justified your initially poor choice. ↑

[21] Now to show evidence of change. [↑](#)

[22] *Thinking Defensively* [↑](#)
<https://thebehavioral.tech/p/thinking-defensively>

[23] Sometimes you can tell your interviewer is going to be strict. Be attentive to their demeanor during the ice breaker phase, their facial expressions, tone of voice, etc. [↑](#)

[24] See [Handling Sensitive Conversations](#). [↑](#)

[25] *How to handle sticky situations in behavioral interviews* [↑](#)
<https://thebehavioral.tech/p/how-to-handle-sticky-situations>

[26] Remember that everything in life is either a good time or a good story. The way you view your own experience will reflect in how you present yourself to an interviewer. [↑](#)

[27] *Handling Layoffs* from Ethan Evans [↑](#)
<https://thebehavioral.tech/p/ethan-evans-layoffs>

[28] I cite a number of research results throughout this chapter but it's worth noting that none of them target the tech industry specifically. [↑](#)

[29] I present these US-based research results acknowledging that other cultures have other expectations. See Erin Meyer's *The Culture Map*, 2014. [↑](#)

[30] *Interviewer evaluations of interviewee nonverbal behavior*. McGovern et al., 1978 [↑](#)
<https://thebehavioral.tech/p/evaluations-non-verbal>

[31] *Speaking without words: A meta-analysis of over 70 years of research on the power of nonverbal cues in job interviews*. Martín-Raugh et al., 2022 [↑](#)

<https://thebehavioral.tech/p/speaking-without-words>

[32] Science of People [↑](#)

<https://thebehavioral.tech/p/science-of-people>

[33] One of my mentees once complained their Amazon interviewer stopped them to say, “Please stick to the STAR method when responding.” lol [↑](#)

[34] *The Impact of Interviewee’s Anxious Nonverbal Behavioral on Interview Performance Ratings*. Mastrella et al., 2022. [↑](#)

<https://thebehavioral.tech/p/anxious-non-verbal>

[35] *Brain Fog in Interviews*. Dagna Bieda, 2025. [↑](#)

<https://thebehavioral.tech/p/brain-fog-in-interviews>

[36] *The Benefit of Power Posing Before a High-Stakes Social Evaluation*. Cuddy et al., 2012. [↑](#)

<https://thebehavioral.tech/p/power-pose>

[37] *Face-to-face and video-mediated communication: A comparison of dialogue structure and task performance*. Doherty-Sneddon et al., 1997.

[↑](#)

<https://thebehavioral.tech/p/video-conference-dialog>

[38] *Video-call glitches trigger uncanniness and harm consequential life outcomes*. Brucks, et al., 2025. [↑](#)

<https://thebehavioral.tech/p/video-quality>

[39] *Bias in the background? The role of background information in asynchronous video interviews*. Roulin et al., 2024. [↑](#)

<https://thebehavioral.tech/p/video-background-research>

PART 4

ADAPTING TO CONTEXTS

The core principles of behavioral interviewing remain constant, but their application shifts with context.

BEHAVIORALS THROUGHOUT YOUR CAREER

The fundamentals of behavioral interviewing remain constant but how you apply these principles evolves based on where you are in your career and what type of organization you're targeting.

New Graduates

New grads face a challenging mismatch: behavioral interviews assess past behaviors to predict future performance, but new grads have limited behaviors from which to draw. The key is to authentically represent what you do have—mostly internships, side projects, and coursework—while using *professional language* and focusing on *repeatable behaviors* that predict success in a work environment.

[\[1\]](#)

Story selection for new grads

Here's a prioritization framework for you to use when choosing stories as a new grad:

1. **Industry-specific work experience first** (internships, co-ops, part-time jobs), even if brief
2. **School projects**, especially longer ones
3. **Personal projects** that demonstrate initiative and technical skills
4. **Leadership roles** in clubs, organizations, volunteer work
5. **Non-industry work experience** for questions related to interpersonal dynamics, like conflicts

Compensating for limited scope

A healthy hiring process assesses new grads for potential instead of skill. In light of that, interviewers will focus on your approach and thinking process more than outcomes. Show them you understand what professional excellence looks like, even if you haven't achieved it at scale yet, focusing on these signal areas:

- **Growth:** you're constantly learning and improving

- **Ownership:** you take initiative on projects, from ideation to deployment, which is common for school or personal projects
- **Perseverance:** you work through difficult technical challenges
- **Communication:** you can work with teams and explain your work

Also remember the strategic use of the Halo Effect, listing multiple smaller accomplishments, to create an impression of breadth.

[\[2\]](#)

✓ *I've worked on various projects including a machine learning classifier, a full-stack web app, contributing to an open-source library, and leading my university's hackathon team.*

Big Tech vs. Startups vs. Traditional Company

Behaviorals are more similar across companies than different. A solid foundation in Parts 1 and 2 will take you pretty far, and if executed well, would likely result in a hire decision at most of these companies, assuming you have sufficient scope for the target role.

That said, understanding what different types of organizations prioritize can help you tailor your preparation and reduce anxiety about the unknown.

FAANG+ and Big Tech Companies

Large tech companies tend to have the most structured behavioral interview processes. They've conducted thousands of interviews, refined their signal areas, and trained interviewers on consistent evaluation frameworks. Here's what to expect:

- Multiple behavioral rounds, often 1–2 sessions during an onsite loop
- Structured questions targeting specific signal areas or company values
- Trained interviewers following consistent frameworks
- Written feedback that gets reviewed by hiring committees
- Clear leveling expectations that interviewers assess against

The weight placed on behaviorals varies significantly between companies. Amazon structures their entire interview process around their 16 Leadership Principles, dedicating substantial time to behavioral assessment. Google places less emphasis on behaviorals relative to technical evaluation.

Meta is somewhere in between. Of course, all companies value behavioral rounds significantly for senior roles.

Research the company's stated values—they're typically published on career pages or in job descriptions. At Amazon especially, knowing their Leadership Principles and using their terminology naturally in responses matters. For most other large tech companies, understanding the core signal areas covered in Part 1 matters more than memorizing company-specific language.

Be sure to read [*Big Tech Behaviorals*](#) for a more detailed discussion.

Startups and small companies

Smaller organizations present more variability in their interview processes. You are much more at the whim of the founders' hiring philosophy or hiring manager's preferences and skill level. Here's what to expect:

- Less structured, both across the whole process and in the interviews themselves
- More varied interviewer experience, where you could talk with a seasoned hiring manager or a founder conducting their first interview
- Greater emphasis on immediate role fit since they're often hiring for specific skill gaps rather than general capability
- More scrutiny about your ability to move fast and adapt to changing market, technology, and resource conditions
- A need to sense genuine excitement in the candidate about the product space and company
- More weight on personal connection as you'll likely work closely with your interviewers
- Direct exposure to founders or executives earlier in the process

Research who you'll speak with specifically. LinkedIn profiles reveal whether your interviewer previously worked at structured companies (suggesting they might follow more formal frameworks) or is new to hiring (suggesting a more conversational approach).

Understand the company's immediate challenges and technical stack. Smaller companies often care deeply about whether you can contribute to their specific problems using their specific technologies.

Prepare for broader conversations. You might discuss everything from technical architecture to company culture to your career motivations in a single session. Have your core stories ready but stay flexible about the direction the conversation takes.

Emphasize rapid contribution. Startups value people who can operate independently and make an impact quickly. Stories demonstrating your ability to figure things out with limited guidance, ship quickly, and wear multiple hats resonate strongly.

Traditional Enterprises and Non-Tech Companies

Financial services, healthcare, manufacturing, and other traditional industries increasingly hire software engineers and technical talent, but their interview processes often differ from pure tech companies. Here's what to expect:

- Varied behavioral interview maturity as some have adopted tech company practices while others use more traditional approaches
- Greater emphasis on stability and process
- A focus on domain knowledge
- Different cultural values, where hierarchy may be valued over the “Embrace Conflict” mentality of Silicon Valley

BIG TECH BEHAVIORALS

Even though behavioral interviews are more similar than they are different across company types, it is common for candidates from smaller companies or traditional enterprises to struggle translating their experience to what a Big Tech interviewer expects. They are often technically skilled enough, but moving to FAANG+ presents two challenges: *scale* and *culture*.

[3]

If you're experienced at a FAANG+ company or Silicon Valley startup, you might already have intuition about how to adapt your stories. But if you're coming from a different industry, geographic region, or company culture, this section will help you decode the unwritten expectations and recode your responses.

The Shift in Scale

"Because of the scale," is the top reason cited whenever I ask a candidate why they want to join FAANG+. It's understandable: that's where the impact, skill growth, and career growth are the largest.

The scale differences at Big Tech manifest themselves throughout:

Users and systems. Billions of users is different than tens of thousands. The complexity of the codebase, the number of interlocking systems, is extreme.

Organizational. Serving millions or billions requires a lot of employees. Decisions have implications that propagate through tens or hundreds of thousands of employees. Projects require coordinating across dozens of teams spanning multiple functions.

Pace. Big Tech can afford to pay for the best employees and they expect them to deliver fast. Yes, they are hindered by their organizational scale at times, but each individual is expected to handle multiple projects at once with aggressive timetables.

Measurement and impact. Because they can, Big Tech companies measure everything: success is expressed through metrics and decisions are expected to be rigorously data-driven. This overflows into career planning and performance management, with each person's contributions quantified and compared.

The Shift in Culture: The Silicon Valley Mythos

The images of Myth are reflections of Spiritual and Depth potentialities of every one of us. Through contemplating those we evoke those powers in our own lives to operate through ourselves.

Joseph Campbell

These things never happened, but always are.

Sallustius

But the practicalities of scale alone don't explain why candidates struggle. Even when they have the right technical and organizational experience, their stories often don't resonate. This disconnect stems from how FAANG+ culture prioritizes certain behaviors, attitudes, language, and ways of working that differ from traditional enterprises or smaller companies. This shift represents another layer to how tech companies assess you as a candidate in addition to the structured signal areas approach: the intuitive cultural patterns that determine whether you "feel right" for their organization.

To some biased or poorly trained interviewers, this "culture fit" simply means whether you went to the right schools, look a certain way, or whether they want to have a beer with you. Those are unhelpful in establishing successful hiring practices, but beyond these lie something deeper: the unwritten expectations about what success looks like in tech culture. How do you understand what makes a story sound good? How do you know what language to use and whether your experience will resonate in that FAANG+ interview you have coming up?

[4]

These shifts are subtle and interconnected, more felt than enumerated. Rather than giving you a disconnected list of advice to adapt your responses, I'll show you the underlying *mythology* that shapes Big Tech culture. Understanding these archetypal stories lets you present your experiences in ways that naturally resonate with interviewers' expectations.

These stories, let's call them *myths* since they aren't literally true but psychologically true, are deeply entrenched, but you won't find them in employee handbooks. They're more powerful than company values statements; they're the raw materials from which those values were derived.

[5]

The hero of these myths is what I call the *Builder*—someone who creates technology solutions, drives impact, and embodies the values these myths celebrate.

Let's examine the core myths that define tech culture and how the Builder frames their experience to resonate in a Big Tech behavioral.

Out of the Garage

On Addison Avenue in Palo Alto, California, sits a small 12×18-foot garage tucked behind a Craftsman-style house. In 1938, two Stanford graduates, Bill Hewlett and Dave Packard, collected \$538 and a dream to build their first technology product. They started not only their own company but Silicon Valley itself.



Out of the Garage. The HP Garage on Addison Ave in Palo Alto, California.

Larry Page and Sergey Brin rented Susan Wojcicki's Menlo Park garage sixty years later as Google's first office. Jeff Bezos even looked for a house with a garage in Seattle so he could say he started Amazon "out of a garage."

The tradition lives on in today's "war rooms"—cramped conference rooms where small teams hunker down to ship critical features, decorated

with whiteboards covered in diagrams and powered by determination rather than budget.

Builders start with what they have rather than waiting for what they need. Scrappy resourcefulness trumps well-funded comfort every time. Constraints spark creativity. You begin small, prove the concept works, then scale from there.

The Builder is an owner, an entrepreneur. Even in a more traditional top-down business, they find a way to make progress quickly.

<i>Standard Framing</i>	<i>Builder Framing</i>
<i>We evaluated vendor solutions and selected the enterprise platform recommended by our procurement process, due to SOC2 requirements.</i>	<i>I decided that going with a vendor would allow us to get to market faster in exchange for some limitations on user experience. It also has the benefit of getting a headstart on SOC2 compliance for us.</i>
<i>I submitted the request for additional server capacity from our infrastructure team to handle the increased load.</i>	<i>I convinced our infrastructure team that the additional server investment would pay for itself through improved customer retention.</i>

The Lone Hacker



The Lone Hacker. Somehow, they always have a hoodie and headphones.

Linus Torvalds created Linux while studying at the University of Helsinki. Susan Kare drew pixel art for the first Mac on graph paper she bought at a Palo Alto art store. John Carmack spent week-long programming retreats revolutionizing 3D graphics at id Software. Mark Zuckerberg built Facebook from his Harvard dorm room. Recalling the lonesome gunslinger roaming the American West, they set the world straight with nothing but personal conviction and skill.

This Lone Hacker archetype persists in the trope of the 10× engineer—a builder who produces ten times the output of their peers through a combination of technical brilliance and relentless focus. Or the college dropout who leaves the traditional life path for immortality as a tech CEO.

Personal effort and expertise are highly valued, even in team environments. Builders take chances, being right in the face of opposition, and pursue ideas others dismiss as impossible.

Builders cut through red tape and don't use "business speak" but deliver clear and direct communication.

Standard Framing

My manager assigned me the feature...

The team identified performance issues during our weekly review.

We leveraged cross-functional synergies to ideate solutions for the scalability challenge.

Builder Framing

→ I took on this feature...

→ I noticed unusual patterns in our monitoring data that the team later confirmed were performance bottlenecks.

→ I worked with the platform team to fix our performance bottlenecks.

Move Fast and Break Things

"Real artists ship," Steve Jobs declared to his Mac team in the early 1980s, pushing them to launch with incomplete features and software that barely worked. Getting the product into users' hands mattered more than achieving perfection.

Mark Zuckerberg made this philosophy literal, adopting "Move Fast and Break Things" as a semi-official motto and plastering it on the walls of Facebook. They shipped features that sometimes crashed the site, prioritizing rapid iteration and user feedback over bulletproof code. Google embraced the same spirit with their "perpetual beta" approach—Gmail stayed in beta for five years while they continuously shipped improvements, treating millions of users as willing co-developers.



Move Fast and Break Things. Always be focused on shipping.

The Builder has a bias for action over analysis. The market moves too fast to wait for perfect solutions. Taking initiative means making decisions with incomplete information, cutting corners strategically, and learning from failures in production rather than in planning meetings. In Silicon Valley, the greatest risk isn't shipping something imperfect—it's letting a competitor ship first while you're still polishing.

Builders embrace rapid change, ready to pivot their strategy to match market conditions or learning new skills with energy. Others resist change and that creates opportunities for the Builder.

Standard Framing

We followed our standard security review process to ensure compliance with company policies.

Builder Framing

I worked through our security requirements early to avoid any surprises that could delay our launch.

We postponed the launch by two weeks to address all the remaining edge cases and polish the user interface.

We thought about the impact of delaying vs shipping as is—but the potential for brand damage was too great.

Fail Fast

YouTube started as a dating site where users uploaded videos of themselves. Pinterest began as Tote, a mobile shopping app nobody wanted. Slack emerged from the wreckage of a failed gaming company called Tiny Speck. Instagram was originally Burbn, a cluttered location check-in app until the founders stripped away everything except photo sharing.

These were the results of recognizing that the original vision wasn't working, followed by quick action to try something else. The founders didn't spend years trying to make their first ideas succeed. They watched user behavior, acknowledged failure quickly, and moved to what was actually gaining traction.

This philosophy extends beyond product development to how teams handle mistakes and outages. John Allspaw brought the concept of blameless postmortems from the aviation industry into tech culture, first at Flickr and later at Etsy. Blaming pilots for issues made flying less safe—it discouraged reporting near-misses and system problems. Allspaw argued the same principle applied to software: if you blame engineers for outages, they'll hide problems instead of surfacing them.

Fail Fast. The Twitter Fail Whale, a cultural institution of the early 2010s, was a whimsical character shown to users when Twitter was down.



Builders acknowledge what's not working without fear of punishment, surface problems early, and pivot quickly to solutions that work.

Standard Framing

Due to changing market conditions, we had to adjust our timeline and deliverables.

Builder Framing

I recognized early that our original approach wasn't going to work, so I killed the project and reallocated resources to a more promising solution.

Embrace Conflict

Silicon Valley's formative years coincided with the West Coast's countercultural movement of the 1960s. The challenging of traditional power structures and anti-hierarchical approaches to management became a competitive advantage.

The cost of avoiding conflict became clear through one of Silicon Valley's earliest cautionary tales. William Shockley invented the transistor, but his paranoid management style drove away his eight most talented employees. Rather than endure a dysfunctional hierarchy, they left to establish Fairchild Semiconductor, embracing the derisive name Shockley branded them with—the Traitorous Eight—and built what became the most

commercially successful transistor company, launching the modern semiconductor industry in the process.

[6]

Later, one of the Eight and founder of Intel, Andy Grove institutionalized this philosophy through “constructive confrontation,” encouraging engineers to challenge each other’s ideas aggressively. He believed the best technical solutions emerged from intellectual combat, not polite consensus.

Venture capital amplified this culture of productive conflict. VCs brought rigorous questioning and challenging of founders’ assumptions. This “tough love” approach became normalized throughout startup culture, where being wrong was less dangerous than being unchallenged.

Standard Framing

There was disagreement, so we escalated to management for resolution.

There was competition for engineering resources, so we worked through the proper channels to get our project prioritized.

Builder Framing

When I disagreed with the proposed approach, I scheduled a meeting to → present my alternative solution with supporting data.

When I couldn’t get the engineering resources I needed, I made the → business case directly to leadership with data on customer impact.



Embrace Conflict. Builders turn conflicts into productive ways to move the team's goals forward.

The Builder initiates difficult conversations about disagreements. They bring data, prototypes, and proof-of-concepts to settle disputes. They believe good ideas can emerge from anywhere in the organization, regardless of hierarchy, and that intellectual friction produces better solutions than artificial harmony

Change the World

[\[7\]](#)

Steve Jobs wanted to “put a dent in the universe” and positioned his personal computer as liberating humanity from conformity; the Macintosh as the tool of revolution. Bill Gates proclaimed Microsoft’s mission as putting “a computer on every desk and in every home” when most people had never seen a computer and the idea seemed laughably grandiose. Google started with Larry Page and Sergey Brin’s vision to organize the world’s information and make it universally accessible and useful—essentially democratizing all human knowledge. Genentech, the world’s first biotech company, sought to transform healthcare through genetic engineering. OpenAI was founded on the mission to ensure artificial general intelligence benefits all humanity, not just the wealthy or powerful. Even Facebook, which started as Mark Zuckerberg’s crass college dating site, quickly pivoted to the grandiose vision of connecting every person on the globe. Elon Musk wasn’t content with changing just this world; he set out to make humans a multi-planetary species while simultaneously revolutionizing transportation and energy.



Change the World. The 1984 Macintosh Super Bowl commercial promising an end to the tyranny of sameness.

The Builder tackles fundamental challenges facing humanity. They frame their work in terms of at least business-wide impact if not global impact and societal transformation, positioning technology as a force for human progress. They are an optimist.

Standard Framing

The new feature improved our customer satisfaction scores and met the business requirements.

The automation project eliminated manual tasks and increased team efficiency.

Builder Framing

I built this feature to help small business owners save hours of manual work each week, freeing them to focus on growing their companies.

I automated these workflows to eliminate the repetitive work that was burning out our engineers, so they could focus on building features that directly impact users.

Being the Builder: Adapting your Responses to Big Tech

You don't need experience with all this scale or culture to be successful with it—all their employees were new to Big Tech at one point. But you will need to change some of the language and framing you use during a behavioral.

Emphasize ownership. Talk about how you went beyond assigned tasks, caring about and taking responsibility for results, even when that means working outside your formal role.

Use data-driven language. Instead of saying *I improved our deployment process*, try *I reduced deployment time from 2 hours to 15 minutes, which would have saved our 20-person engineering team about 40 hours per month*. This shows you think about gains in measurable terms but also in ways that extend beyond yourself.

Uplevel impact conversations. Your decisions likely had broader impact than you realize. That database optimization you implemented didn't just speed up queries, it improved user experience for your entire customer base and reduced infrastructure costs. Frame decisions in terms of their systemic impact, showing you think beyond just your immediate feature.

Use terminology that signals familiarity with Big Tech practices. Rather than, *We tried out a new feature*, describe *A/B testing the new user experience*.

Show familiarity with modern patterns. Even if your previous company used different approaches, you can still demonstrate forward-thinking. *While we used a monolithic architecture, I designed my features with clear service boundaries, which would make future extraction into microservices straightforward*.

Express systems thinking. Even when working on smaller projects, show you consider broader implications: *While implementing this feature, I considered how it would scale if our user base grew 10× and designed the data model accordingly*.

Emphasize working with other people. Highlight any experience you have working across organizational boundaries, even if those “teams” were just different roles at your smaller company. *I worked with our designer and product manager to align on requirements* becomes *I coordinated across*

design, product, and engineering to ensure alignment on user experience and technical feasibility.

Describe direct communication. It's expected that you raise disagreements directly, use data to resolve disputes, and view productive conflict as leading to better outcomes. Avoiding difficult conversations or deferring to hierarchy signals weakness.

Emphasize initiative. The story about building a feature with limited guidance becomes evidence of handling ambiguity and taking ownership. Rather than *My manager assigned me to improve the checkout flow*, try *Checkout abandonment rate was high so I worked on a redesign that reduced friction*.

Demonstrate comfort with rapid iteration. If your previous company had slower release cycles, emphasize experience with rapid iteration through side projects, hackathons, or pilot programs. Show that you're energized by fast-paced environments that prioritized shipping rather than overwhelmed by them.

Closing thoughts and pitfalls

If you've spent a few years building technology customers actually use, your stories are probably enough for Big Tech: managers are looking to understand how you operate across the software development life cycle, which you've done, so simply tell them about all those parts.

That being said, be sure you avoid these patterns:

- **Disclaimers:** "I know this wasn't as big as what you do here, but..."
- **Defensive language:** "We didn't have the tools that big companies have"
- **Downplaying your value:** "We only served a few thousand users..."
- **Excessive process:** Stories emphasizing bureaucracy you navigated rather than obstacles you overcame
- **Limited ownership:** "That's not my job" attitudes in your narratives

Good luck and have fun!

SPECIALIZED INTERVIEW TYPES

Not every conversation during your hiring process will be labeled a “behavioral interview,” but the same principles apply—decode what signal they’re seeking, select relevant stories, and deliver clear narratives. A call with a recruiter, a deep dive with the hiring manager, or a casual team-matching conversation all benefit from the frameworks in this book. Leadership roles introduce additional considerations and specialized formats that require adapted approaches. This chapter covers how to adjust your preparation and delivery for these variations.

Calls with Recruiters

[8]

Screening calls with recruiters—those early conversations that determine whether you advance to the onsite rounds—require a different approach than behavioral interviews with hiring managers. The recruiter’s job is to assess basic qualifications and cultural fit, not to dive deep into your capabilities or leadership philosophy.

Landing a strong TMAY

Your “Tell Me About Yourself” response becomes even more critical in recruiter screens since it may be your only chance to make a compelling case for moving forward. The goal is straightforward: secure the next round.

[9]

Focus on connecting your past experience directly to elements of the job description. If the role emphasizes “building scalable systems for millions of users,” highlight projects where you worked at that scale. If they’re looking for “cross-functional collaboration,” mention your partnerships with product managers and designers.

Consider leveraging the Halo Effect by efficiently listing accomplishments or past roles that demonstrate breadth of experience. Instead of deep-diving into any single project, provide a concise summary:

[10]

✓ *I’ve worked on everything from payment processing systems to machine learning infrastructure, led teams ranging from 3 to 15 engineers, and*

collaborated across product, design, and data science.

Be ready for subject matter screening

For some roles, recruiters will ask basic questions to verify you have the foundational skills for the role. They may be multiple choice questions or have you talk about past projects. If you're not sure what the content will be, I recommend preparing concise descriptions of key projects that demonstrate competence without over-explaining. The recruiter likely isn't experienced enough to evaluate the nuances of your response, but they can assess whether you sound credible and knowledgeable.

Describe past work in company-relevant terms

Research what you can about the company and role beforehand, then frame your experience in those terms. If you're applying to a fintech company, emphasize your experience with financial systems, regulatory compliance, or security. For a startup, highlight your comfort with ambiguity and rapid iteration.

Gather intelligence on subsequent rounds

Use the recruiter conversation strategically to understand what's coming next. Ask questions like:

- What does the interview process look like after this call?
- Are there particular subject matter areas the interviewers will focus on?
- What signal areas are most important for this role's behavioral interviews?
- What are the main reasons candidates fail the onsite rounds?
- What is the reason candidates get down-leveled?

Recruiters often have templates or talking points about the interview process and can give you valuable insight into what the hiring team prioritizes.

Prepare reasonable questions

Ask thoughtful questions that the recruiter can actually answer. They're probably not deeply embedded in the team and won't know specifics about the tech stack, business strategy, or day-to-day challenges.

Good recruiter questions focus on:

- **Process:** How many rounds of interviews should I expect?
- **Team structure:** How large is the engineering organization?
- **Company trajectory:** What's driving the growth in this team?
- **Role context:** How does this position fit into the broader product roadmap?

Avoid questions better suited for the hiring manager, like “What’s your approach to technical debt,” or “How does the team handle on-call rotations?”

The recruiter screen is about demonstrating basic qualifications and enthusiasm while gathering intelligence for the real interviews ahead. Keep it conversational, stay focused on the fundamentals, and use the opportunity to set yourself up for success in subsequent rounds.

Screening Interviews

Sometimes the first call with a hiring manager or member of the team is a behavioral screening interview. This format is common at smaller companies where cultural fit is paramount and candidate volume is manageable, or for leadership positions where behavioral signal is the most critical factor in the hiring decision.

The primary goal of these screens is rapid assessment across three key questions:

- Do you seem like the right fit for this specific role?
- Should we place you in the standard interview loop or some specialized track?
- Are you operating at the appropriate level for the target position?

Expect breadth over depth: Unlike deep behavioral rounds that explore topics thoroughly with follow-up questions, screening interviews typically involve many shallow questions designed to assess breadth of experience. The interviewer wants to quickly sample your capabilities across different signal areas rather than dive deep into any single competency.

Hone your TMAY for appropriate scope: Your “Tell Me About Yourself” becomes even more crucial since it sets the tone for the entire conversation.

Focus on hitting clear notes about scope that align with the target role. Avoid the temptation to undersell yourself out of modesty—the screening interview is specifically designed to assess whether you’re operating at the right level.

Maximize signal per minute: Given the breadth-focused format, every response needs to efficiently demonstrate competence. Use shorter, more focused stories that quickly establish your capabilities without excessive detail.

[\[11\]](#)

Connect as a person: If the screening is conducted by the hiring manager rather than a recruiter, assume cultural fit assessment is a significant component. This person is evaluating whether they want to work closely with you, not just whether you have the right qualifications.

[\[12\]](#)

Prepare thoughtful questions: Even more so than in a general behavioral interview, the questions you ask them reflect on your fitness for the role. Show genuine interest in their challenges, ask thoughtful questions about, the product, the team, etc.

Leadership Interviews

Leadership interviews—whether for senior individual contributors (Principal+) or management roles—require a more extensive approach relative to standard behavioral rounds. The stakes are higher, with more of the overall hiring decision based on behavioral signals, while the structure and formats vary more widely between companies, roles, and among interviewers themselves.

You might encounter project deep dives that explore a single initiative for the entire session, rapid-fire questions about different leadership scenarios, or philosophical discussions about your management style and organizational beliefs. Some interviews focus heavily on people management—hiring, performance conversations, team building—while others emphasize cross-functional relationship management or technical leadership across multiple teams.

All the principles you’ve learned so far continue to apply to leadership interviews, but here are some specific thoughts.

Additional signal areas for leadership roles

Beyond the core behavioral competencies, leadership interviews assess several additional dimensions:

[\[13\]](#)

Driving impact on projects: Moving beyond individual execution to orchestrating outcomes across multiple people and workstreams. This includes setting vision, aligning stakeholders, removing blockers, setting and tracking measurable goals, and ensuring follow-through.

People management: For management roles, direct assessment of hiring, coaching, performance management, and team organization skills. Even for senior IC roles, mentoring and influence capabilities are crucial.

[\[14\]](#)

Cross-functional relationships: Your ability to work effectively with engineers, product managers, designers, data scientists, marketing, sales, and other functions. Senior roles require fluency across disciplines and the ability to translate between different organizational languages.

Know your audience

Leadership interview dynamics vary significantly based on your interviewer's background and priorities. Pay attention to what they're probing for and adjust your emphasis accordingly:

- A **VP of Design** might care most about business impact, organizational alignment, and how you think about tradeoffs between quality and velocity.
- A **Product Manager** will likely focus on your experience working with non-technical partners, how you balance engineering constraints with product vision, and your approach to technical communication.
- A **Principal Engineer** might dive deep into technical decision-making frameworks, how you drive architectural decisions across teams, and your approach to mentoring other engineers.

Also know that leadership interviewers tend to be more risk averse, since hiring leaders is inherently more risky to the organization. See [Thinking Defensively](#).

Observe and respond to your interviewer

Leadership roles require emotional intelligence and adaptability, qualities your interviewer will assess through how you conduct yourself in the interview itself. Watch for engagement signals and adjust accordingly.

If they're frequently asking for elaboration on details you're glossing over, start providing more context upfront in subsequent stories. If they haven't taken notes or asked questions in a while, you might be spending too much time on background and should focus more on specific actions you took.

If they seem energized by business impact discussions but less engaged with technical details, emphasize the strategic and organizational aspects of your work.

The interview itself becomes a demonstration of your ability to read an audience and adapt your communication style—a core leadership competency.

Prepare for interruptions and deep dives

Leadership behavioral interviews tend to be more conversational but the interviewers also tend to be more impatient than in standard rounds. Experienced interviewers will interrupt to dive deep into specific decisions, explore alternatives you considered, or understand particular aspects of team dynamics that caught their attention.

To handle this, prepare **modular story components** that you can expand or condense based on their interests. If they want to explore your conflict resolution approach in detail, be ready to elaborate on the interpersonal dynamics and specific conversation techniques you used. If they're more interested in business impact, have the metrics and outcomes readily available.

[\[15\]](#)

After answering a detailed follow-up question, gently guide the conversation back to your main narrative: *That decision ended up being crucial for the project's success. The next major challenge we faced was...* This ensures you maintain control over covering all the key points that demonstrate your capabilities.

Prepare for Values questions

You are more likely to encounter Values questions during leadership interviews than during IC interviews. Review the concepts of selecting responses to values questions [→] and delivering them [→].

Common pitfalls for leadership interviews

See the [*Common Pitfalls for Senior Candidates and Leaders*](#).

Follow-Up Interviews

[16]

Sometimes in life you get a mulligan, a break, a do-over. Follow-up interviews are like that. I got one after I flubbed my first coding screen when applying to Facebook.

While that experience was certainly about my own performance, tech companies often conduct follow-up interviews when something went amiss the first time—and it's not always about you. The decision to bring you back reflects a complex interplay between company processes, interviewer ability, and candidate performance.

Understanding why you're offered a follow-up

Let's explore some reasons companies might call you back for a follow-up behavioral interview, categorized by whose "fault" it might be: the company's, yours, or perhaps nobody's.

Inexperienced interviewer (company's fault): Most tech companies have rigorous review processes where written interview feedback is evaluated by experienced committees of engineering leaders. If they determine that an interview result doesn't align with their expectations—based on your background or the other interview results—and the interviewer lacks experience, the committee may request a follow-up with a more seasoned interviewer.

Inadequate signal (company's fault): Even when the interviewer is experienced, they might not have gathered enough information or the right type of information. Hiring committees frequently have specific questions they need answered like, "Does this senior-level candidate have experience leading others?" or "Conflict resolution wasn't addressed here; we need to understand the level of skill this candidate possesses in this area." These gaps in understanding prompt a follow-up to complete your packet.

Poor story choice (your fault): The hiring committee may notice that while your resume features relevant projects that would showcase your qualifications, you chose to discuss older or less significant work. In these cases, they might offer another opportunity to tell your stories.

Poor performance (your fault): Sometimes committees recognize that interviews can be stressful and are willing to give you another chance—a true mulligan—because they understand you might have been nervous or perhaps it was your first interview of the day and you hadn’t fully warmed up. This especially happens when other aspects of your candidacy are strong.

Logistical issues (nobody’s fault): Technical difficulties like network connectivity or scheduling issues like you or the interviewer being late compromise the quality of the interview. Companies often opt for a follow-up rather than making decisions based on an incomplete assessment.

[\[17\]](#)

Preparing for a follow-up

Ideally the recruiter tells you why the follow-up is being conducted and you tailor your responses appropriately.

If you think your stories are in line with the target level of the position, then I would tell the same ones. Focus on telling them with more emphasis on your own actions and connecting them to the company’s values.

There’s an important exception to this approach: if you have multiple stories that might address the same question effectively. In this scenario, I suggest offering both options to the interviewer (see [The Menu Technique: Letting Your Interviewer Choose](#)):

✓ *In the previous interview, I responded to that question with a story about a large backend refactor. Would you prefer I elaborate on that example, which might be the best fit, or should I share another story about improving the performance of a large-scale production system?”*

If you’re not sure your stories were in line with the target level, then be sure you’re picking ones that represent your best work at the target scope. Get a mock interview from an experienced coach to help calibrate you.

Be prepared for targeted questions

If the interviewer has been tasked with acquiring specific signal, they may skip over context setting questions like, “Tell me about your favorite project,” and jump directly into a question like, “Tell me about a time when you had to persevere through a difficulty?” This is where having a well-curated set of stories aligned with behavioral interview signal areas and company values comes in handy, enabling you to easily choose and tell the story that fits the best.

A follow-up interview is never a bad sign—it’s a sign that the company is still interested. Spend additional time preparing your responses and go in with confidence the second time.

Deep Dives or Project Retrospectives

Some interviews, especially for leadership candidates, take the form of deep dives—extended conversations that focus intensively on a single project or initiative from your career. Senior roles require complex judgment across multiple dimensions, and the best way to assess that judgment is through detailed exploration of how you navigated a substantial challenge.

The structure of these interviews is more uncertain than traditional behavioral rounds. It could be a free-flowing conversation about one project of your choice, primarily driven by where you take the conversation, or it could involve many targeted questions all focused around the same project. The interviewer might let you present your project narrative uninterrupted for 10–15 minutes, then spend the remaining time drilling into specific decisions, tradeoffs, and outcomes. Alternatively, they might interrupt frequently with clarifying questions, turning your story into a guided dialogue.

This uncertainty makes preparation more challenging but also more important. Unlike standard behavioral interviews where you can pivot between different stories based on the questions asked, deep dive interviews may require you to have one project that can demonstrate the full range of your capabilities. Story organization and knowing exactly what you want to communicate about the project become critical, since you’ll need to adapt your narrative based on the interviewer’s interests while ensuring you hit all the key points that showcase your leadership.

Choosing a project

Generally, you should apply the same framework you did in [*Tell Me About Your Favorite Project*](#)—the intersection of impact, scope, and involvement. Additionally, choose projects with leadership complexity, like initiatives that involved multiple teams, ambiguous requirements, and significant risk. Look for projects that demonstrate the full leadership spectrum:

- **Technical leadership:** architecture decisions, technology choices, technical debt management
- **People leadership:** mentoring, conflict resolution, team building, hiring and performance management
- **Process and operational leadership:** establishing best practices, incident response, capacity planning
- **Strategic, product, and business leadership:** roadmap planning, product ideation and refinement, stakeholder management, resource allocation

Organizing your conversation

Senior-level projects are inherently complex, often spanning months or years with multiple workstreams and stakeholders. Because the stories are longer, traditional CARL formatting breaks down when trying to capture this complexity. Here are some approaches to address this:

Use the *Table of Contents* approach: Identify the most relevant themes that demonstrate your impact and begin by listing them. Use that as the structure for the rest of your conversation.

[\[18\]](#)

Front-load the impact: Don't bury your results at the end. Senior interviewers may interrupt with detailed follow-ups, and you might never reach your conclusion. Include the results in your initial response, with the business motivation, then explain how you achieved it.

Add details that demonstrate scope: Every detail should serve a purpose—to show technical complexity, organizational challenge, or strategic thinking. Mentioning “coordinating across 12 engineers in 4 time zones” signals scope. Describing “migrating 500M+ daily transactions with zero downtime” demonstrates both technical and business impact.

Leave out details that result in redundant takeaways: The natural follow-up to adding those kinds of details is leaving out ones that have the same takeaways you've established. If you're discussing the third challenging technical situation on this project in gory detail, you are spending time harping on the same message—"I am technical"—when you could be using that time to establish other aspects of your career.

Cross-Functional or Partner Interviews

Some leadership or senior IC interviews include dedicated cross-functional (XFN) rounds, where you're interviewed by someone from a different discipline—perhaps a product manager interviewing an engineer or a researcher evaluating a designer. Technology is a team sport.

The presence of an XFN interview signals that the role requires substantial communication, alignment, and partnership across different functions. These interviews assess whether you can:

- Translate between different organizational languages and priorities
- Build productive relationships with people who think differently than you
- Navigate competing objectives while maintaining trust
- Advocate effectively for your perspective while respecting others' constraints

XFN Question Examples

On balance, XFN interviews aren't dramatically different from standard behavioral rounds, but they'll focus on the signal areas most relevant to partnerships: Communication, Conflict Resolution, and Leadership.

Relationship-focused questions:

- Tell me about your best partnership with someone from [other function]
- Describe your worst cross-functional collaboration and what you learned from it
- How do you build trust with partners who have different priorities than you?

Communication-focused questions:

- Explain something technical to me as if I don't have a technical background
- Tell me about a time when you were misunderstood by a cross-functional partner
- Describe a situation where you had to translate between different teams with different vocabularies

Conflict and influence questions:

- Tell me about a time when you got something from someone who didn't want to give it to you
- Describe a disagreement you had with a [other function] and how you resolved it
- Give me an example of when you had to advocate for your function's needs while respecting another team's constraints

Philosophy and approach questions:

- What is your philosophy when working with partner teams?
- How do you handle situations where your function's goals conflict with another team's priorities?
- What makes a great cross-functional relationship?

Thoughts on XFN Interviews

Understand the interviewer. This is one interview where it's common for you to ask questions in the early part of the interview. See if you can identify how their function works with yours and what they value.

Prepare thoughtful questions. When the interviewer creates space for you to ask questions, be sure you have some insightful ones prepared to show your eagerness and competence when working with partners.

Beware of “fairytale endings.” Real cross-functional work involves genuine tradeoffs. Don't sanitize your stories to the point where they lose authenticity. It's fine to acknowledge that sometimes one function had to compromise.

Avoid jargon. This is good advice anyway, but be especially aware of technical or role-specific terms in this interview. How well you communicate is being evaluated throughout.

Avoid portraying XFN partners as adversaries. Even when discussing conflicts, frame them as differences in perspective or priorities, not personality clashes or competence issues. The interviewer is evaluating whether you'll be pleasant to work with.

Use it as an opportunity to learn about the company. The interviewer is likely outside your reporting chain and perhaps more candid about the company and the team.

Presentations

Presentations—where you lead a long discussion through some topic, often a project from your past—are an inconsistent element of tech interviews. They most commonly appear in Product Manager or Design interviews, sales-related roles for Software Engineering, Data Science or Research positions, and some leadership interviews. Academia-to-industry candidates may also be required to give presentations on their past research.

While providing a comprehensive guide to giving high-quality presentations is beyond the scope of this book, the core principles of behavioral interviewing still apply: understand the signal needed in the interview, choose the right material to cover, and focus on delivering that efficiently.

Apply the signal areas framework

Treat your presentation like an extended behavioral response. The interviewer is still evaluating you across similar signal areas—Scope, Ownership, Communication, Leadership, etc.—just in a different format. Before preparing your presentation, clarify with your recruiter or hiring manager what competencies they will assess. Are they looking for technical depth, stakeholder management skills, or your ability to drive complex initiatives?

Beware of over-reliance on visuals

Many candidates fall into the trap of creating slide-heavy presentations that become a scripted monologue. This approach fails in interviews because:

- Interviewers will interrupt with questions, disrupting your flow
- [\[19\]](#)Dense slides shift focus from your actions to your design skills
- You lose the conversational dynamic that builds rapport

Instead, prepare minimal visuals that support your narrative. Architecture diagrams, key design iterations, or simple before/after metrics work well. Your slides should be aids that help illustrate your points, not crutches that carry the entire presentation.

Use themes to tell your story

Structure your presentation around themes, similar to what we discussed in [Telling Longer Stories](#). Organize 3–4 key challenges or achievements that demonstrate different signal areas; chronological is often not the most engaging format.

For example, instead of “Timeline of the Mobile App Redesign,” try:

- **Research and strategy:** how I identified user pain points and aligned stakeholders
- **Technical architecture:** the design system decisions that enabled scalable development
- **Cross-functional leadership:** coordinating engineering, design, and marketing through launch
- **Impact and iteration:** results and how we applied learnings to future projects

Emphasize Actions and inflection points

Remember that interviewers care more about what you did than what happened. The most compelling presentations feel like guided tours through your thinking process, where the audience understands not just what you accomplished, but how your specific actions and decisions drove those outcomes.

Hiring Manager Chats and Team Matching

A common part of most companies’ process is talking with the hiring manager, often outside of a formal behavioral interview. It might come early in the process, as a kind of introduction, or late, like more of a final fit evaluation. Some large companies with many open roles (like Google and

Meta) formalize this phase into something they call *Team Matching*, sending candidates through a series of meetings with hiring managers to ensure a final team assignment.

Here are some thoughts on preparing for these kinds of conversations:

Do your best to **understand the team and the manager** in advance. Is it an infrastructure team with an experienced manager, a product team with a manager new to the company, a new developer experience team, etc.?

Polish your response to “**Tell Me About Yourself**” and consider extending it a bit, expanding on your accomplishments and why you’re right for the team. This is less of an interview where the manager has a lot of questions to cover and more of a get-to-know-you session, so spending extra time is not a problem.

Be **prepared for questions** about how your experience aligns with the team.

Prepare questions for the manager. Understand what makes you successful and what a great team for you might be, then ask the manager about those aspects.

Part of a hiring manager’s role during these conversations is ensuring a social fit for you on the team. **Be likable and positive.** Check out the advice in [*Connecting Authentically*](#).

Specifically on Team Matching

If you’re advanced to the team match phase, the good news is that you’re very likely to be hired—you’ve cleared the highest hurdles. They will tell you this is not another interview and I suppose technically they are correct; however, if a manager cannot be found that will take you on their team, your candidacy will likely expire.

Here’s how you can prepare for the team match phase and run through the tape to the finish line:

Make sure your **resume presents your experience accurately** and quickly. Add a summary at the top that’s relevant to the role. Ask the recruiter to update your resume before it gets sent to the hiring managers.

Accept the first team that accepts you. I generally don’t advise making fear-based decisions or following greedy-algorithm-style life advice, but the implicit assumption behind having the team matching phase at all is that open headcount is scarce.

How recruiters are incentivized is not clear or consistent so **stay on top of them** during the team matching phase. As with everything else in life, advocate for yourself and stay in close contact with your recruiter.

[1] I acknowledge that you probably don't have much experience even hearing professional language. Hopefully the examples throughout this book help with that. [↑](#)

[2] See [\[⇒\]](#). [↑](#)

[3]
If you're jumping directly here after opening this book, I would recommend you review at least Part 1 to understand the foundation. [↑](#)

[4] Structured interviewing according to well-defined signal areas has been shown to reduce this kind of bias, see Levashina et al., 2014. [↑](#)
<https://thebehavioral.tech/p/structured-interview-bias-research>

[5]
I want to acknowledge that these stories may reinforce harmful stereotypes. But these do shape how interviewers pattern-match candidates with their vision of successful teams. Sometimes we need to understand the world we seek to change before we can change it.
I also focus on Silicon Valley's formative years, acknowledging that other tech hubs have their own stories. [↑](#)

[6] Fun fact: Fairchild Semiconductor also began in a garage. [↑](#)

[7] Listing these feels like a brainstorming session for a Bay Area Shakespeare—idealistic visions destined to clash with the complexities of scale and human nature. We all know these grand missions proved more complicated than first imagined. The tension between world-changing idealism and profit-driven reality creates ongoing challenges that each new generation of Builders must grapple with. [↑](#)

[8] For this section, I mean calls early in the process, not subsequent scheduling or negotiation calls. [↑](#)

[9] See [Tell Me About Yourself \[→\]](#). [↑](#)

[10] See [\[→\]](#). [↑](#)

[11] See [Managing the Human Element](#). [↑](#)

[12] See [Step 5: Prepare Questions for the Interviewer](#). [↑](#)

[13] See the section on Project Retrospectives and Deep Dives [\[→\]](#). [↑](#)

[14] There's a discussion of XFN interviews below [\[→\]](#). [↑](#)

[15] See [Telling Longer Stories](#). [↑](#)

[16] *Handling Follow Up Behavioral Interviews* [↑](#)
<https://thebehavioral.tech/p/handling-follow-up-interviews>

[17] If they don't tell you the reason, it's likely one of the "yours" reasons above—meaning something about your performance needs adjustment. [↑](#)

[18] See [\[→\]](#). [↑](#)

[19] Maybe this is acceptable if you are a designer. [↑](#)

A NOTE FROM THE AUTHOR

As we reach the end of this journey together, I want to share something I've learned from coaching hundreds of candidates: your stories are better than you think they are.

Every day, people just like you land roles at the world's most competitive tech companies. They don't have perfect stories, experience with immense scale, or flawless delivery. I certainly didn't. What they have is the clarity to see their own value and the confidence to communicate it authentically.

The frameworks and techniques in this book are important, but don't let them overshadow you as a person.

Have fun with it. I know that sounds counterintuitive when the stakes feel so high, but the best interviews I've ever witnessed felt more like engaging conversations between colleagues than formal assessments. Lean into that conversational tone. Connect with your interviewer as one human to another.

Your career journey—with all its ups and downs, pivots and breakthroughs—has prepared you for this moment in ways you might not even realize. Trust in that preparation. Trust in your stories. Trust in yourself.

For additional resources and ongoing support as you prepare for your behavioral interviews, visit <https://thebehavioral.tech>.

I'll be cheering you on.

Austen McDonald

Sunnyvale, California, 2025

APPENDIX: QUESTION REPOSITORY AND SCRIPTS FOR MOCK INTERVIEWS

Other than the Big Three questions [\[→\]](#), I don't think anyone can tell you with any confidence what you'll be asked in a behavioral interview. It's why I recommend not relying on "question practice" as your primary form of preparation.

However, there is value in seeing examples of questions, especially when ensuring you have coverage in your stories for all signal areas and when you want to practice a continuous interview.

Here are some scripts you can practice yourself or, better yet, give to a friend to ask you, as well as a list of common questions for each signal area. The scripts are both realistic and provide coverage across the signal areas.

Script #1

- Tell me about yourself. [Scope]
- Tell me about the largest scoped project you've ever built. [Scope]
- Tell me about a time when you had to solve a complex technical problem. [Scope]
- Tell me about a time you did something impactful that was not assigned to you. [Ownership]
- Share an example of when you had to adapt to a requirement that changed quickly. [Perseverance]
- Tell me a time when you disagreed with your manager or another leader. [Conflict Resolution]
- Tell me about a time when you failed. [Growth]
- Tell me about when you went above and beyond the requirements. [Perseverance]
- What questions do you have for me?

Script #2

- Tell me about yourself. [Scope]
- Why are you interested in this role? [Scope]
- Tell me about the project you're most proud of. [Scope]
- Share an example of when you saw an opportunity that others missed and took action on it. [Ownership]
- Tell me about a time when you had to get something from someone who didn't want to give it to you. [Conflict Resolution]
- Tell me about a time when you were faced with a problem with many possible solutions. How did you approach it? [Ambiguity]
- Tell me about a time when you were misunderstood. [Communication, Conflict Resolution]
- Tell me about a time when a project was behind and how you responded. [Perseverance]
- Tell me about a time when you learned something. [Growth]
- What questions do you have for me?

Script #3

- Tell me about yourself. [Scope]
- Tell me about the project where you've had the largest business impact. [Scope]
- Tell me about when you've demonstrated leadership. [Scope]
- Tell me about a time when you were involved in a conflict at work. [Conflict Resolution]
- Tell me about how you handled a project that was very ambiguous. [Ambiguity]
- Can you describe an instance where you proactively prepared for a potential issue before it became a problem? [Ownership]
- Tell me about a time when you balanced planning with rapid execution. [Perseverance, Ambiguity]
- How did you communicate something technical to someone non-technical? [Communication]
- Tell me about recent constructive feedback you've been given by your manager. [Growth]

- What questions do you have for me?

Scope

- What was your most impactful project?
- What's the hardest technical/product/organizational problem you've ever solved?
- Walk me through your career progression and how your responsibilities have evolved.
- Tell me about a time when you had to make a decision with significant business consequences.
- Describe a situation where you had to handle both technical and organizational complexity simultaneously.
- What's the most challenging architectural or system design decision you've made?
- How would you improve our product?
- How would you build [complex system] from scratch?

Ownership

- What's the biggest project you originated and drove yourself?
- What decision did you make that significantly influenced a project's direction?
- Give me an example of when you identified a problem that others hadn't noticed.
- Tell me about a time when you went above and beyond your assigned responsibilities.
- Describe a situation where you had to make a tough business decision and take responsibility for the outcome.
- Describe how you've measured and tracked the success of your initiatives.

Ambiguity

- When and how did you get started on something without a clear direction?
- When did you have to make a tough prioritization decision?
- Tell me about a project where the requirements kept changing throughout development.
- Describe a time when you had to make a decision with incomplete information.
- Tell me about a situation where you had to define success criteria for an ambiguous project.
- Give me an example of when you had to navigate conflicting stakeholder expectations.
- Describe a time when you had to pivot strategy mid-project due to changing conditions.
- Give me an example of when you had to structure an approach for a completely new type of problem.
- Describe a situation where you had to move forward despite uncertainty about the outcome.

Perseverance

- When was something delayed or blocked and how did you handle it?
- Describe a project that took much longer than expected and how you maintained momentum.
- Give me an example of when you had to overcome significant technical or organizational obstacles.
- Tell me about a time when you had to work through a particularly challenging debugging or problem-solving process.
- Describe a project where you had to persist despite lacking resources or support.
- Give me an example of when you had to overcome resistance from stakeholders or team members.
- Describe a situation where you had to maintain quality standards under pressure.

- Tell me about a time when you had to learn a difficult skill or technology to complete a project.
- When have you had to cancel a project and why?

Conflict Resolution

- Tell me about a time when you strongly disagreed with your manager or teammate.
- Describe a situation where you had to work with someone difficult and how you handled it.
- Describe a situation where you had to get something from someone who didn't want to give it to you.
- Tell me about a time when you had to deliver difficult feedback to a colleague.
- Give me an example of when you had to find common ground with someone who had opposing views.
- Describe a situation where you had to navigate a conflict between different departments or teams.
- Describe a situation where you had to manage up and influence your manager's decision.
- How would you handle a difficult coworker?
- What would you do if you disagreed with a technical decision?

Growth

- Tell me about the biggest mistake you've made and what happened next.
- Describe a time when you had to learn something completely new under pressure.
- Tell me about a time when you received difficult feedback and how you applied it.
- Describe a situation where you had to admit you were wrong and change course.
- Give me an example of when you actively sought out feedback to improve your performance.

- Describe a time when you had to develop a new skill to advance in your career.
- Tell me about a situation where you had to unlearn an old approach and adopt a new one.
- Describe a time when you mentored someone.
- Give me an example of when you had to overcome a significant weakness or blind spot.

Communication

- Tell me about a time when you had to convince others when the stakes were high.
- Describe how you've facilitated conversation among others.
- Tell me about a time when you had to explain something technical to a non-technical audience.
- Tell me about a time when you were misunderstood and/or misunderstood someone else.
- Describe a situation where you had to manage up effectively.
- Tell me about a time when you had to write documentation or communication that others could easily follow.
- Give me an example of when you had to adapt your communication style for different audiences.
- Tell me about a situation where you had to ensure alignment across multiple teams.
- How did you adapt your communication style to achieve a better outcome in a potentially negative situation?

Leadership – General

- Tell me about a time you drove change across multiple teams or the organization.
- Describe a situation where you had to influence without authority.
- How have you significantly mentored those around you who don't report to you?
- Tell me about a time you had to convince leadership to change direction on a major initiative.

- Describe how you've scaled yourself as your responsibilities grew.
- What is your leadership style?
- What's your approach to building and maintaining team culture?

Leadership – Cross-Functional

- Give me an example of when you had to coordinate a complex, multi-team initiative.
- Tell me about a conflict you resolved between your team and another team.
- What is your most/least effective partner relationship?
- Tell me about how you communicate with partner teams.
- How do you ensure your team's work aligns with broader company objectives?
- How do you approach building relationships with new cross-functional partners?

Leadership – People Management

- Tell me about a time when you encountered an under-performing report.
- Describe your approach to career development planning for your reports.
- How did you help a senior report grow?
- Tell me about how you conduct one-on-ones.
- What is your hiring philosophy?
- Tell me about a time you made a mistake hiring or delegating to someone.
- Tell me about a conflict you encountered between members of your team.
- Give me an example of when you had to restructure or reorganize your team.
- What is feedback you've been receiving from your reports?
- Tell me about a time you had to manage through a significant organizational change.

Values – General

- How do you approach technical debt?
- What's your philosophy on code reviews?
- How do you handle competing priorities?
- What's your approach to mentoring?
- What's your strategy for stakeholder management?
- What's your framework for prioritization?
- How do you balance short-term wins with long-term strategy?
- How do you think about giving and receiving feedback?
- What's your strategy for managing up?

Values – People Management

- What's your management philosophy?
- How do you approach one-on-ones?
- What's your philosophy on delegation?
- How do you think about career development for your reports?
- What's your approach to performance management?
- How do you think about team structure and organization?
- What's your philosophy on hiring?
- How do you approach team goal-setting?
- What's your framework for handling underperformers?
- How do you think about team motivation and retention?

Values – Product Management

- What's your philosophy on customer feedback?
- How do you think about product quality?
- What's your approach to MVP vs. fully-featured launches?
- How do you balance user needs with business goals?
- What's your framework for measuring product success?

Hypotheticals – General

- How would you be productive around a difficult coworker?
- How would you define success for a project?
- What would you do if you disagreed with the approach the company is taking?
- What would you do to convince a coworker of your position?
- How would you turn around a failing project?
- What would you do if a project was significantly behind schedule?
- How would you approach a problem with many possible solutions?
- Suppose requirements changed dramatically mid-project. How would you adapt?
- How would you approach learning a technology you've never used before?
- What would you do if you made a significant mistake that impacted users?

Hypotheticals – People Management

- How would you go about assigning roles to team members?
- How would you ensure your team is diverse and inclusive?
- How would you prioritize projects on a roadmap?
- What would you do if there was unhealthy competition between team members?
- How would you go about addressing a performance challenge?
- How would you go about keeping the team's skills current?
- How would you communicate bad news to leadership?
- Suppose you had to convince leadership to invest in something without immediate business value (like technical debt or tooling).
- How would you approach building a team from scratch with limited budget?
- How would you maintain team morale during a company reorganization?

Hypotheticals – Product Management

- How would you go about improving [company product]/[product you use yourself]?
- Suppose there were competing visions in the org on how to handle a project. What would you do?
- How would you balance flexibility and process on a team?
- How would you go about prioritizing a roadmap?
- How would you handle a situation where you need something from a resistant partner team?

Follow-ups

While somewhat trivial to list, these are so common they are hard to ignore:

- What would you do differently?
- What was the hardest [technical, etc.] part of that?
- How did you measure success?
- Did you encounter any conflicts during that project?
- What are the next steps with that project?

APPENDIX: QUESTIONS TO ASK THE INTERVIEWER

The questions you ask at the end of an interview are a critical opportunity to demonstrate preparation, strategic thinking, and genuine interest while gathering essential information to evaluate the role. See section [Step 5: Prepare Questions for the Interviewer](#) for why I recommend questions like these.

About the Team

- What is the focus of the team in the near term and why?
- How does planning work for project roadmaps?
- How does the team decide what to work on when there are competing priorities?
- What's the biggest technical debt the team is dealing with, and what's the plan for it?
- How has the team's focus evolved over the past year?
- What would need to happen for this team to be considered wildly successful this year?
- How does the team celebrate wins and handle failures?
- What's the typical onboarding process for new team members, and how long until they're fully productive?
- Can you give me an example of how the team handled a recent crisis or production issue?
- How does this team approach mentoring?

About the Role

- What led to this role being open?
- What does success look like in this role after 30, 60, and 90 days?
- What are the most immediate challenges the person in this role will tackle?
- Can you describe a recent project someone in this role led and its impact?

- What's the most challenging aspect of this role that might not be obvious from the job description?
- How much autonomy does this role have in choosing projects and technical approaches?
- What opportunities exist for this role to influence technical strategy or product direction?

About the Company

- How does this team's work connect to the company's broader mission and strategy?
- What major technical or product bets is the company making for the next 2–3 years?
- What are technical challenges the company is facing now?
- What keeps leadership up at night about the business?
- What aspect of the company culture do you think is most unique compared to other places you've worked?
- What's one thing about the company you'd change if you could?
- What types of people tend to thrive here, and what types struggle?
- How does the company measure and communicate progress toward its goals?
- What's the typical timeline from idea to production for new features?
- How does the company gather and incorporate customer feedback?

About the Manager

- How would your team describe your management style?
- Can you give me an example of how you've helped someone on your team grow recently?
- How do you prefer to give and receive feedback?
- What's the most valuable lesson you've learned as a manager here?
- How do you advocate for your team's needs with upper management?
- What are you most proud of that your team has accomplished?

- How do you handle situations where team members disagree with your decisions?

About Cross-Functional Partners

- How does this team typically collaborate with product management/design/other engineering teams?
- How are decisions made when multiple teams have competing priorities?
- What's the feedback loop like between product teams and customer-facing teams?

For Recruiters

- What specific signal areas or competencies will the behavioral interviewers be assessing?
- How have you seen successful candidates prepare?
- What are the most common issues the hiring teams has found with previous No Hire candidates?
- What can you tell me about each person I'll talk with during the process? What do they value?
- Are there particular aspects of my background the team is most interested in exploring?
- What scope of work is typically handled by the target level?
- What concerns do you have about my background and the fit for the role?

APPENDIX: BEHAVIORAL INTERVIEW FEEDBACK

Here's an example of what written interviewer feedback might look like as it makes its way to the hiring committee:

Hire, average confidence, at L4. Candidate demonstrated consistent L4-level scope and ownership across multiple projects, with strong evidence of business-focused thinking and collaborative approach to cross-functional work. Communication was clear and structured throughout the interview. Primary concern is around conflict resolution and growth. While aware of growth areas, lacks concrete examples of applying learnings from past mistakes. Technical depth and project management capabilities align well with L4 expectations.

Scope

- + Consistent feature-level scope across multiple projects, cycles of gathering requirements for ads team from PM and designer and executing
- + Managing 6-month timeline with dependencies across 3 teams (ads, payments, analytics)
- + Technical complexity appropriate for level (distributed caching, A/B testing infrastructure)

Ownership

- + Good focus on measuring business value via OKRs along the way
- + Proactively identified performance bottleneck that wasn't originally scoped

- + Drove end-to-end solution including post-launch monitoring and iteration

Communication

- + Using written documents for alignment with PM and design
- + Clear explanation of technical concepts during interview
- + Effective stakeholder management across multiple functions

Conflict Resolution

- Some concerns about taking the initiative to resolve what was clearly a launch-blocking conflict with design.
- /+ Seems to be aware of this and lists it as a learning, but doesn't have an example of applying this learning
- When pressed for follow-up example, candidate acknowledged avoiding difficult conversations

Growth

- + Honest about mistakes and extracted clear learnings
- = Demonstrated progression from junior to mid-level responsibilities over 2 years
- = Sought feedback from senior engineers during complex technical decisions

Ambiguity

- + Handled shifting requirements during ads optimization project
- = Made reasonable technical tradeoffs with incomplete information about traffic patterns

Favorite project

Built real-time analytics dashboard for ads team to track campaign performance, replacing manual spreadsheet-based reporting

6-month project, Q3-Q4 2023

Team of 4

High visibility with VP of Growth

Multiple dependencies: data platform team, ads API team, frontend platform

Structured project into 3 phases, by milestones

Created weekly stakeholder updates with metrics and blockers

Identified performance issue with real-time queries early, proposed caching solution

Worked with PM to prioritize features when timeline got tight

Built microservice using Node.js, Redis, Postgres

Integrated with 3 different data sources via REST APIs

Implemented A/B testing framework for dashboard features

Performance: reduced query times from 45 seconds to under 2 seconds

Deployed on schedule

Increased ads team productivity by 40% (estimated)

Feature adoption: 85% of ads team using dashboard daily within 2 weeks

Led to \$2M increase in campaign optimization efficiency (ads team estimate)

+ Scope: Managing multiple technical and organizational dependencies appropriately

+ Ownership: Proactive identification and solution of performance issues

+ Communication: Well-structured project updates, clear technical explanations

= Ambiguity: Adapted approach when timeline constraints emerged

- Conflict Resolution: When asked about disagreement with design team on UI approach, candidate acknowledged “probably should have pushed back harder” but waited for PM to resolve instead

What was the hardest technical part of this project?

...

APPENDIX: HOW TO CONDUCT BEHAVIORAL INTERVIEWS

Perhaps you're an executive at a startup tasked with establishing an interview process or you're a new manager about to evaluate your first candidate. Here are some thoughts about sitting on the other side of the table.

Who Should Conduct Behaviorals

If it's up to you to choose who will conduct behaviorals, you have a number of options to mix and match:

The hiring manager. If the directly supporting manager is known, it seems obvious they should conduct at least one behavioral round and weigh in on the hiring decision.

Cross-functional partners. If more time is available, the people the candidate will collaborate with regularly—PMs, designers, other engineering teams.

Executives. Often a senior leader is included, not only to obtain buy-in from the broader organization, but also because of their (presumed) extensive experience.

Avoid junior team members. Early-career team members lack experience evaluating people.

Scheduling & Time Allocation

Choosing when to have behaviorals. Of course you can place behaviorals anywhere in the process but often they appear toward the end since the more senior members of the team have limited schedules. If cultural fit is particularly important to you and you don't mind the tradeoffs, you can place them at the beginning as a filter.

Sufficient time allocation. Unless you count a quick screen by a recruiter or hiring manager, schedule 45–60 minutes. Rushed behavioral interviews produce poor signal and frustrate candidates who can't adequately tell their stories.

Consider the sequence. If you're doing multiple behavioral rounds (common for senior roles), ensure different interviewers focus on different signal areas to avoid redundancy.

Preparing for the Interview

Your role as a behavioral interviewer is about signal acquisition. You need to know what you're looking for and then gather enough evidence to confidently assess whether this candidate can succeed in the target role and organization.

Identify signal areas. Be crystal clear on what you think makes a hire successful in your organization. Choose some of the signal areas from Part 1 [\[1\]](#) and/or combine them with stated company values to arrive at a concise list of areas you'll assess.

Define good. Do the intellectually rigorous work of defining what “good-enough” means on each of your signal areas, for the appropriate target level. This will help you compare candidates and increase the confidence in your decisions. For example, what does “Good Communication” mean? Maybe it's something like, “Communicates proactively when the need arises, is effective at using face-to-face and written communication, explains their stories clearly and without extraneous detail.”

Understand the candidate. Review their background and any information you have available from references, etc. Form hypotheses about how the candidate will be successful in the role (e.g., worked in similar startups to ours). Also identify potential challenges the candidate may experience (e.g., changed jobs frequently, could be a performance or flight risk). Note these for exploration during the interview.

Prepare your questions. Prepare 8–12 questions that cover your identified signal areas and help you validate or invalidate your hypotheses about the candidate. “Tell me about a time...” questions are the most predictive in academic studies, but asking about values or hypotheticals are also instructive.

[\[2\]](#)

Conducting the Interview

Balance the candidate's comfort with efficiency. Ideally the interview is a conversation, albeit one that you are directing. A conversational tone will

help the candidate communicate effectively, giving you a clearer picture of their abilities. It also serves to leave a positive impression of your company in case of competing offers. However, never sacrifice signal for comfort—that is not helping the candidate.

Interrupt when you're not getting valuable signal. If they're spending too much time on context or going off on tangents, gently redirect: "I'm going to stop you there. That's helpful context. What did you do next?"

Read between the lines and ask follow-up questions. If something doesn't sound right, it probably isn't. Probe further: "Help me understand how you made that decision" or "What was so challenging about that situation?" Candidates may be evasive about their level of involvement or pass off work as being more complex than it really was.

Here are some common follow-up questions:

[\[3\]](#)

- *What would you do differently?* Assesses learning and growth mindset.
- *How did you measure success?* Reveals data-driven thinking and results orientation.
- *What was the hardest part?* Assesses scope and uncovers the candidate's real contribution vs. team accomplishments.

Take notes throughout. Write down enough detail that you could explain their response to someone else later. Note both positive and concerning signals.

Record their questions. What they ask reveals their priorities and judgment.

Common Pitfalls

Limited coverage. Don't let the entire interview focus on one project. If the candidate is providing negative signal in one area, try to confirm it with a follow-up or an adjacent question. For example, if their conflict story revealed a tendency to avoid confrontation, look for that trait in a question about working on a project with many partners.

Letting your hypotheses become biases. Actively test any beliefs you have about the candidate to ensure those hypotheses aren't unfounded

biases. It's healthy to develop a model of the candidate in your mind before you meet them, based on their experience and profile, but your role during the interview is to evaluate their responses. If those contradict your hypotheses you need to let those beliefs go.

Overweighting likability or similarity to yourself/the existing team.

You're not there to test if you'll be great friends with the candidate; you're there to predict their work performance. Fit with the team is important but bringing new ideas and backgrounds to the team are also important.

Ignoring red flags. If someone struggles to give concrete examples of basic collaboration or takes credit for others' work, that's signal too.

After the Interview

Write your assessment right away. Memory fades quickly, and later interviews can color your recollection.

Rate performance by signal area. Use whatever scale your company prefers, but be specific about why you gave each rating.

Include specific behavioral evidence. When you make conclusions, provide evidence from the conversation. For example, "Strong conflict resolution—described methodically gathering data to resolve architecture disagreement, followed up privately with dissenting engineer to maintain relationship."

Note any concerns or gaps. What didn't you get to assess? What felt incomplete or unconvincing?

Look through the other interview feedback for confirmation. Whether the debrief conversation with other interviewers is done in person or in written form, look for examples or counterexamples to discuss that relate to your findings. If someone was disorganized in communication with you, were they the same in other interview types?

APPENDIX: LEVELING AT TECH COMPANIES

[\[4\]](#)

To help you pick stories and align your language to the expectations of interviewers, I've provided a reference for Big Tech company leveling. These examples are from the perspective of engineers, but other functions have analogs at the same levels.

L3: Junior Engineer / SWE I

- Works at the task level: team lead or manager provides a task, L3 does it and returns for the next task
- Tasks typically 1–5 days long
- Expected to need technical, organization, and operational guidance frequently
- Flags disagreements but probably needs outside help to resolve them

L4: Mid-level Engineer / SWE II

- Works at the feature level: breaks a feature down into tasks, completes the feature then returns for the next one
- Features typically 1–2 weeks long
- Makes a feature plan and communicates that to the lead
- Operates mostly independently, with some oversight from senior engineers, especially on technology or architecture choices
- Resolves on-team conflicts themselves (questions over implementation details, feature scope, etc.)
- Often does some light people mentoring, like being an onboarding buddy

L5: Senior Engineer

- Works at the project level: projects have many features, which then have many tasks
- Identifies projects and works often with one or more L3–4s to delegate features/tasks

- Beginning with vague project direction, creates a comprehensive plan, disambiguating as needed and involving all required XFN and anyone required on another team
- Smoothly resolves conflicts with XFN and team members to keep projects moving forward
- Projects typically 1–3 months long
- Should consistently be able to transform L3s into L4s with coaching

L6: Staff Engineer

- Works at the goal level: identifies goals for the team, curates projects to achieve those goals, and delegates those projects to other engineers, coaching them to success
- Typically does the most technically and organizationally complex tasks, taking on the most risk
- Responsible for long-term health of the team (including organizational positioning and technology choice); partners with manager on people health (hiring, firing, coaching, delegating)
- Resolves conflicts that arise between teams
- Can operate ~6 months independently with the team being successful; can fill in for the manager on team direction
- Equivalent to the M1, line-manager level
- Some product teams have no engineers at L6 at all; a typical team might have 1–3

M1: Engineering Manager

- Similar to the L6, working at the goal level, supports a team of 6–12 people
- Responsible for output of the team, which includes product outcomes, technical health, and people growth
- Equal partner with the product manager to plan and execute the team's work
- Typically planning on the 12-month scale

L7: Principal Engineer

- Operating at the org level: an org has many teams, with many goals, with many projects, etc.
- Typically reports to the org leader instead of a line manager
- Oversees long-term technical success of the org (technology choice, technical debt, product direction, etc.)
- Resolves conflicts across the company or between companies

ACKNOWLEDGMENTS

My wife, Kokeb McDonald, for showing me the way to write, publish, and market books successfully. It's been an honor to be on this journey, through all things in life, together.

Henry Cheung, for the many heartfelt and mentally challenging conversations about this book and many other topics, and for helping me keep the connection to what's really important.

Stefan Mai and John Crickett for the advice, encouragement, and for the break I needed prove the value in this content.

To the many mentees in my coaching practice: all of the insights in this book come from the wrestling you do in your journey for the roles that matter. Your perseverance, discipline, and vulnerability are inspiring.

To the many others who reviewed early drafts and provided critical feedback:

Mike Mroczka	Jacqueline Huang
Venkata Chintapalli	Zack Mariscal
Eric Kao	Sohan Patil
Aparna Girme	Joe Woodhouse
Prasanth Parameswaran	Akash Gupta
KK Molugu	Ashokkumaar P Loganathan
Syed Shuttari	Alex Lau
Felix Simon	Ronnie
Diane Chang	Vaibhav Singh
Aditya Balusu	Clément Genzmer
Nathan Spaun	Ian Christopher
Kamlesh Makhija	Dima Berezin
Jake Quist	Jade Wilson
Jason Yip	Richard Banyi
Chia Zhe Min	Beshoy
Antonina Kononova	Amol Saxena
Odelya Holiday	

ABOUT THE AUTHOR

Austen McDonald is a seasoned and empathetic technology leader and former Senior Software Engineering Manager at Facebook/Meta. During his over 9 years there, he served as Hiring Committee Chair for the iOS and Android recruiting pipelines, overseeing hiring decisions, defining leveling standards, and mentoring interviewers. With firsthand experience conducting over 1,000 interviews for major tech companies, Austen has also coached more than 200 engineers to succeed in their Big Tech interview journeys. His career spans both individual contributor and leadership roles across Big Tech and startups, offering a unique perspective on hiring.

Austen earned a degree in Computer Science from Georgia Tech and a PhD from Stanford University, where his passion for teaching and mentorship took root. He draws inspiration from his wife, Kokeb, a best-selling author and occupational therapist, and finds joy in their life together with their two children (and Golden Retriever Willow) in the Bay Area, California.



[2] See [*How Behavioral Interviews Work*](#). [↑]

[3] See [\[→\]](#). [↑]

[4] For comparing levels at tech companies, including salary estimates, check levels.fyi [↑].

<https://thebehavioral.tech/p/levels-fyi>